



Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática

Trabajo Fin de Grado
Grado en Ingeniería Informática
Tecnología Específica de Ingeniería de Computadores

Diseño y evaluación de un servicio inteligente de predicción de información de red para sistemas de Edge Computing

Luis Melero Jareño

Junio, 2026

Trabajo Fin de Grado

Grado en Ingeniería Informática

Tecnología Específica de Ingeniería de Computadores

Diseño y evaluación de un servicio inteligente de predicción de información de red para sistemas de Edge Computing

Autor: Luis Melero Jareño

Tutor: Gabriel Cebrián Márquez

Co-Tutor: Estefanía Coronado Calero

Junio, 2026

*Dedicado a mis padres, a mi
novia y a todos aquellos que han
estado a mi lado estos años.*

Resumen

La creciente complejidad de las redes 5G y los sistemas de *Edge Computing* plantea importantes desafíos para mantener niveles adecuados de calidad de servicio ante demandas de usuario altamente variables. En este contexto, anticipar el comportamiento futuro de la red resulta fundamental para evolucionar de una gestión reactiva a una gestión proactiva de los recursos. Este Trabajo de Fin de Grado aborda el diseño y la evaluación de un servicio inteligente de predicción de indicadores clave de rendimiento (*Key Performance Indicators*, KPIs) de red basado en técnicas de *Machine Learning* (ML), empleando como entorno experimental el simulador MintEDGE.

Para ello, se han generado simulaciones en múltiples ciudades con diversas configuraciones de red y operador, construyendo un conjunto de datos homogéneo de aproximadamente $1,16 \times 10^6$ registros con más de 360 variables. Sobre este conjunto se han entrenado y evaluado seis arquitecturas de modelos, concretamente *DecisionTree*, *RandomForest*, *eXtreme Gradient Boosting* (XGBoost), *Light Gradient Boosting Machine* (LightGBM), *Long Short-Term Memory* (LSTM) y *Convolutional Neural Network* (CNN), con múltiples configuraciones de selección de variables e hiperparámetros y un sistema de puntuación ponderada denominado **score**, que equilibra precisión y eficiencia computacional.

Los resultados demuestran que los modelos tabulares basados en *gradient boosting* son los óptimos para los cuatro tipos de servicios evaluados, alcanzando valores de coeficiente de determinación (R^2) superiores a 0.99 en servicios de alta predictibilidad y de 0.76 a 0.83 en servicios con mayor variabilidad, sin evidencia de sobreajuste y con tiempos de inferencia inferiores a $1 \mu s$ por muestra, compatibles con la operación en tiempo real de un orquestador de *Edge Computing*.

Índice general

Capítulo 1	Introducción	1
1.1	Contextualización	1
1.2	Motivación	2
1.3	Objetivos	3
1.3.1	Objetivo principal	3
1.3.2	Objetivos secundarios	3
1.4	Competencias	4
1.5	Planificación temporal	5
1.6	Uso de herramientas de inteligencia artificial	6
1.7	Estructura del proyecto	6
Capítulo 2	Estado del arte	9
2.1	Redes 5G	9
2.1.1	Arquitectura 5G	10
2.1.2	Despliegue del 5G: Arquitecturas Non-Standalone y Standalone	11
2.1.3	Virtualización y softwarización en el sistema 5G	13
2.2	Edge Computing	13
2.2.1	MEC en redes móviles	15
2.3	Principales parámetros de rendimiento en redes 5G y Edge Computing	16
2.3.1	Parámetros de tráfico y demanda	16
2.3.2	Parámetros de latencia	17
2.3.3	Parámetros de congestión y QoS	18
2.3.4	Parámetros de utilización de recursos de infraestructura	18
2.4	Simulación y emulación	19

2.5	Modelos de predicción aplicados a series temporales	21
2.5.1	Tipos de modelos predictivos	21
2.5.2	Métodos de selección de variables	25
2.5.3	Métricas de evaluación	27
Capítulo 3	MintEDGE	29
3.1	Descripción general del simulador	29
3.2	Integración de herramientas externas	30
3.3	Arquitectura y componentes principales	31
3.4	Configuración de escenarios y ejecuciones	34
3.4.1	Configuración de escenarios geográficos	34
3.4.2	Modelado de tráfico y movilidad de usuarios	35
3.4.3	Operación del orquestador y caracterización de servicios	35
3.5	Resultados generados y métricas obtenidas	36
Capítulo 4	Metodología y desarrollo	37
4.1	Introducción y metodología general	39
4.2	Construcción del conjunto de datos homogéneo	40
4.2.1	Datos de partida generados por MintEDGE	40
4.2.2	Homogeneización de escenarios y ciudades	40
4.2.3	Ingeniería de características	42
4.2.4	Alineación temporal de métricas de QoS	44
4.3	Preparación del problema predictivo	46
4.3.1	Definición de variables objetivo	46
4.3.2	División del conjunto de datos	48
4.3.3	Selección de variables	49
4.4	Modelos predictivos y configuración experimental	50
4.4.1	Modelos tabulares	51
4.4.2	Modelos secuenciales	51
4.5	Organización de experimentos y criterio de selección	53
Capítulo 5	Evaluación experimental del servicio predictivo	55
5.1	Introducción	55
5.2	Evaluación de modelos tabulares	56

5.2.1	Comparativa por variable objetivo	56
5.2.2	Impacto del ajuste de hiperparámetros	58
5.2.3	Selección del mejor modelo tabular y análisis de variables	59
5.3	Evaluación de modelos secuenciales	66
5.3.1	Comparativa por variable objetivo	66
5.3.2	Análisis por arquitectura (LSTM frente a CNN)	68
5.4	Comparación global de modelos	73
5.5	Evaluación final sobre el conjunto de prueba y discusión de resultados	76
5.5.1	Resultados sobre el conjunto de prueba	76
5.5.2	Comparativa entre validación y prueba	78
5.5.3	Análisis y discusión	81
Capítulo 6	Conclusiones y trabajo futuro	83
6.1	Conclusiones	83
6.2	Trabajo futuro	84
Bibliografía		87

Índice de figuras

Figura 1.1. Planificación temporal del desarrollo del Trabajo de Fin de Grado.	5
Figura 2.1. Arquitectura basada en servicios del 5GS y principales funciones de red. Fuente: 3GPP TS 23.501 [1].	10
Figura 2.2. Comparativa entre las arquitecturas de despliegue NSA y SA del sistema 5G. Fuente: Vochin et al. [11].	12
Figura 2.3. Arquitectura conceptual del <i>Edge Computing</i> organizada en distintos niveles. Fuente: Instituto Nacional de Ciberseguridad (INCIBE) [15].	14
Figura 2.4. Arquitectura del sistema MEC e integración con el núcleo de red 5G. Fuente: ETSI [16].	15
Figura 2.5. Arquitectura de una red neuronal con múltiples capas ocultas. Fuente: IBM [43].	24
Figura 2.6. Arquitectura de una celda LSTM con sus tres puertas de control (<i>forget gate, input gate y output gate</i>). Adaptado de [46].	25
Figura 3.1. Red viaria de la ciudad de Maastricht generada por SUMO a partir de datos de OpenStreetMap. Fuente: elaboración propia.	31
Figura 3.2. Arquitectura de MintEDGE: módulos internos, relaciones y herramientas externas. Fuente: elaboración propia a partir del análisis del código fuente de MintEDGE [27].	32
Figura 3.3. Flujo de ejecución del ciclo de orquestación de MintEDGE. Fuente: elaboración propia a partir del análisis del código fuente de MintEDGE [27].	33
Figura 4.1. Flujo metodológico del servicio predictivo desarrollado. Fuente: elaboración propia.	38
Figura 4.2. Homogeneización del conjunto de datos mediante agregación estadística por estación base. Fuente: elaboración propia a partir de los datos generados por MintEDGE [27].	41
Figura 4.3. Desfase temporal en el registro de métricas de QoS en MintEDGE y corrección aplicada. Fuente: elaboración propia a partir del análisis del código de MintEDGE [27].	45
Figura 4.4. Esquema temporal de construcción de características y variable objetivo, donde las variables de entrada utilizan información del instante actual o anterior y la variable objetivo corresponde a un instante futuro. Fuente: elaboración propia.	47
Figura 4.5. Distribución de las cuatro variables objetivo consideradas en el estudio. Fuente: elaboración propia.	47

Figura 4.6. Esquema de división temporal rotativa por ciudad para modelos secuenciales. Fuente: elaboración propia.	49
Figura 4.7. Diferencia en la representación de la entrada entre modelos tabulares y secuenciales. Fuente: elaboración propia.	50
Figura 5.1. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y score para la variable objetivo global . Fuente: elaboración propia.	60
Figura 5.2. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y score para la variable objetivo connected_vehicles . Fuente: elaboración propia.	60
Figura 5.3. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y score para la variable objetivo augmented_reality . Fuente: elaboración propia.	61
Figura 5.4. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y score para la variable objetivo video_analysis . Fuente: elaboración propia.	61
Figura 5.5. Distribución de residuos y diagrama de dispersión para la variable objetivo global . Fuente: elaboración propia.	62
Figura 5.6. Distribución de residuos y diagrama de dispersión para la variable objetivo connected_vehicles . Fuente: elaboración propia.	62
Figura 5.7. Distribución de residuos y diagrama de dispersión para la variable objetivo augmented_reality . Fuente: elaboración propia.	63
Figura 5.8. Distribución de residuos y diagrama de dispersión para la variable objetivo video_analysis . Fuente: elaboración propia.	63
Figura 5.9. Importancia de variables del modelo LightGBM ganador para global . Fuente: elaboración propia.	64
Figura 5.10. Importancia de variables del modelo LightGBM ganador para connected_vehicles . Fuente: elaboración propia.	64
Figura 5.11. Importancia de variables del modelo XGBoost ganador para augmented_reality . Fuente: elaboración propia.	65
Figura 5.12. Importancia de variables del modelo XGBoost ganador para video_analysis . Fuente: elaboración propia.	65
Figura 5.13. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por score para la variable objetivo global . Fuente: elaboración propia.	69
Figura 5.14. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por score para la variable objetivo connected_vehicles . Fuente: elaboración propia.	69
Figura 5.15. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por score para la variable objetivo augmented_reality . Fuente: elaboración propia.	70

Figura 5.16. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por score para la variable objetivo video_analysis . Fuente: elaboración propia.	71
Figura 5.17. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en global . Fuente: elaboración propia.	72
Figura 5.18. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en connected_vehicles . Fuente: elaboración propia.	72
Figura 5.19. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en augmented_reality . Fuente: elaboración propia.	73
Figura 5.20. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en video_analysis . Fuente: elaboración propia.	73
Figura 5.21. Comparativa entre LightGBM y CNN filters=256 por R^2 , MAE y tiempo de inferencia para global . Fuente: elaboración propia.	74
Figura 5.22. Comparativa entre LightGBM y CNN filters=32 por R^2 , MAE y tiempo de inferencia para connected_vehicles . Fuente: elaboración propia.	74
Figura 5.23. Comparativa entre XGBoost y CNN filters=32 por R^2 , MAE y tiempo de inferencia para augmented_reality . Fuente: elaboración propia.	75
Figura 5.24. Comparativa entre XGBoost y CNN filters=32 por R^2 , MAE y tiempo de inferencia para video_analysis . Fuente: elaboración propia.	75
Figura 5.25. Predicción frente al valor real sobre el conjunto de prueba para global . Fuente: elaboración propia.	77
Figura 5.26. Predicción frente al valor real sobre el conjunto de prueba para connected_vehicles . Fuente: elaboración propia.	77
Figura 5.27. Predicción frente al valor real sobre el conjunto de prueba para augmented_reality . Fuente: elaboración propia.	78
Figura 5.28. Predicción frente al valor real sobre el conjunto de prueba para video_analysis . Fuente: elaboración propia.	78
Figura 5.29. Predicciones sobre validación y prueba superpuestas para global . Fuente: elaboración propia.	78
Figura 5.30. Predicciones sobre validación y prueba superpuestas para connected_vehicles . Fuente: elaboración propia.	79
Figura 5.31. Predicciones sobre validación y prueba superpuestas para augmented_reality . Fuente: elaboración propia.	79
Figura 5.32. Predicciones sobre validación y prueba superpuestas para video_analysis . Fuente: elaboración propia.	79
Figura 5.33. Análisis de residuos comparativo entre validación y prueba para global . Fuente: elaboración propia.	80

Figura 5.34. Análisis de residuos comparativo entre validación y prueba para connected_vehicles .	
Fuente: elaboración propia.....	80
Figura 5.35. Análisis de residuos comparativo entre validación y prueba para augmented_reality .	
Fuente: elaboración propia.....	80
Figura 5.36. Análisis de residuos comparativo entre validación y prueba para video_analysis .	
Fuente: elaboración propia.....	81
Figura 5.37. Modelos tabulares finales generados.....	82

Índice de tablas

Tabla 2.1. Principales categorías de parámetros de rendimiento en entornos 5G y <i>Edge Computing</i> . .19	
Tabla 3.1. Principales métricas generadas por MintEDGE.36	
Tabla 5.1. Mejor configuración base por modelo y variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.57	
Tabla 5.2. Impacto de hiperparámetros en modelos de gradiente sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.58	
Tabla 5.3. Clasificación de modelos tabulares por score para cada variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.59	
Tabla 5.4. Comparativa del mejor CNN frente al mejor LSTM por variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.66	
Tabla 5.5. Comparativa del mejor modelo tabular frente al mejor modelo secuencial sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.74	
Tabla 5.6. Resultados de los modelos ganadores sobre el conjunto de prueba frente a los obtenidos sobre validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.76	

Capítulo 1

Introducción

En este primer capítulo se presenta el contexto general del trabajo, así como la motivación que justifica el desarrollo de un servicio inteligente de predicción de información de red en sistemas de *Edge Computing*. Además, se establecen los objetivos principales y secundarios del trabajo, se explican las competencias desarrolladas y finalmente se describe la estructura general del trabajo.

1.1 Contextualización

Las redes de comunicación móvil han evolucionado progresivamente hasta la actual tecnología 5G [1], incorporando mejoras en capacidad, velocidad de transmisión, latencia y calidad de servicio (*Quality of Service*, QoS). Estas características permiten dar soporte a nuevos escenarios de uso que requieren altas tasas de datos, baja latencia y conectividad masiva, como aplicaciones de realidad aumentada, sistemas de transporte inteligentes o servicios industriales basados en comunicaciones en tiempo real [2].

Ante estos nuevos requisitos de latencia, capacidad de procesamiento y volumen de datos generado, surge el paradigma de *Edge Computing*, que propone acercar las capacidades de computación y almacenamiento a los puntos más próximos al usuario o a la fuente de generación de datos [3] [4]. Este enfoque permite reducir la latencia, disminuir la carga sobre la red troncal y mejorar la eficiencia en el uso de los recursos disponibles.

El paradigma de *Edge Computing* engloba cualquier infraestructura que permita situar la capacidad de cómputo en el borde, independientemente de la tecnología de red. Una de las implementaciones más relevantes de este paradigma es la arquitectura *Multi-access Edge Computing* (MEC). Promovida por *European Telecommunications Standards Institute* (ETSI), MEC representa una solución concreta para ejecutar aplicaciones y procesar información directamente en la red de acceso, ofreciendo a los operadores un entorno de ejecución distribuido. Su integración con redes 5G resulta especialmente útil para aplicaciones sensibles a la latencia o con grandes volúmenes de datos, ya que facilita una provisión de servicios más cercana al usuario y con mejores niveles de QoS [5] [6].

1.2 Motivación

En redes 5G y sistemas de *Edge Computing*, la gestión eficiente de recursos constituye un reto crítico. La reducción de la latencia, el aumento de la capacidad y la necesidad de soportar un número creciente de dispositivos conectados [1] [2] hacen que pequeñas variaciones en el rendimiento de la red puedan degradar significativamente la QoS de aplicaciones sensibles al retardo [7]. Por ello, en entornos donde la demanda de los usuarios varía de forma rápida e impredecible, resulta necesario disponer de mecanismos capaces de gestionar de forma eficiente los recursos de red y computación.

La gestión de recursos en redes móviles y sistemas de *Edge Computing* se ha basado tradicionalmente en enfoques reactivos, que actúan una vez que se ha producido una congestión o una degradación del servicio. Sin embargo, este tipo de estrategias presentan importantes limitaciones en la dinámica de las redes de acceso actuales, caracterizadas por una alta movilidad de los usuarios y requisitos heterogéneos y variables de las aplicaciones. En este contexto, la transición desde una gestión reactiva hacia una gestión proactiva se presenta como una necesidad para la actual y próxima generación de redes [8].

La posibilidad de anticipar el comportamiento de indicadores de rendimiento de red, como la carga de los servidores, el ancho de banda disponible o la latencia de los enlaces, mediante el uso de modelos predictivos, permite adoptar un enfoque proactivo en la orquestación de recursos. De este modo, es posible optimizar la asignación de capacidades antes de que se produzcan situaciones críticas, reducir la probabilidad de congestión y garantizar niveles constantes de QoS [8].

Por todo ello, este Trabajo Fin de Grado surge de la necesidad de avanzar hacia sistemas de *Edge Computing* más inteligentes y adaptativos, capaces de anticiparse a las variaciones de la demanda mediante el uso de técnicas de *Machine Learning* (ML). A través de un entorno de simulación controlado, este proyecto aborda el diseño, implementación y evaluación de un servicio inteligente de predicción de indicadores clave de rendimiento (*Key Performance Indicators*, KPIs) de red, demostrando la viabilidad de anticipar el comportamiento del sistema de forma precisa, robusta y compatible con las restricciones de tiempo real de la computación en *Edge Computing* y explorando su potencial para la optimización de estos sistemas y redes de próxima generación.

1.3 Objetivos

En esta sección se definen los objetivos que se persiguen a lo largo del presente Trabajo de Fin de Grado. Estos se estructuran en un objetivo principal, que resume la finalidad global del trabajo, y una serie de objetivos secundarios que descomponen dicho propósito en tareas concretas y alcanzables, necesarias para su correcta consecución.

1.3.1 Objetivo principal

El objetivo principal de este Trabajo de Fin de Grado es el diseño, desarrollo y evaluación de un servicio inteligente capaz de predecir determinados parámetros de QoS de red en sistemas de *Edge Computing*, en el contexto de redes 5G, con el propósito de mejorar la gestión y asignación de recursos de forma proactiva.

La finalidad de este servicio es anticipar el comportamiento futuro de la tasa de satisfacción de peticiones aceptadas, permitiendo apoyar los procesos de orquestación y toma de decisiones, facilitar la adaptación dinámica frente a variaciones de la demanda y analizar su viabilidad como herramienta de apoyo a la orquestación de recursos en sistemas de *Edge Computing*.

1.3.2 Objetivos secundarios

El desarrollo de este Trabajo de Fin de Grado requiere la consecución de una serie de objetivos secundarios que permiten estructurar y orientar el proceso de investigación y desarrollo. Estos objetivos secundarios abordan distintos aspectos necesarios para

alcanzar el propósito global del proyecto, proporcionando tanto el marco conceptual como el entorno de trabajo y los mecanismos de evaluación adecuados.

1. Análisis del estado del arte

Realizar una revisión de los conceptos fundamentales relacionados con redes 5G y *Edge Computing*, los principales parámetros de rendimiento utilizados en la evaluación de redes de comunicaciones, así como de las técnicas de ML empleadas en el análisis de sistemas de red.

2. Análisis del simulador MintEDGE

Estudiar la arquitectura, funcionamiento y capacidades del simulador MintEDGE, utilizado como base de este Trabajo Fin de Grado, mediante el análisis de sus módulos internos, la revisión de su código fuente y la exploración de los datos generados en las simulaciones.

3. Desarrollo del servicio inteligente de predicción

Diseñar e implementar un servicio capaz de predecir determinados indicadores de rendimiento de red a partir de la información generada en el entorno simulado, con el objetivo de anticipar el comportamiento del sistema.

4. Evaluación experimental del servicio propuesto

Evaluar el rendimiento del servicio predictivo desarrollado mediante experimentación sistemática sobre datos de simulación, analizando el comportamiento de distintas arquitecturas de modelos y configuraciones a través de métricas de precisión y eficiencia computacional para cada variable objetivo.

1.4 Competencias

En este Trabajo de Fin de Grado se desarrollan y aplican diversas competencias específicas relacionadas con la intensificación de Ingeniería de Computadores. A continuación, se describen las principales competencias abordadas y su relación con el desarrollo del trabajo:

[IC4] **Capacidad de diseñar e implementar software de sistema y de comunicaciones.** Se utilizará el simulador MintEDGE y se diseñará un servicio inteligente de predicción, lo que permitirá trabajar en el desarrollo e integración de soluciones software aplicadas a sistemas de comunicaciones.

[IC7] **Capacidad para analizar, evaluar, seleccionar y configurar plataformas hardware para el desarrollo y ejecución de aplicaciones y servicios informáticos.** Se estudiarán arquitecturas de redes 5G y sistemas de *Edge Computing*, analizando cómo influyen los diferentes recursos del sistema en el rendimiento de la red y en la ejecución de servicios inteligentes.

[IC8] **Capacidad para diseñar, desplegar, administrar y gestionar redes de computadores.** Se analizará el funcionamiento de redes 5G habilitadas con *Edge Computing*, lo que permitirá comprender los mecanismos necesarios para mantener niveles adecuados de QoS en entornos dinámicos.

1.5 Planificación temporal

El desarrollo de este Trabajo de Fin de Grado se ha llevado a cabo entre febrero y junio de 2026. Las primeras fases se han dedicado al planteamiento del problema, al estudio del simulador MintEDGE y a la revisión del estado del arte. A continuación, se ha abordado el núcleo experimental del trabajo, comprendiendo la ejecución de simulaciones, la construcción del conjunto de datos, el entrenamiento y evaluación de modelos y el desarrollo del servicio predictivo final. La redacción de la memoria se ha realizado de forma paralela a lo largo de todo el proceso. La **Figura 1.1** muestra la distribución temporal de estas fases.



Figura 1.1. Planificación temporal del desarrollo del Trabajo de Fin de Grado.

1.6 Uso de herramientas de inteligencia artificial

Durante la elaboración de este trabajo se han empleado los asistentes conversacionales Claude (Anthropic) y ChatGPT (OpenAI) como herramientas de apoyo. Su uso ha abarcado la consulta de dudas técnicas relacionadas con la implementación y el funcionamiento de modelos predictivos, un ámbito que no había sido cubierto en profundidad durante la formación académica recibida en la rama de Ingeniería de Computadores, así como la búsqueda y verificación de fuentes bibliográficas fiables y la mejora de la expresión y redacción de determinadas secciones de la memoria. En todos los casos, las herramientas se han utilizado como soporte al proceso de trabajo del autor, quien ha tomado todas las decisiones de diseño, ha desarrollado el código fuente, ha ejecutado los experimentos y ha interpretado los resultados de forma autónoma. Todo el contenido ha sido revisado y validado por el autor, que asume la plena responsabilidad sobre la originalidad, exactitud y autoría del trabajo.

1.7 Estructura del proyecto

La memoria de este Trabajo de Fin de Grado se organiza en los siguientes capítulos:

- **Capítulo 1: Introducción.** Se presenta el contexto general en el que se desarrolla el trabajo, introduciendo la evolución de las redes de comunicaciones móviles y del paradigma *Edge Computing*. Asimismo, se describe la motivación del proyecto, los objetivos planteados, las competencias desarrolladas y la organización de la memoria.
- **Capítulo 2: Estado del Arte.** Se describen los fundamentos teóricos y técnicos necesarios para contextualizar el trabajo. En este capítulo se analizarán las principales características de las redes 5G y de los sistemas de *Edge Computing*. Además, se estudiarán los principales parámetros de rendimiento, así como los enfoques de simulación y emulación aplicados a entornos de red.
- **Capítulo 3: MintEDGE.** Se describe el simulador empleado como entorno experimental para el desarrollo del trabajo, analizando su arquitectura, sus principales módulos y su funcionamiento general. Además, se analizan los parámetros de configuración y distintos escenarios de simulación a considerar.

- Capítulo 4: **Metodología y desarrollo.** Se presenta la metodología seguida para el diseño e implementación del servicio inteligente de predicción explicando las fases de desarrollo del programa.
- Capítulo 5: **Evaluación experimental del servicio predictivo.** Se presentan los resultados experimentales. Se evalúan y comparan los modelos tabulares y secuenciales sobre validación, se selecciona el modelo ganador para cada variable objetivo y se realiza la evaluación definitiva sobre el conjunto de prueba.
- Capítulo 6: **Conclusiones y trabajo futuro.** Se presentan las principales conclusiones del trabajo, resaltando las aportaciones realizadas y las limitaciones identificadas. Asimismo, se proponen posibles líneas de mejora y futuras ampliaciones del servicio desarrollado.

Capítulo 2

Estado del arte

En este segundo capítulo se presenta el estado del arte en relación con las tecnologías y conceptos teóricos y técnicos que enmarcan el desarrollo de este Trabajo de Fin de Grado. Se revisan las principales características de las redes 5G, así como su relación con las diferentes arquitecturas de computación de *Edge Computing*. También se describen los principales parámetros de rendimiento, los enfoques de simulación empleados para su estudio y el concepto de modelos de predicción con un enfoque específico relacionado con el comportamiento de la red.

2.1 Redes 5G

La quinta generación de sistemas de comunicaciones móviles, conocida como 5G o sistema 5G (*5G System*, 5GS), estandarizada por *3rd Generation Partnership Project* (3GPP) a partir de la *Release 15*, la cual fue congelada funcionalmente en junio de 2018 y completamente especificada en septiembre de 2019, redefine la arquitectura de red completamente con respecto a generaciones anteriores incluyendo los protocolos, interfaces de red y mecanismos necesarios para el control de sesiones, la gestión de la movilidad y el aprovisionamiento de servicios [1].

Entre las mejoras introducidas en 5G para soportar las crecientes demandas en latencia y rendimiento destaca la definición de tres tipos de servicios heterogéneos. Estos servicios incluyen *Enhanced Mobile Broadband* (eMBB), orientado a proporcionar

mayores velocidades de transmisión de datos; *Ultra Reliable and Low Latency Communications* (URLLC), diseñado para aplicaciones críticas con requisitos estrictos de latencia y fiabilidad; y *massive Machine Type Communications* (mMTC), orientado a soportar una gran densidad de dispositivos con bajo consumo energético [1] [2].

Estas capacidades permiten soportar aplicaciones avanzadas como el control de tráfico conectado, la operación de drones en entornos urbanos, la cirugía remota asistida o los sistemas de automatización industrial, no viables con generaciones anteriores de redes móviles. Además, estas aplicaciones generan grandes volúmenes de datos y requieren infraestructuras de comunicación capaces de soportar una demanda creciente de conectividad y procesamiento. Se estima que el volumen global de datos aumentará en varios cientos de zettabytes durante la próxima década, lo que pone de manifiesto la necesidad de evolucionar las infraestructuras de red existentes [2].

2.1.1 Arquitectura 5G

Desde el punto de vista arquitectónico, el sistema 5G mantiene los elementos fundamentales presentes en generaciones anteriores, incluyendo el equipo de usuario (*User Equipment*, UE), la red de acceso radio (*Next Generation Radio Access Network*, NG-RAN) y el núcleo de red (*5G Core*, 5GC). No obstante, introduce una reorganización profunda de sus funciones internas [1].

La NG-RAN tiene como entidad principal el *gNodeB* (gNB), responsable de gestionar la interfaz radio con los dispositivos y de coordinar la transmisión de datos entre el UE y el núcleo de red. A diferencia del *eNodeB* (eNB) de las redes 4G, el gNB introduce una nueva técnica de transmisión radio que permite tiempos de respuesta más rápidos [1]. La **Figura 2.1** muestra la arquitectura basada en servicios del 5GS.

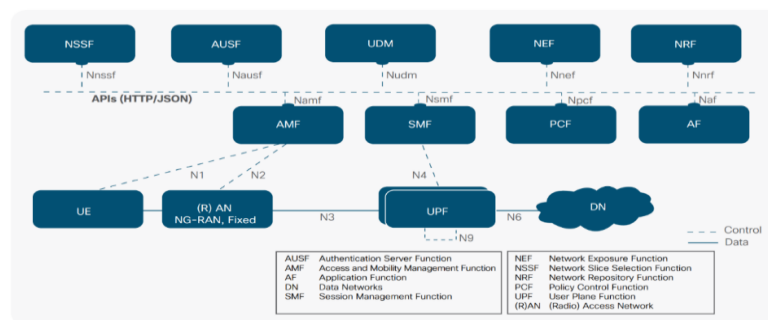


Figura 2.1. Arquitectura basada en servicios del 5GS y principales funciones de red.
Fuente: 3GPP TS 23.501 [1].

Por su parte, el núcleo de red 5G separa el plano de usuario, gestionado por la función *User Plane Function* (UPF), del plano de control, administrado por funciones como *Access and Mobility Management Function* (AMF). Esta modularidad permite una mayor flexibilidad, reutilización de funciones y adaptación del sistema a distintos escenarios y sectores industriales [1].

La comunicación entre la red de acceso y el núcleo de red se realiza a través del punto de referencia NG (*Next Generation*), compuesto principalmente por las interfaces N2 y N3. La interfaz N2 transporta la señalización entre la NG-RAN y la función AMF, permitiendo la gestión de la movilidad y el control de acceso de los dispositivos. Por su parte, la interfaz N3 transporta el tráfico de datos de usuario entre la NG-RAN y la función UPF, siendo responsable del encaminamiento de los paquetes hacia los servicios de destino [1].

La arquitectura del 5GC se basa en el paradigma *Service-Based Architecture* (SBA), en el que los componentes se definen como *Network Functions* (NFs) que ofrecen servicios a otras funciones de red mediante interfaces estandarizadas. Este enfoque introduce una mayor modularidad y flexibilidad respecto a arquitecturas anteriores, permitiendo desplegar y reutilizar funciones de red de forma más dinámica [1].

Entre las funciones más relevantes del núcleo 5G destacan la *Session Management Function* (SMF), responsable de gestionar las sesiones de datos y coordinar la asignación de recursos con la UPF; la *Unified Data Management* (UDM), encargada de almacenar la información de los usuarios; y la *Policy Control Function* (PCF), que define las políticas de control y QoS aplicadas al tráfico de red. Además, la arquitectura incorpora funciones adicionales relacionadas con la gestión de la red, la seguridad y el soporte de capacidades avanzadas como el *network slicing*, que permite adaptar la red a distintos tipos de servicios y aplicaciones [1].

2.1.2 Despliegue del 5G: Arquitecturas Non-Standalone y Standalone

El sistema 5G contempla dos opciones principales de despliegue definidas por 3GPP como *Non-Standalone* (NSA) y la arquitectura *Standalone* (SA), las cuales facilitan la transición desde las redes 4G hacia un sistema 5G completo, permitiendo a los operadores adaptar el despliegue a sus infraestructuras existentes y a los requisitos de los servicios ofrecidos [9]. La **Figura 2.2** muestra una comparativa entre ambas.

La arquitectura NSA se concibe como una solución transitoria que combina la nueva interfaz *5G New Radio* (NR) con el núcleo de la red 4G existente, *Evolved Packet Core* (EPC). En este escenario, conocido como *E-UTRA-NR Dual Connectivity* (EN-DC), la red *Long Term Evolution* (LTE) funciona como nodo maestro para el plano de control y la gestión de la movilidad, mientras que la radio 5G opera como nodo secundario. El despliegue permite mejores prestaciones sin necesidad de sustituir inmediatamente el núcleo de red 4G, por lo que resulta principalmente beneficioso para servicios de eMBB y en la mejora de la experiencia de usuario en aplicaciones como vídeo de alta calidad o servicios multimedia inmersivos [9] [10].

Por otra parte, surge la arquitectura *Standalone* (SA), la cual representa el despliegue completo del 5GS, donde la red de acceso radio (NG-RAN) se conecta directamente al núcleo de red 5G, independiente de cualquier elemento de las redes 4G [9]. El núcleo 5G introduce una arquitectura basada en servicios y diseñada para entornos nativos de la nube, permitiendo una mayor escalabilidad, flexibilidad y capacidad de adaptación frente a distintos tipos de servicios. A diferencia del modelo NSA, el despliegue SA es el único capaz de habilitar todas las funcionalidades del sistema 5G, incluyendo ultra baja latencia, gestión masiva de dispositivos del Internet de las Cosas (*Internet of Things*, IoT), y *network slicing* de extremo a extremo. Además, introduce mejoras en términos de seguridad y privacidad, lo que convierte a esta arquitectura en la base indispensable para garantizar los requisitos estrictos de QoS en entornos distribuidos de *Edge Computing* y casos de uso como conducción autónoma, servicios industriales o aplicaciones de misión crítica [9] [10].

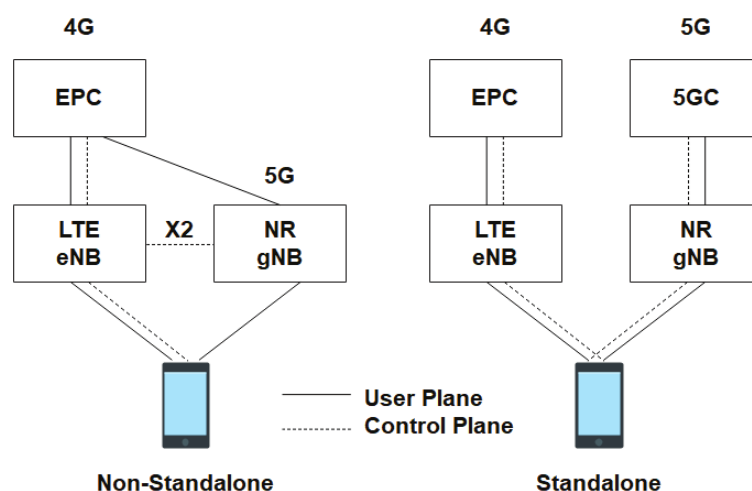


Figura 2.2. Comparativa entre las arquitecturas de despliegue NSA y SA del sistema 5G. Fuente: Vochin et al. [11].

2.1.3 Virtualización y softwarización en el sistema 5G

La transformación del sistema 5G desde una infraestructura basada en hardware rígido y propietario hacia un entorno flexible y desacoplado es posible gracias a la adopción de dos paradigmas tecnológicos complementarios: *Software Defined Networking* (SDN) y *Network Functions Virtualization* (NFV). Por un lado, SDN introduce la separación entre el plano de control y el plano de datos, permitiendo una gestión y configuración de red por software dinámica, programable y agnóstica al hardware, optimizando así de manera más específica los flujos de tráfico en tiempo real [12]. Por otro lado, NFV consiste en desacoplar el software de red de los equipos físicos, permitiendo que las diferentes funciones tradicionales de red como pueden ser el encaminamiento, el filtrado, el balanceo de carga, etc. sean implementados como software virtualizado que se ejecuta sobre servidores estándar en lugar de sobre hardware dedicado [13]. Estas instancias software se conocen como *Virtual Network Functions* (VNFs).

A pesar de actuar en capas distintas de la infraestructura, ya que SDN es responsable del control lógico del tráfico y NFV de la abstracción y ejecución de las funciones, su convergencia constituye la base de la softwarización de las redes 5G [14] y habilita capacidades avanzadas como la automatización y escalabilidad [13], así como la creación de *network slices* adaptados a distintos servicios y requisitos.

Esta cooperación reduce la complejidad de la infraestructura, disminuye los costes operativos y facilita la adaptación dinámica de la red a distintos servicios y requisitos, aspectos especialmente relevantes en redes 5G [12] [13].

2.2 Edge Computing

El crecimiento exponencial del número de dispositivos conectados y del volumen de datos generado por tecnologías como IoT, la automatización industrial o los sistemas inteligentes ha puesto de manifiesto algunas limitaciones de los modelos tradicionales de computación en la nube. En estos entornos, el envío continuo de grandes cantidades de datos hacia centros de datos remotos puede provocar problemas de latencia, congestión de red y un uso ineficiente del ancho de banda, dificultando la prestación de servicios que requieren tiempos de respuesta reducidos y un procesamiento cercano al usuario. En este contexto surge el paradigma de *Edge Computing*, cuyo objetivo principal es

acercar las capacidades de computación y almacenamiento a nodos próximos al usuario o al lugar donde se generan los datos [3] [4].

Este enfoque permite reducir de forma significativa la latencia, mejorar la estabilidad de los servicios y optimizar el uso de los recursos de red. Además, al procesar los datos localmente, se disminuye la cantidad de información que debe ser transmitida a la nube, lo que contribuye a reducir los costes de red, evitar restricciones de ancho de banda y mejorar el control sobre datos sensibles. Gracias a estas características, *Edge Computing* facilita el análisis de datos casi en tiempo real y habilita nuevos casos de uso que requieren respuestas rápidas y fiables [3] [4].

Desde un punto de vista conceptual, una infraestructura de *Edge Computing* se organiza en distintos niveles lógicos que se extienden desde la nube centralizada hasta el borde de la red. En el nivel más interno se sitúan los sistemas centrales de las empresas y proveedores de nube, correspondientes a los entornos tradicionales de computación centralizada. En niveles más próximos al usuario se localizan los recursos de computación desplegados en infraestructuras empresariales o en extremo de la red del operador. Finalmente, en el nivel más alejado del centro de datos se sitúa el *Edge* del dispositivo, formado por sistemas independientes que conectan directamente sensores y actuadores. Esta organización jerárquica permite distribuir las capacidades de computación y procesamiento de forma flexible, adaptándose a los requisitos de latencia, ancho de banda y seguridad de cada aplicación [4]. La **Figura 2.3** ilustra esta organización por niveles.

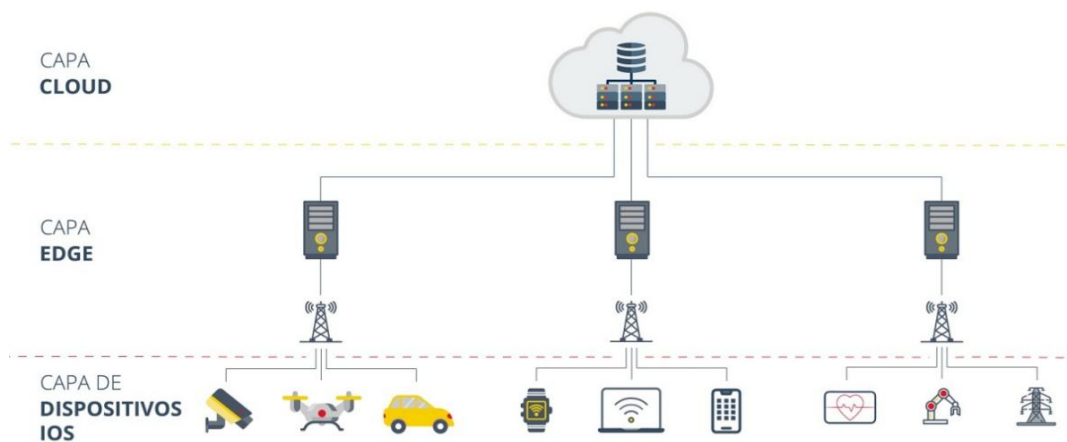


Figura 2.3. Arquitectura conceptual del *Edge Computing* organizada en distintos niveles. Fuente: Instituto Nacional de Ciberseguridad (INCIBE) [15].

2.2.1 MEC en redes móviles

Dentro del paradigma general de *Edge Computing*, surge MEC como una arquitectura específica estandarizada por ETSI orientada al despliegue de servicios en redes móviles y en infraestructuras de proveedores de telecomunicaciones. MEC proporciona capacidades de computación en la nube y servicios de tecnologías de la información directamente en el borde de la red de acceso, integrándose con la infraestructura de los operadores de red. Su objetivo principal es reducir la latencia, mejorar la eficiencia del uso de los recursos de red y facilitar la prestación de servicios avanzados a los usuarios finales [5] [6].

MEC expone un entorno de ejecución distribuido en la propia red 5G del operador y acoplado a la función UPF del núcleo de la red, lo que permite desviar el tráfico de datos de manera local y ejecutar aplicaciones críticas, como análisis de vídeo o guiado de vehículos conectados directamente en el borde. Además, esta arquitectura, además de baja latencia, refuerza las comunicaciones ultra fiables y permite aprovechar las características nativas de 5G (como URLLC) para ejecutar aplicaciones avanzadas basadas en análisis de datos, inteligencia artificial o ML, aprovechando tanto el procesamiento local como la conectividad proporcionada por la red móvil [5]. La **Figura 2.4** muestra la arquitectura del sistema MEC y su integración con el núcleo de red 5G.

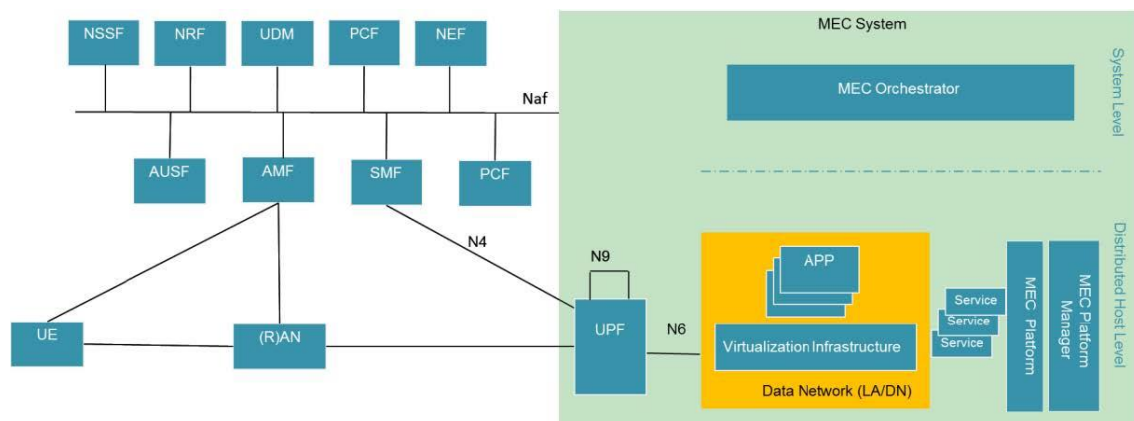


Figura 2.4. Arquitectura del sistema MEC e integración con el núcleo de red 5G. Fuente: ETSI [16].

Mientras que *Edge Computing* engloba diferentes escenarios y arquitecturas orientadas a acercar la computación al usuario final, MEC se centra en el entorno de las telecomunicaciones y en la integración con las redes móviles, especialmente con la tecnología 5G. De esta forma, MEC representa un caso particular de *Edge Computing*.

adaptado a los requisitos de latencia, movilidad y gestión propios de las redes de comunicaciones móviles [5].

En este contexto, la combinación de *Edge Computing*, MEC y redes 5G constituye una base tecnológica clave para el desarrollo de servicios inteligentes capaces de adaptarse dinámicamente a las condiciones de la red y a las demandas de los usuarios.

2.3 Principales parámetros de rendimiento en redes 5G y Edge Computing

Los KPIs se definen como métricas cuantitativas utilizadas para medir el rendimiento de un sistema en relación con unos objetivos previamente definidos [17] [18].

En redes 5G y entornos de *Edge Computing*, estos KPIs permiten supervisar de forma continua parámetros críticos del sistema, identificar desviaciones respecto a los valores objetivo, y evaluar su impacto sobre el servicio ofrecido. Debido a la naturaleza extremo a extremo de 5G, el rendimiento depende de múltiples dominios, incluyendo el acceso radio, el núcleo de la red y los recursos de computación en el borde. Por ello, es necesario analizar conjuntamente diferentes categorías de KPIs para tener una visión global del sistema y verificar el cumplimiento de los niveles de servicio establecidos en acuerdos de nivel de servicio (*Service Level Agreements*, SLA) [19].

En este contexto, la QoS se evalúa mediante un conjunto de KPIs adaptados a la infraestructura *edge*, como latencia, ancho de banda, pérdida de paquetes o prioridad del tráfico. Estos KPIs no solo garantizan el rendimiento técnico, sino que además constituyen una herramienta fundamental para garantizar la calidad del servicio y apoyar la toma de decisiones orientadas a la optimización y gestión eficiente de los recursos de red con requisitos heterogéneos.

A continuación, se describen las principales categorías de parámetros utilizados para caracterizar el rendimiento de un sistema.

2.3.1 Parámetros de tráfico y demanda

El volumen de tráfico constituye uno de los principales indicadores para caracterizar la carga y dimensionar la infraestructura del sistema. En términos técnicos el tráfico puede analizarse mediante métricas como el *throughput* agregado, la tasa de llegada de paquetes, el número de usuarios concurrentes, la distribución temporal de la demanda y el volumen de datos por servicio o por tipo de aplicación.

En entornos 5G, el tráfico presenta una elevada heterogeneidad y variabilidad, debido a la coexistencia de aplicaciones con comportamientos y requisitos muy distintos, como IoT, servicios multimedia interactivos o comunicaciones industriales. Esta dinámica da lugar a patrones de tráfico dinámicos, influenciados por factores como la movilidad de los usuarios, y disponibilidad de recursos, lo que complica su gestión y análisis mediante enfoques reactivos tradicionales. En entornos de *Edge Computing*, la predicción de la demanda es imprescindible, ya que los picos de tráfico no solo saturan la red de acceso y el núcleo, sino también los nodos de computación, donde la concentración de solicitudes puede afectar la capacidad de procesamiento y el rendimiento extremo a extremo del servicio [20].

2.3.2 Parámetros de latencia

La latencia extremo a extremo representa el tiempo transcurrido entre el envío de un paquete desde origen a destino. Una latencia elevada puede provocar tiempos de carga excesivos, degradación del servicio y una percepción negativa por parte de los usuarios finales. Además, puede generar sobrecostes operativos y dificultar la mejora progresiva de los servicios ofrecidos [7].

El análisis de latencia suele realizarse mediante latencia media, latencia máxima, o *jitter*. A diferencia del ancho de banda, que indica la capacidad máxima de transmisión en el canal, la latencia mide el tiempo de respuesta del sistema. Por su parte, el rendimiento (*throughput* efectivo) refleja la cantidad real de datos transferidos y puede verse afectado tanto por la latencia como por pérdidas de paquetes o congestión [21].

En sistemas de *Edge Computing*, la reducción de latencia se basa en el procesamiento de peticiones de forma muy cercana al usuario final, por lo que resulta imprescindible. No obstante, la latencia final depende de múltiples componentes, incluyendo no solo segmentos de red (radio, transporte y núcleo), sino también el propio procesamiento en los servidores de *Edge Computing*, su comunicación y su carga de trabajo, lo que hace necesario analizar este parámetro de forma integral para garantizar el cumplimiento de los requisitos de servicio.

2.3.3 Parámetros de congestión y QoS

La congestión de red se produce cuando una red recibe más tráfico del que puede gestionar de forma eficiente, lo que provoca una disminución de la calidad del servicio. Esta situación suele manifestarse mediante incrementos del retardo, *jitter*, pérdida de paquetes o el bloqueo de nuevas conexiones. La congestión puede aparecer de forma puntual debido a picos de tráfico, aunque cuando se mantiene en el tiempo suele indicar limitaciones en la capacidad de la red o en la gestión de sus recursos [22].

En este contexto, la QoS se utiliza como un conjunto de mecanismos encargados de gestionar los recursos disponibles en situaciones de saturación mediante la clasificación y la priorización de distintos flujos de tráfico [23]. De este modo, la monitorización de parámetros de congestión y QoS aporta al sistema la capacidad de anticipar situaciones de bajo rendimiento, permitiendo que las funciones de orquestación implementen políticas preventivas de control de admisión o asignación dinámica de ancho de banda antes de que se produzca la degradación del servicio percibido por los usuarios.

2.3.4 Parámetros de utilización de recursos de infraestructura

En entornos 5G y de *Edge Computing* el rendimiento no depende únicamente de la capacidad de transmisión, sino también del uso eficiente de recursos computacionales y de infraestructura. Entre las principales métricas de utilización de recursos se encuentran el uso de CPU, el consumo de memoria, la ocupación de enlaces, y la tasa de almacenamiento. Una utilización de recursos sostenida cercana a los límites de la infraestructura afecta al tiempo de respuesta de los servidores que ejecutan las aplicaciones en el borde, provocando un aumento del retardo de procesamiento extremo a extremo.

Además del uso de recursos computacionales, la eficiencia energética se ha convertido en un aspecto clave en redes de comunicaciones modernas debido al incremento sostenido del tráfico y la densificación de la infraestructura. En este contexto, la monitorización de la utilización de recursos y del consumo energético resulta fundamental para evaluar la eficiencia del sistema y avanzar hacia redes más sostenibles y capaces de adaptarse a las demandas crecientes de tráfico [24].

A continuación, se presenta la **Tabla 2.1**, que muestra un resumen con las principales categorías de parámetros de rendimiento analizadas y su relevancia en entornos 5G y *Edge Computing*:

Tabla 2.1. Principales categorías de parámetros de rendimiento en entornos 5G y *Edge Computing*.

Categoría	Métricas principales	Descripción
Tráfico y demanda	Tasa de peticiones, peticiones rechazadas	Miden la carga que recibe el sistema en cada instante. Un aumento brusco de peticiones puede anticipar situaciones de congestión.
Latencia	Latencia media, latencia máxima, <i>jitter</i>	Miden el tiempo de respuesta. Determinan el cumplimiento de los requisitos de SLA en servicios sensibles a la latencia.
Congestión y QoS	Pérdida de paquetes, <i>throughput</i> , QoS	Miden el impacto de la saturación sobre la calidad del servicio, degradándose cuando el sistema no puede gestionar el tráfico.
Utilización de recursos	Uso de CPU, memoria, ocupación de servidores	Miden la eficiencia de la infraestructura. Una utilización sostenida cercana al 100% indica riesgo de saturación.

2.4 Simulación y emulación

En el ámbito de las redes de comunicaciones, la simulación y la emulación son dos enfoques ampliamente utilizados para analizar y validar el comportamiento de una red antes de aplicar cambios en un entorno real. Ambos métodos permiten estudiar cómo afectan distintas configuraciones, topologías o cargas de tráfico al rendimiento del sistema, reduciendo riesgos y costes asociados a pruebas directas sobre infraestructuras productivas [25].

La simulación de red se basa en la creación de modelos que representan los distintos componentes de una red, como dispositivos, enlaces y flujos de datos. La mayoría de las herramientas disponibles operan bajo el paradigma de simulación por eventos discretos, donde el estado de cada elemento se va actualizando conforme ocurren acciones como la llegada de paquetes, su reenvío o posibles colisiones. La simulación permite analizar cambios en la configuración de la red, estudiar distintos escenarios de tráfico o evaluar el impacto de nuevas topologías de forma controlada y flexible.

Por otro lado, la emulación de red consiste en replicar el comportamiento de una red combinando elementos software con hardware real. A diferencia de la simulación, un emulador interactúa con aplicaciones y protocolos de red en tiempo real, inyectando

retardos artificiales, pérdidas de paquetes o restricciones de ancho de banda directamente en el plano de datos. Este enfoque puede requerir hardware especializado para replicar con mayor precisión determinados componentes de red.

Aunque simulación y emulación se consideran herramientas complementarias, el análisis masivo de escenarios heterogéneos y la generación de telemetría a gran escala se aborda habitualmente mediante herramientas de simulación por eventos discretos debido a su determinismo y viabilidad para replicar patrones de movilidad y distintas configuraciones. En el ámbito concreto de las redes 5G y *Edge Computing*, existen varios simuladores que implementan estos enfoques [25].

El simulador utilizado en este Trabajo de Fin de Grado es MintEDGE (*Multi-tier simulator for eNergy-aware sTrategies in Edge Computing*), una herramienta de simulación desarrollada en la Universidad de Castilla-La Mancha y presentado en MobiCom 2023, conferencia organizada por la *Association for Computing Machinery* (ACM) [26], cuyo código está disponible públicamente [27].

MintEDGE es una herramienta de simulación orientada específicamente a entornos de *Edge Computing* en redes celulares, que permite evaluar estrategias de optimización de recursos y energía en escenarios configurables con movilidad de usuarios y múltiples tipos de servicios con requisitos heterogéneos de QoS. A diferencia de los simuladores de propósito general, MintEDGE genera de forma nativa métricas y trazas de telemetría detalladas por servicio y estación base, denominada *Base Station* (BS) en el contexto del simulador, lo que proporciona el volumen y la homogeneidad de datos necesarios para cumplir el objetivo de este trabajo. Su descripción arquitectónica y funcional detallada se aborda en el Capítulo 3.

Existen otros simuladores ampliamente utilizados en la comunidad investigadora que abordan problemáticas similares y que han sido analizados como alternativas de simulación. Por un lado, ns-3 (*Network Simulator 3*) [28] es uno de los simuladores de redes más extendidos en el ámbito académico, de propósito general y altamente configurable, aunque su uso en escenarios 5G requiere extensiones específicas, su curva de aprendizaje es elevada y carece de extensiones nativas para la orquestación elástica de tareas y energía en entornos *Edge Computing*. Por otro lado, OMNeT++ (*Objective Modular Network Testbed in C++*) [29], junto con extensiones de simulación como Simu5G [30], permite simular redes 5G con soporte para comunicaciones entre

dispositivos y núcleo de red, siendo una referencia habitual en trabajos de investigación sobre redes móviles. No obstante, requiere desarrollos adicionales complejos para simular carga y computación de los servidores *edge*. Por último, EdgeCloudSim [31] está orientado específicamente a entornos de *Edge Computing*, permitiendo modelar la distribución de tareas entre dispositivos móviles y servidores *edge*, con soporte para movilidad de usuarios. Sin embargo, el modelado de los protocolos de la red celular es demasiado simplificado.

2.5 Modelos de predicción aplicados a series temporales

Los modelos de predicción constituyen una herramienta fundamental dentro del análisis avanzado de datos, ya que permiten estimar el comportamiento futuro de un sistema a partir de información histórica. De forma general, el modelado predictivo puede definirse como un conjunto de técnicas estadísticas y computacionales que utiliza datos previamente observados para construir modelos matemáticos capaces de generar una estimación sobre resultados futuros. Estos modelos emplean un conjunto de variables de entrada relevantes para producir una variable de salida que representa la predicción del fenómeno analizado [32].

En el contexto de las infraestructuras de comunicaciones y los entornos de *Edge Computing*, donde la telemetría genera flujos de datos continuos, el análisis predictivo es el componente clave para evolucionar desde sistemas de gestión reactivos hacia estrategias de orquestación proactivas. En este sentido, el desarrollo de modelos predictivos apoyados en técnicas estadísticas y algoritmos de ML permiten generar predicciones sobre el comportamiento futuro de los sistemas estudiados [33].

2.5.1 Tipos de modelos predictivos

Para construir el servicio inteligente de este Trabajo Fin de Grado, y dado que la telemetría de los nodos de cómputo *edge* se registra de forma cronológica, el problema se modela formalmente bajo el paradigma de la predicción de series temporales. Una serie temporal se define como una sucesión ordenada de observaciones cuantitativas $Y = \{y_1, y_2, \dots, y_n\}$, indexadas en el tiempo, donde el objetivo es estimar el valor de la variable en un instante futuro, denominado horizonte de predicción $(t + h)$ [34].

Tradicionalmente, los modelos estadísticos han sido utilizados para identificar relaciones entre variables y realizar predicciones basadas en datos históricos. Sin embargo, el avance de la ciencia de datos ha impulsado el desarrollo de nuevos enfoques basados en ML y *Deep Learning* (DL), capaces de detectar patrones más complejos y mejorar la precisión de las predicciones [35]. A continuación, se describen las principales categorías de modelos utilizadas en tareas de predicción.

2.5.1.1 Modelos estadísticos

Los modelos estadísticos utilizan ecuaciones matemáticas para representar la información extraída de los datos y describir la relación entre distintas variables [36]. Estos modelos se basan en principios probabilísticos y en suposiciones sobre cómo se generan los datos observados, lo que permite realizar estimaciones o predicciones a partir de muestras de datos históricos [37]. Al requerir un coste computacional bajo, suelen emplearse como punto de partida o referencia para técnicas más complejas.

Entre los ejemplos más representativos de este tipo se encuentran la regresión lineal, la regresión logística y los modelos ARIMA (*Autoregressive Integrated Moving Average*), ampliamente utilizados para la predicción de series temporales.

2.5.1.2 Modelos de *Machine Learning*

El ML es una rama de la inteligencia artificial centrada en el desarrollo de algoritmos capaces de aprender patrones a partir de datos de entrenamiento sin depender de reglas predefinidas o ecuaciones rígidas, ajustando sus parámetros internos de forma automatizada para maximizar la precisión de la predicción. Gracias a esta capacidad de aprendizaje, estas técnicas permiten analizar grandes volúmenes de datos y realizar tareas de clasificación o regresión sobre nuevos datos no observados previamente [35] [38].

En el presente trabajo el problema se aborda como una tarea de regresión, dado que el objetivo es predecir un valor continuo a partir de variables de entrada numéricas. Se emplean cuatro modelos de ML en esta variante, seleccionados por su eficiencia computacional, su buen rendimiento sobre datos tabulares estructurados y su amplia adopción en tareas de predicción en entornos de ingeniería de datos.

El **árbol de decisión** (*DecisionTree*) es un método de aprendizaje supervisado no paramétrico aplicable tanto a clasificación como a regresión. En su variante de regresión, el modelo aprende reglas de decisión simples a partir de las variables de entrada para predecir el valor de una variable continua, interpretable como una aproximación por tramos constantes del espacio de entrada. Su principal ventaja es la interpretabilidad, ya que las reglas aprendidas pueden visualizarse explícitamente. No obstante, árboles de profundidad elevada tienden a sobreajustarse a los datos de entrenamiento, por lo que en este trabajo se emplea como modelo de referencia frente a técnicas de mayor capacidad [39].

El **bosque aleatorio** (*RandomForest*) es un método de aprendizaje por conjuntos que entrena múltiples árboles de decisión sobre distintas submuestras del conjunto de datos obtenidas mediante muestreo con reemplazamiento, combinando sus predicciones mediante promediado para mejorar la precisión y reducir el sobreajuste [40]. Esta diversidad entre árboles, permite reducir la varianza del modelo respecto a un árbol individual sin incrementar su sesgo.

XGBoost (*eXtreme Gradient Boosting*) es un sistema de *gradient boosting* basado en árboles de decisión que construye el modelo de forma secuencial, donde cada árbol nuevo corrige los errores cometidos por los anteriores. A diferencia del bosque aleatorio, XGBoost optimiza una función objetivo que incorpora términos de regularización para controlar la complejidad del modelo y reducir el sobreajuste. Además, destaca por su eficiencia computacional y su capacidad para escalar a grandes volúmenes de datos [41].

LightGBM (*Light Gradient Boosting Machine*) es una implementación eficiente de *gradient boosting* sobre árboles de decisión como alternativa más eficiente a XGBoost para conjuntos de datos de alta dimensionalidad y gran volumen. Implementa una estrategia de crecimiento de árboles por hojas e introduce las técnicas de muestreo basado en gradientes (*Gradient-based One-Side Sampling*, GOSS), que descarta las instancias con gradientes pequeños en el cálculo de la ganancia de información, y el empaquetado exclusivo de características (*Exclusive Feature Bundling*, EFB), que agrupa variables mutuamente excluyentes para reducir la dimensionalidad efectiva [42].

Los modelos mencionados anteriormente carecen de noción nativa de secuencialidad temporal. Por ello, para permitir su uso sobre series temporales como el caso de este Trabajo de Fin de Grado, se implementa el método de la ventana deslizante. Esta técnica

transforma la secuencia unidimensional en una estructura tabular supervisada, donde los valores pasados de la métrica ($y_t, y_{t-1}, \dots, y_{t-W+1}$) actúan como características de entrada que se añaden como nuevas métricas para predecir el valor futuro de la métrica objetivo (y_{t+1}). Durante este proceso, se garantiza una partición cronológica para evitar la filtración de información.

2.5.1.3 Modelos de Deep Learning

El DL constituye una rama del ML, basada en redes neuronales artificiales inspiradas en el funcionamiento del cerebro humano, y están compuestas por una capa de entrada, múltiples capas ocultas y una capa de salida, lo que permite abordar problemas de mayor complejidad [43].

La **Figura 2.5** muestra la estructura básica de una red neuronal con varias capas, donde cada nodo procesa la información recibida y la transmite hacia la siguiente capa.

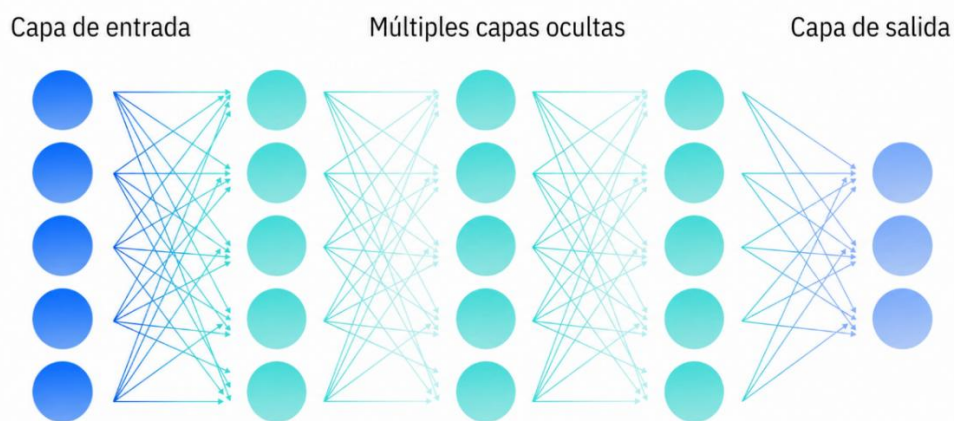


Figura 2.5. Arquitectura de una red neuronal con múltiples capas ocultas. Fuente: IBM [43].

En este Trabajo de Fin de Grado se emplean dos arquitecturas de DL para la predicción de KPIs de QoS en series temporales como son las redes de memoria a largo plazo (*Long Short-Term Memory*, LSTM) y las redes neuronales convolucionales (*Convolutional Neural Network*, CNN). Estas arquitecturas se seleccionaron por su capacidad demostrada para modelar dependencias temporales en datos secuenciales y por ser ampliamente utilizadas en tareas de predicción sobre series temporales en entornos de red.

Los **modelos LSTM** son un tipo de red neuronal recurrente especialmente diseñada para aprender dependencias temporales en datos secuenciales [44]. Las LSTM incorporan un mecanismo de celdas de memoria controlado por tres puertas (*forget gate*, *input gate*

y *output gate*) que regulan qué información se conserva, se actualiza o se descarta a lo largo de la secuencia [45].

La **Figura 2.6** ilustra la arquitectura interna de una celda LSTM, donde el estado de celda C_t fluye de $t - 1$ a $t + 1$ manteniendo la memoria a largo plazo mediante las puertas de olvido (*forget gate*) y entrada (*input gate*), y el estado oculto h_t transporta la información relevante hacia el siguiente instante temporal a través de la puerta de salida (*output gate*).

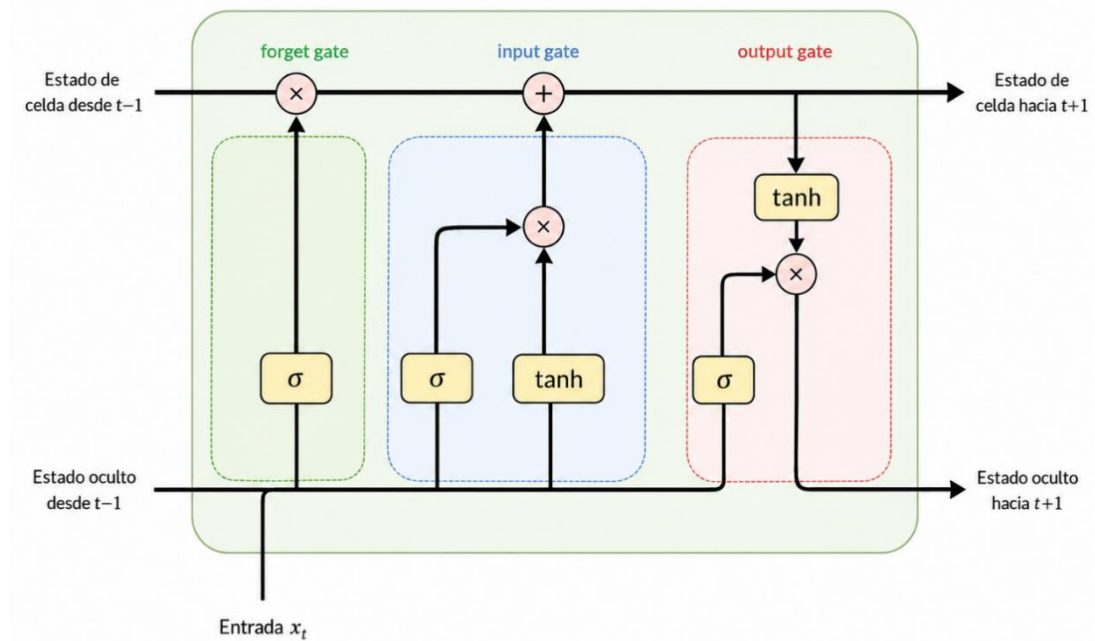


Figura 2.6. Arquitectura de una celda LSTM con sus tres puertas de control (*forget gate*, *input gate* y *output gate*). Adaptado de [46].

Los **modelos CNN** son una arquitectura de aprendizaje profundo que se basa en la aplicación de filtros o *kernels* para extraer patrones de forma jerárquica. Desarrolladas originalmente para el procesamiento de imágenes [47], su aplicación se ha extendido al análisis de series temporales mediante arquitecturas unidimensionales (1D CNN). En su variante 1D, los filtros se desplazan a lo largo de la secuencia temporal extrayendo patrones locales entre instantes consecutivos, lo que permite capturar dependencias a corto plazo de forma eficiente sin necesidad de aprender explícitamente a partir de valores retardados [34].

2.5.2 Métodos de selección de variables

La selección de variables es el proceso de identificar el subconjunto óptimo $S \subseteq F$ de variables de entrada más relevante para la predicción de la variable objetivo y ,

eliminando variables irrelevantes que no aportan información útil al modelo [48]. Su uso mejora la precisión del modelo, reduce el sobreajuste y acorta los tiempos de entrenamiento al reducir la dimensionalidad del espacio de entrada. En este Trabajo de Fin de Grado se aplican métodos de selección basados en correlación estadística, calculados sobre el conjunto de entrenamiento para evitar filtración de información hacia los conjuntos de validación y prueba. A continuación, se describen los cinco métodos empleados.

El coeficiente de correlación de Pearson (r) mide la fuerza y la dirección de la relación lineal entre dos variables X e Y en el rango $[-1, 1]$ donde valores próximos a 1 o -1 indican una relación lineal fuerte y 0 indica ausencia de relación [49]. Presenta la limitación de ignorar dependencias complejas no lineales [50].

El coeficiente de correlación de Spearman (ρ) es una medida no paramétrica que evalúa la fuerza y la dirección de la relación monótona entre dos variables a partir de sus rangos (a diferencia de Pearson), sin requerir que los datos sigan una distribución normal ni que la relación sea lineal [51], lo que lo hace más robusto ante valores atípicos y relaciones no lineales [52]. Como consecuencia, permite identificar relaciones entre variables, aunque estas no sean estrictamente proporcionales.

El coeficiente de Kendall (τ) es una medida no paramétrica que cuantifica la asociación entre dos variables a partir de la proporción de pares concordantes y discordantes, sin asumir linealidad ni distribución normal. Se considera más robusto que Spearman en datos empatados y ofrece una interpretación probabilística más directa, por lo que se emplea para ordenar características según su asociación con la variable objetiva [53].

El estadístico F de regresión lineal evalúa la significancia estadística de cada variable de entrada respecto a la variable objetivo mediante una regresión lineal univariante mediante una regresión lineal univariante, ordenándolas según su p-valor [54].

CFS (*Correlation-based Feature Selection*) selecciona subconjuntos de variables que sean altamente correlacionadas con la variable objetivo pero poco correlacionadas entre sí, penalizando así la redundancia entre variables seleccionadas a diferencia de los métodos univariantes anteriores [55].

2.5.3 Métricas de evaluación

La evaluación del rendimiento de los modelos de regresión se realiza mediante métricas cuantitativas que permiten medir el error de predicción y la bondad del ajuste. En este trabajo se emplean tres métricas complementarias.

El **RMSE (*Root Mean Squared Error*)** cuantifica el error medio entre los valores predichos y los valores reales. Al elevar los errores al cuadrado, penaliza de forma desproporcionada los errores grandes. [56].

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

El **MAE (*Mean Absolute Error*)** calcula el promedio de las diferencias absolutas entre los valores predichos y los reales, siendo más robusto ante valores atípicos [57] [58].

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

El **R² (Coeficiente de determinación)** mide la proporción de la varianza de la variable objetivo explicada por el modelo, tomando valores entre 0 y 1 [59].

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2}$$

Capítulo 3

MintEDGE

Una vez presentado el contexto teórico del trabajo, en este capítulo se aborda el entorno de simulación empleado para el desarrollo experimental del Trabajo de Fin de Grado. En concreto, se describe el simulador MintEDGE, utilizado como base para la generación de escenarios, la obtención de telemetría de red y la evaluación del servicio inteligente de predicción desarrollado. Se analiza su arquitectura interna, sus principales componentes y los parámetros de configuración disponibles.

3.1 Descripción general del simulador

MintEDGE es una herramienta de simulación de código abierto orientado a entornos de *Edge Computing* sobre redes celulares móviles. Su objetivo principal es ofrecer un entorno controlado y determinista para evaluar estrategias de optimización de recursos, analizando de forma simultánea el consumo energético de la infraestructura, la movilidad de usuarios, los mecanismos de asignación de recursos, y la QoS [26]. Su código fuente se encuentra disponible públicamente en GitHub [29].

A diferencia de herramientas de propósito general en redes, MintEDGE destaca por modelar de forma conjunta la capa de red y la de computación *edge*. El simulador permite la integración con herramientas para la generación de escenarios de movilidad realistas mediante mapas e infraestructuras reales, la importación de trazas de movilidad y la definición de estrategias propias de asignación de recursos, orientadas a objetivos

energéticos o de QoS. Además, el simulador permite incorporar predictores de carga computacional [26]. Estas características resultan fundamentales para este Trabajo de Fin de Grado, ya que proporcionan la base experimental para integrar el servicio predictivo propuesto y analizar la transición desde una gestión reactiva hacia el enfoque proactivo necesario en sistemas de *Edge Computing*.

Desde un punto de vista funcional, MintEDGE se articula en torno al orquestador, encargado de controlar tres elementos principales del sistema: el vector de estado, que indica qué estaciones base disponen de servidor *edge* y si dicho servidor se encuentra activo; la matriz de asignación, que determina dónde se procesan las peticiones recibidas en cada estación base; y la matriz de asignación de recursos, que define cómo se reparte la capacidad de cómputo entre los distintos servicios en cada servidor [26].

3.2 Integración de herramientas externas

Para dotar de realismo a la simulación sin introducir una sobrecarga de computación redundante, MintEDGE delega las funciones de movilidad y cartografía en varias herramientas externas consolidadas en la comunidad científica, interactuando con ellas en tiempo de ejecución.

SimPy es un entorno de simulación de eventos discretos basado en procesos e implementado en Python [60]. En MintEDGE, SimPy proporciona el entorno sobre el que se orquesta la ejecución completa del simulador: al arrancar una simulación, se inicializa un entorno SimPy sobre el que se registran y lanzan los procesos correspondientes a la movilidad de los usuarios y a la lógica de orquestación [27].

Simulation of Urban MObility (SUMO) es un simulador de tráfico de código abierto que permite modelar el comportamiento de vehículos y peatones sobre redes viarias reales [61]. En MintEDGE se integra para generar escenarios de movilidad realista a partir de las coordenadas geográficas definidas en **settings.py**. El control de SUMO en tiempo de ejecución se realiza a través de libsumo, una implementación de la interfaz *Traffic Control Interface* (TraCI) que enlaza SUMO directamente como librería, eliminando la sobrecarga de comunicación y mejorando el rendimiento [27].

La **Figura 3.1** muestra la red viaria generada por SUMO a partir de los datos cartográficos de la ciudad de Maastricht.

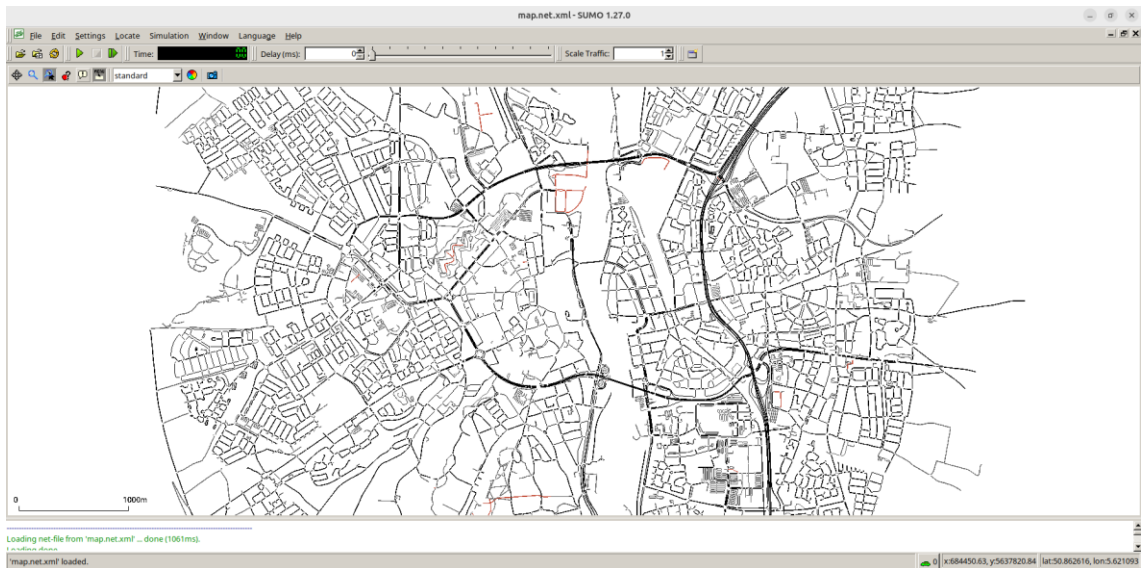


Figura 3.1. Red viaria de la ciudad de Maastricht generada por SUMO a partir de datos de OpenStreetMap. Fuente: elaboración propia.

OSMnx es una librería Python que permite descargar y procesar redes de calles a partir de los datos de OpenStreetMap [62]. En MintEDGE se utiliza durante la inicialización para obtener automáticamente los datos cartográficos del área geográfica del escenario y convertirlos al formato de red requerido por SUMO, de forma que no es necesario proporcionar un fichero de red externo [27].

3.3 Arquitectura y componentes principales

MintEDGE se organiza en torno a un paquete principal **mintedge/**, que contiene los módulos de la lógica del simulador, y un fichero de entrada **MintEDGE.py** que actúa como punto de arranque. La configuración del escenario se gestiona de forma centralizada desde **settings.py**, que es importado por el resto de módulos en tiempo de ejecución [27].

La **Figura 3.2** muestra la arquitectura general del simulador y las relaciones entre estos módulos y las herramientas externas sobre las que se apoya.

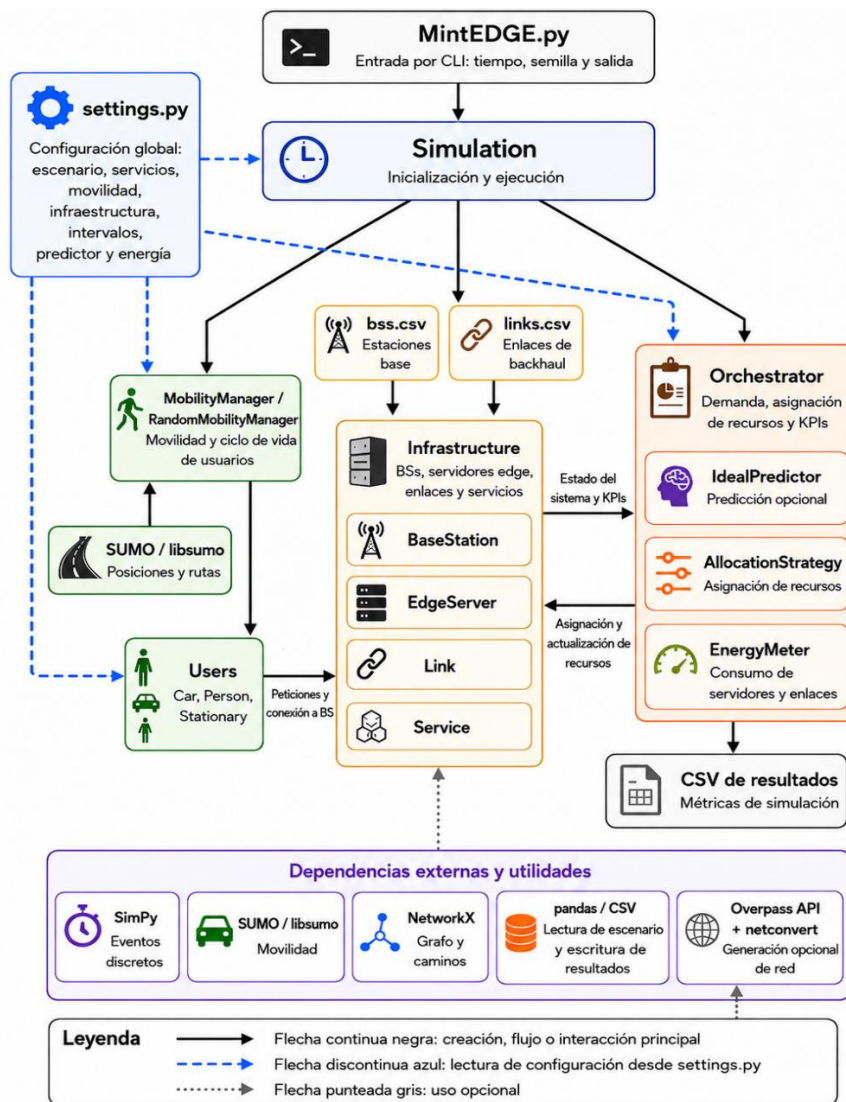


Figura 3.2. Arquitectura de MintEDGE: módulos internos, relaciones y herramientas externas. Fuente: elaboración propia a partir del análisis del código fuente de MintEDGE [27].

A continuación, se describen en detalle los tres componentes principales del simulador, cuyo funcionamiento conjunto determina el comportamiento global de cada ejecución.

Simulation

La clase **Simulation** constituye el núcleo de arranque del simulador [27]. Se encarga de validar la configuración del escenario, comprobando la existencia de los ficheros de infraestructura, la coherencia de los parámetros de red y la validez de los servicios definidos. Tras la validación, inicializa el entorno de SimPy, construye la infraestructura de red, instancia el gestor de movilidad y el orquestador, y arranca la ejecución. Al finalizar la simulación, los resultados se almacenan en el fichero de salida.

MobilityManager y RandomMobilityManager

El módulo de movilidad gestiona el posicionamiento y el ciclo de vida de los usuarios, y sirve de interfaz con SUMO [27]. La clase base **MobilityManager** trabaja con trazas de movilidad predefinidas en formato SUMO (archivo `.rou.xml`), mientras que **RandomMobilityManager** genera rutas aleatorias en tiempo de ejecución mediante la distribución horaria configurada en `USER_COUNT_DISTRIBUTION`. En cada paso, los usuarios se clasifican por tipo y se asocian a la estación base más próxima, determinando así qué servidor *edge* gestionará sus peticiones.

Orchestrator

El **Orchestrator** es el componente central de la lógica de asignación de recursos y el principal generador de métricas. Se activa periódicamente según `ORCHESTRATOR_INTERVAL` para recalculer la distribución de recursos sobre la infraestructura y actualizar el estado del sistema, adaptándolo a las condiciones de carga. En cada ciclo, evalúa la demanda actual y, si `USE_PREDICTOR` está activado, solicita al predictor la demanda máxima esperada para el siguiente intervalo, pasando de un enfoque reactivo a uno proactivo. Sobre este valor se aplica un margen de seguridad (`CAPACITY_BUFFER`) para absorber variaciones imprevistas. Tras cada ciclo, las métricas del sistema se consolidan y se vuelcan al fichero de resultados [26] [27]. La **Figura 3.3** ilustra el flujo de ejecución.

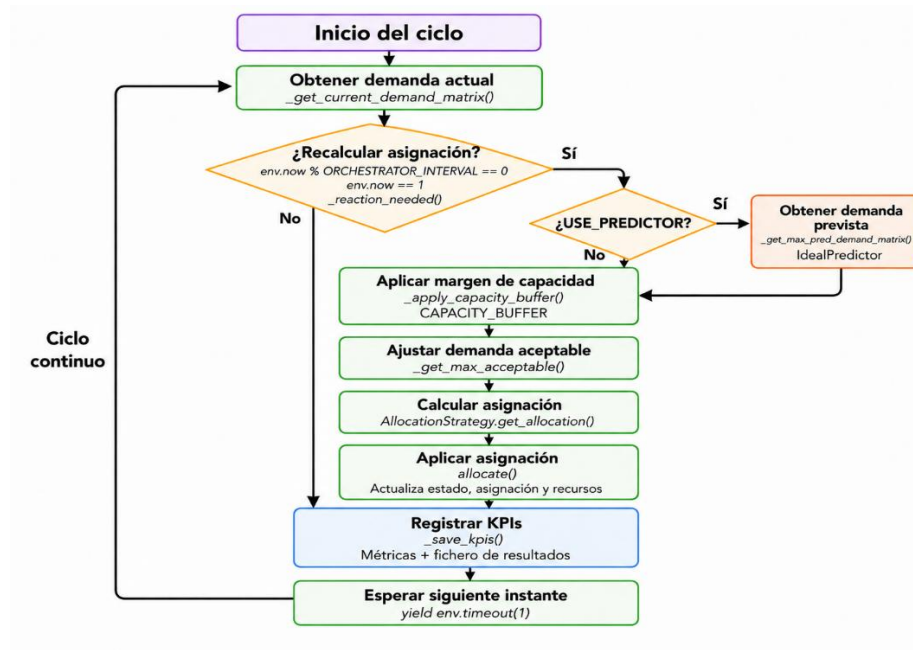


Figura 3.3. Flujo de ejecución del ciclo de orquestación de MintEDGE. Fuente: elaboración propia a partir del análisis del código fuente de MintEDGE [27].

El paquete *mintedge/* incluye además los siguientes módulos de soporte:

- **infrastructure.py** Implementa la clase **Infrastructure**, que representa la topología de red del escenario: estaciones base, servidores *edge*, enlaces de *backhaul* y el grafo de conectividad entre nodos. Es creada por **Simulation** al inicio de cada ejecución a partir de los ficheros de entrada.
- **allocation_strategy.py** Contiene la clase **AllocationStrategy**, que encapsula la lógica de asignación de recursos. Está diseñada para ser modular, permitiendo evaluar distintas estrategias de orquestación sin modificar el resto del simulador.
- **demand_predictor.py** Define la clase **IdealPredictor**, un predictor de demanda con información perfecta. Este módulo es especialmente relevante para este Trabajo de Fin de Grado, ya que el servicio inteligente de predicción desarrollado está concebido para sustituir o complementar este predictor ideal.
- **energy.py** Implementa la clase **EnergyMeter**, que mide el consumo energético de los servidores y los enlaces de *backhaul*.
- **services.py** y **users.py** Definen, respectivamente, los tipos de servicio disponibles en el escenario y los tipos de usuario que pueden generarse.

3.4 Configuración de escenarios y ejecuciones

El comportamiento del simulador está determinado por los parámetros definidos en el módulo **settings.py**, que actúa como punto centralizado de control de una ejecución [27]. A través de este archivo es posible parametrizar escenarios con propiedades heterogéneas, modificando desde la zona geográfica o la infraestructura desplegada, hasta el patrón de movilidad de los usuarios y el modo de operación del orquestador.

3.4.1 Configuración de escenarios geográficos

El escenario geográfico se define mediante cuatro coordenadas (**NORTH**, **SOUTH**, **EAST** y **WEST**). A partir de ellas, el simulador filtra las estaciones base contenidas en **BSS_FILE** correspondientes al operador especificado en **PROVIDER** [27]. MintEDGE incluye configuraciones geográficas predefinidas para regiones como Maastricht, Luxemburgo o Twente, aunque permite la introducción de cualquier área arbitraria.

Una vez determinadas las estaciones base, el parámetro **SHARE_OF_SERVERS** establece la fracción de ellas que recibirán un servidor *edge*. La asignación prioriza los nodos centrales del grafo de *backhaul*, combinando su grado de conectividad y su distancia media a los nodos vecinos [27]. Cada servidor se configura a partir de los modelos definidos en **SERVERS**, especificando potencia máxima, consumo en reposo y capacidad de procesamiento. **PLOT_SCENARIO** permite generar al inicio una imagen del escenario completo con la red viaria, las estaciones base y los servidores asignados.

3.4.2 Modelado de tráfico y movilidad de usuarios

MintEDGE contempla tres tipos de usuario con demandas de servicio diferentes como son vehículos, peatones y usuarios estacionarios (cámaras, sensores, etc.), cuya cantidad máxima se define respectivamente mediante **NUMBER_OF_CARS**, **NUMBER_OF_PEOPLE** y **NUMBER_OF_STATIONARY** [27].

La generación de trayectorias se controla mediante **RANDOM_ROUTES**. Cuando está activado, SUMO genera en tiempo de ejecución rutas aleatorias origen-destino para cada usuario, obteniendo la red viaria automáticamente de OpenStreetMap si no se proporciona un mapa base en **NET_FILE**. Cuando está desactivado, el simulador utiliza trayectorias predefinidas a través de **ROUTES_FILE**, siendo en este caso obligatorio indicar también el **NET_FILE** correspondiente. Para reproducir comportamientos realistas y evaluar fluctuaciones de carga, la fluctuación de usuarios activos durante el ciclo diario se modela mediante **USER_COUNT_DISTRIBUTION**, una lista de 24 valores que expresa la fracción de usuarios activa en cada hora del día [27].

3.4.3 Operación del orquestador y caracterización de servicios

El comportamiento del orquestador se controla principalmente mediante **ORCHESTRATOR_INTERVAL**, que define la frecuencia de recálculo de recursos, y **MEASUREMENT_INTERVAL**, que determina la frecuencia de registro de métricas. Valores más largos de **ORCHESTRATOR_INTERVAL** reducen la capacidad de reacción ante cambios bruscos de demanda, haciendo más relevante disponer de un predictor que los anticipe. **CAPACITY_BUFFER** reserva capacidad adicional sobre la demanda estimada, equilibrando QoS y eficiencia energética, mientras que **USE_PREDICTOR** determina el

modo de operación: desactivado, el orquestador actúa sobre la demanda observada; activado, incorpora una estimación futura pasando a un enfoque proactivo [26] [27].

Los servicios se definen a través del objeto **SERVICES**, especificando para cada uno su carga de trabajo por petición, tasa de llegada, tamaño de *payloads* y presupuesto de retardo máxima tolerable (**delay_budget**), umbral a partir del cual una petición se contabiliza como no satisfecha. En este trabajo se utilizan tres servicios: vehículos conectados, realidad aumentada y análisis de vídeo, cada uno con requisitos de latencia y carga diferenciados [26].

3.5 Resultados generados y métricas obtenidas

Una vez ejecutada la simulación, MintEDGE genera un fichero de resultados que recoge de forma continua la evolución temporal del sistema. La resolución temporal del registro está determinada por el parámetro **MEASUREMENT_INTERVAL**, de forma que el número total de registros depende directamente de la duración de la simulación configurada [27]. Además de métricas globales del sistema, el fichero incluye métricas desagregadas por estación base y servicio, por lo que el número de columnas crece proporcionalmente al número de estaciones base del escenario, manteniendo siempre la misma estructura.

La **Tabla 3.1** recoge las principales métricas generadas, indicando su descripción y si corresponden a nivel global o desagregado.

Tabla 3.1. Principales métricas generadas por MintEDGE.

Métrica	Descripción	Nivel
time	Instante de simulación	Global
active_users	Usuarios activos en el instante	Global
total_requests	Peticiones totales recibidas	Global
total_rejected	Peticiones rechazadas por falta de capacidad	Global
dynamic_W_servers	Consumo dinámico de servidores activos	Global
idle_W_servers	Consumo en reposo de servidores encendidos	Global
W_links	Consumo de enlaces de <i>backhaul</i>	Global
requests	Peticiones recibidas	Por BS y servicio
rejected_req	Peticiones rechazadas por saturación	Por BS y servicio
delay	Retardo medio extremo a extremo	Por BS y servicio
max_delay	Retardo máximo extremo a extremo	Por BS y servicio
unsatisf_req	Peticiones cuyo retardo superó el presupuesto de retardo máximo tolerable	Por servicio
satisfied_rate_over_accepted	Tasa de peticiones satisfechas sobre peticiones aceptadas	Por servicio
server_util	Fracción de capacidad de cómputo utilizada	Por BS con servidor

Capítulo 4

Metodología y desarrollo

En este capítulo se describe la metodología seguida para el diseño y desarrollo del servicio de predicción de KPIs de QoS en sistemas de *Edge Computing*. Previamente al desarrollo del flujo de trabajo definitivo, se llevó a cabo una fase de exploración inicial con un subconjunto reducido de datos que permitió identificar las limitaciones del enfoque original y sentar las bases del diseño final. A partir de estas conclusiones, se detallan las distintas etapas que componen dicho flujo de trabajo: la construcción del conjunto de datos homogéneo, la preparación del problema predictivo, la selección de variables, la configuración de los modelos empleados y la organización de los experimentos de entrenamiento y validación.

El código desarrollado, el script de construcción del conjunto de datos, el notebook experimental y la documentación asociada a los datos brutos y procesados utilizados en este trabajo se encuentran disponibles públicamente en el repositorio de GitHub **edge-network-kpi-predictor**, mantenido por el autor de este Trabajo de Fin de Grado, Luis Melero Jareño. El repositorio puede consultarse en: <https://github.com/luismelero-7/edge-network-kpi-predictor> [63]. Debido al tamaño de los ficheros de datos, estos no se incluyen directamente en GitHub, sino que se proporcionan mediante enlaces externos de OneDrive incluidos en la carpeta `dataset_builder/data`.

La **Figura 4.1** muestra una visión general del flujo completo seguido para el desarrollo del servicio predictivo, desde la obtención de los datos de simulación hasta la generación de los modelos finales.

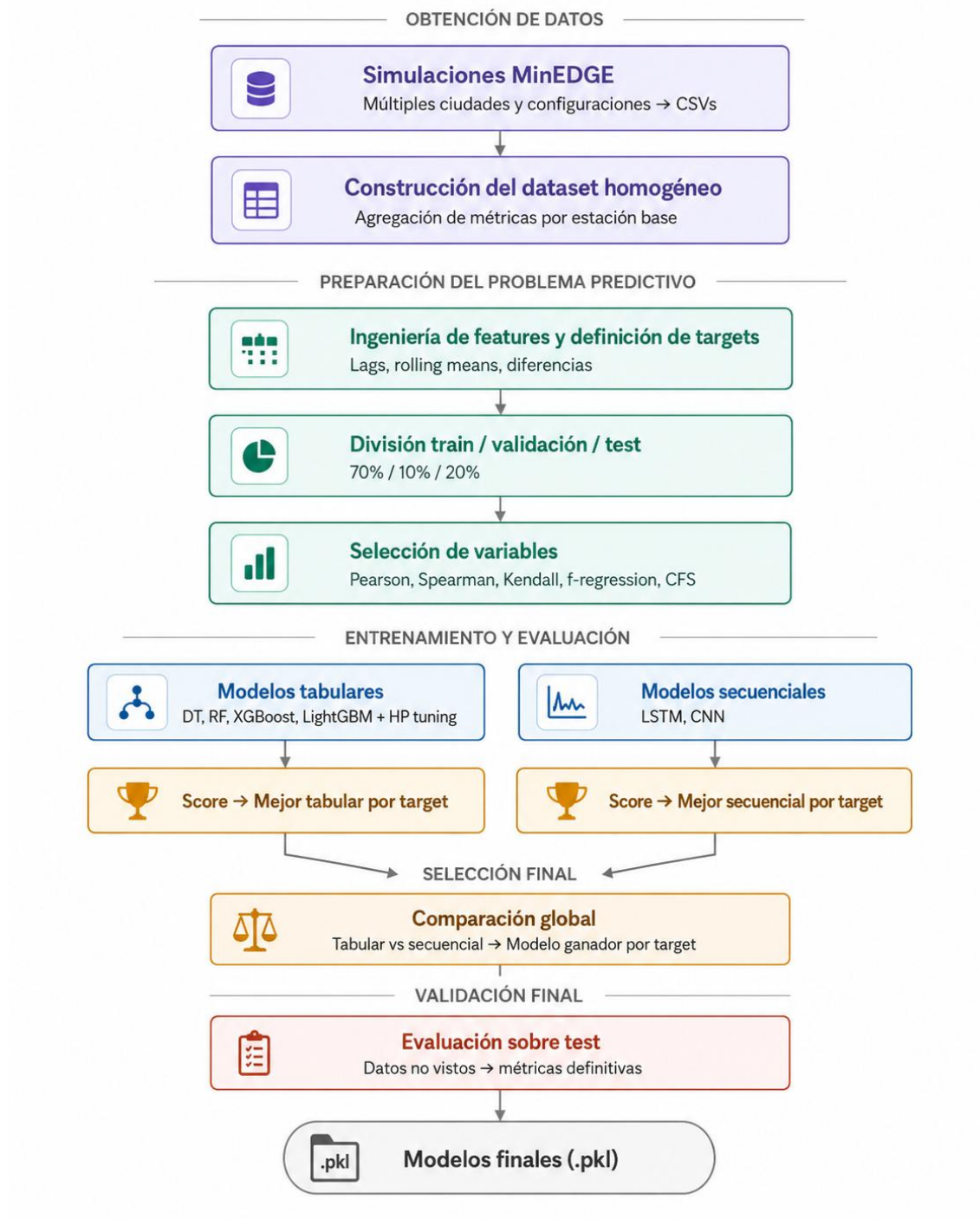


Figura 4.1. Flujo metodológico del servicio predictivo desarrollado. Fuente: elaboración propia.

4.1 Introducción y metodología general

El desarrollo del servicio de predicción se ha guiado por una metodología experimental e iterativa. A diferencia de un diseño cerrado desde el inicio, el proceso evolucionó en ciclos sucesivos de experimentación, análisis de resultados e identificación de limitaciones, lo que permitió refinar progresivamente tanto el conjunto de datos como la selección de modelos y configuraciones. Este enfoque refleja la naturaleza del problema abordado: un proceso científico de prueba y mejora continua en el que cada decisión de diseño se tomó a partir de las conclusiones extraídas en la etapa anterior.

Como punto de partida, se realizó una revisión del ciclo de vida estándar de un proyecto de ML, que fue adaptado al problema concreto de predicción de KPIs de QoS en sistemas de *Edge Computing*. El ciclo resultante quedó estructurado en las siguientes etapas: generación de datos mediante el simulador MintEDGE, construcción de un conjunto de datos homogéneo, selección de variables, división del conjunto de datos en conjuntos de entrenamiento, validación y prueba, entrenamiento de los modelos, y evaluación final sobre el conjunto de prueba.

Con carácter previo a este flujo definitivo, una fase de exploración inicial a partir de un script que generaba un conjunto de datos de tan solo 49 columnas trabajando con datos de dos ciudades reveló que los resultados de MintEDGE presentan un número variable de columnas en función del número de estaciones base del escenario, lo que hace que conjuntos de datos contruidos directamente a partir de ellos sean incompatibles entre ciudades. Dado que los modelos de ML exigen vectores de entrada con una forma idéntica y fija con independencia del escenario geográfico, los datos en bruto hacían inviable la generalización de los modelos entre diferentes ciudades. Esto motivó un replanteamiento del enfoque hacia un conjunto de datos homogéneo basado en agregaciones estadísticas por estación base, escalable a cualquier número de estaciones base.

Finalmente, se procedió a la definición de las variables objetivo del modelo, analizando cuáles aportarían información más relevante sobre la QoS en el contexto del sistema de orquestación. La selección final y los criterios que la justifican se presentan en la sección 4.3.

4.2 Construcción del conjunto de datos homogéneo

La construcción del conjunto de datos de entrenamiento constituye la primera y más crítica etapa del flujo de trabajo. En esta sección se describe el proceso seguido, desde la obtención de los datos de simulación hasta la ingeniería de variables y la corrección del desfase temporal presente en determinadas métricas de QoS.

4.2.1 Datos de partida generados por MintEDGE

Para conformar un conjunto de datos robusto frente a fluctuaciones de carga y escenarios de congestión, se ejecutaron simulaciones en MintEDGE combinando distintas ciudades, operadores y volúmenes de usuarios. Los entornos geográficos corresponden a los escenarios preconfigurados en MintEDGE incluyendo Elburg, Enschede, Luxemburgo, Maastricht y Twente. Los operadores empleados son KPN, T-Mobile, Vodafone y Luxembourg Online, supeditados a la presencia real de su infraestructura en cada región. Los archivos generados siguen un patrón de nomenclatura `results_{ciudad}{proveedor}{coches}Car_{personas}People_s1.csv`, lo que permite identificar de forma sencilla la ciudad, el operador y el volumen de usuarios correspondiente a cada simulación.

Dado que MintEDGE construye la topología basándose en despliegues reales, el número de estaciones base varía considerablemente entre ciudades y operadores. A modo de ejemplo, el escenario de Elburg con KPN cuenta con 36 estaciones base, mientras que el de Twente con KPN llega a 230. Puesto que el simulador genera columnas independientes para combinación de estación base, servicio y métrica, los archivos resultantes poseen una dimensionalidad asimétrica e incompatible entre sí. Esta limitación motivó el proceso de homogeneización que se detalla a continuación.

4.2.2 Homogeneización de escenarios y ciudades

Para posibilitar el entrenamiento de modelos de ML multicidadad, se requiere que todas las muestras compartan el mismo espacio de características. Esta etapa se encuadra en la fase de preprocesado y transformación del ciclo de vida del ML, y su objetivo es resolver el problema sin perder información contenida en los datos originales. La solución adoptada consiste en sustituir las columnas individuales por estación base por estadísticos agregados calculados fila a fila en cada segundo de la simulación: media,

desviación típica, máximo y percentil 95. El número de columnas resultante es siempre el mismo, lo que hace el conjunto de datos generalizable a todos los escenarios sin modificar la estructura de entrada del modelo. Las columnas resultantes no introducen información adicional respecto a la contenida en el CSV original, sino que la reorganizan en un formato homogéneo, tal y como ilustra la **Figura 4.2**.

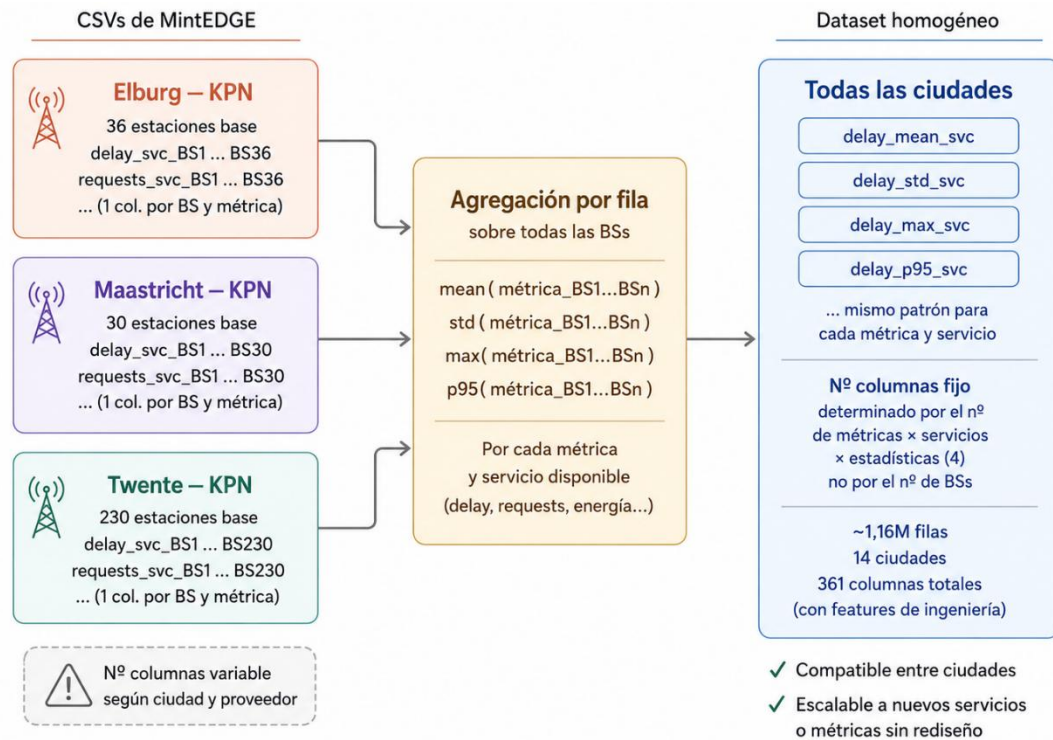


Figura 4.2. Homogeneización del conjunto de datos mediante agregación estadística por estación base. Fuente: elaboración propia a partir de los datos generados por MintEDGE [27].

Las variables globales independientes de la topología (por ejemplo, `active_users`, `total_requests`, `total_rejected`, `dynamic_W_servers`, `idle_W_servers`, `W_links`) se conservan en su formato original. Adicionalmente se añaden tres variables de estructura del escenario: `num_base_stations`, `num_servers` y `server_ratio`, que caracterizan el tamaño y la densidad de infraestructura. Un mismo valor de demanda media tiene distinto significado en una red de 10 estaciones base que en una de 200, por lo que el modelo necesita este contexto para interpretar correctamente las métricas agregadas.

El proceso genera las siguientes familias de variables:

- Variables de tráfico por servicio (`requests_{svc}_total/mean/std/max/p95`, `requests_{svc}_share`, `rejected_{svc}_total/mean/std/max/p95`):

colapsan las columnas por estación base de cada servicio en cinco estadísticos independientes del tamaño de la red. La desviación típica permite al modelo diferenciar entre una carga homogénea y la presencia de puntos de congestión. Se calculan de forma independiente para cada uno de los tres servicios (**connected_vehicles**, **augmented_reality** y **video_analysis**), obteniendo estadísticos separados por servicio, dado que cada uno presenta volúmenes de tráfico y patrones de comportamiento propios que no deben mezclarse en una agregación conjunta.

- **Variables de latencia por servicio** (**delay_{svc}_mean/std/max/p95/weighted_mean**, **max_delay_{svc}_mean/std/max/p95**, **hot_bs_ratio_{svc}**): agregan el estado del retardo extremo a extremo para cada servicio. El percentil 95 y el máximo son los estadísticos más relevantes para la predicción de QoS ya que determinan las violaciones de los acuerdos de nivel de servicio.
- **Variables de utilización de servidores** (**server_util_mean/std/max/p90**, **share_servers_over_80**, **share_servers_over_90**): resumen el estado de carga de los servidores *edge*. Se calcularon los umbrales de saturación ponderando la fracción de servidores que superan el 80% y 90% de uso, identificando cuellos de botella locales que las medias globales suelen enmascarar.

4.2.3 Ingeniería de características

Partiendo del conjunto de datos homogeneizado, esta etapa crea variables adicionales que no existían en el CSV original. Estas nuevas variables tienen como objetivo capturar la dinámica temporal y las interdependencias del sistema. La ingeniería de características, también encuadrada en la fase de preprocesado del ciclo de vida del ML, aborda estas limitaciones mediante cinco grupos de variables:

- **Variables de normalización global** (**rejection_rate**, **requests_per_user**, **requests_per_bs**, **requests_per_server**, **total_W**, **W_per_request**): normalizan métricas globales respecto al tamaño del escenario. Por ejemplo, **requests_per_server** expresa la carga relativa por servidor independientemente del número total de servidores del escenario, mientras que **W_per_request** relaciona el consumo energético con la demanda atendida, actuando como indicador

indirecto de saturación. Estas variables se incluyen como candidatas en el proceso de selección de variables, que determinará cuáles resultan relevantes para cada variable objetivo.

- **Variables de interacción** (`capacity_margin`, `load_imbalance`, `server_pressure_index`, `server_delay_interaction`, `congestion_index`, `backhaul_pressure`, `backhaul_to_compute_ratio`, `backhaul_energy_share`): combinan variables existentes para capturar relaciones que de forma aislada no revelan el estado real del sistema. `capacity_margin` expresa el margen restante hasta la saturación del servidor más cargado; `congestion_index` combina la tasa de rechazo con la utilización máxima del servidor, detectando instantes en que ambos factores son elevados simultáneamente; `server_delay_interaction` relaciona la utilización máxima con el retardo, señalando los momentos en que saturación y latencia elevada se producen a la vez.
- **Variables de QoS** (`unsatisfied_{svc}`, `unsatisfied_rate_{svc}`, `unsatisfied_total`, `unsatisfied_rate`, `satisfied_requests`, `satisfied_rate`, `satisfied_rate_over_accepted`, `satisfied_rate_over_total`, `qos_failure_rate`, `delay_mean_all`, `delay_p95_all`, `delay_max_all`, `max_delay_all`, `max_delay_p95_all`, `delay_mean_weighted_all`): métricas de QoS agregadas, necesarias tanto como descriptores del estado del sistema como para la definición de los objetivos de predicción. Su cálculo requiere una corrección del desfase temporal inherente al simulador, descrita en la Sección 4.2.4.
- **Variables temporales:** calculadas de forma agregada por homogeneización y divididas en tres tipos principales. Los valores históricos de cada variable, identificados en el conjunto de datos mediante el sufijo `_lag_`, se calculan en los pasos 1, 2, 4, 6, 8 y 10 y aportan al modelo memoria inmediata. Las medias móviles, identificadas mediante el sufijo `_roll_mean_` y calculadas con ventanas de 5, 10 y 30 segundos, suavizan fluctuaciones puntuales y capturan la tendencia a corto y medio plazo. Las diferencias temporales, identificadas mediante el sufijo `_diff_` y calculadas en los pasos 1, 5 y 10, expresan la velocidad de cambio de cada variable. A partir de estas últimas se derivan adicionalmente `delay_acceleration` y `unsatisfied_acceleration`,

que miden la segunda derivada del retardo máximo y de la tasa de insatisfacción, respectivamente, capturando si el deterioro del sistema se está acelerando. Todas las variables temporales se calculan dentro de cada simulación por separado para evitar mezclar instantes de ciudades distintas.

- **Variables temporales cíclicas (`time_sin`, `time_cos`):** codificación cíclica del instante de simulación mediante funciones seno y coseno sobre el período de 24 horas, que permiten al modelo aprender los patrones diarios sin que exista una discontinuidad artificial entre el final y el inicio del día.

El conjunto de datos definitivo queda homogéneo respecto a la topología, aunque su dimensionalidad depende del número de servicios definidos en la simulación, ya que una parte de las variables se genera por servicio. En el caso de MintEDGE, los tres servicios configurados por defecto (`connected_vehicles`, `augmented_reality` y `video_analysis`) determinan dicha dimensionalidad, si bien la incorporación de nuevos servicios ampliaría el número de columnas de forma proporcional. El resultado incluye todas las variables candidatas generadas, si bien no todas tendrán la misma relevancia predictiva para cada variable objetivo. La importancia de cada variable se analiza en la etapa de selección de variables de la Sección 4.4, donde se determina el subconjunto óptimo para cada modelo y variable objetivo a partir de distintos métodos de correlación estadística.

4.2.4 Alineación temporal de métricas de QoS

Durante la construcción del conjunto de datos se detectó que determinadas métricas de QoS producían valores negativos en algunos instantes de simulación, lo cual es físicamente imposible para una tasa definida en el intervalo $[0, 1]$. El problema afectaba a las métricas `unsatisfied_rate`, `satisfied_rate` y `qos_failure_rate`, tanto en su versión global como por servicio.

El análisis del código fuente de MintEDGE reveló que la causa se encuentra en una asincronía del reloj de simulación. Cuando llegan peticiones en el segundo T , el simulador registra inmediatamente `total_requests` y `total_rejected` en `kpis[env.now] = T` mediante las funciones `_register_requests()` y `register_rejections()`. Sin embargo, la evaluación de si cada petición ha cumplido o superado el retardo máximo se realiza dentro de `complete_req()`, que introduce un retardo de simulación (`yield`

`env.timeout(1)`). Como consecuencia, cuando las peticiones insatisfechas (`unsatisf_req_{srv}`) se consolidan en el archivo, el reloj de simulación ya ha avanzado a $T+1$, quedando registradas en esa fila. La **Figura 4.3** ilustra este desfase temporal entre el registro de peticiones y la evaluación de la QoS.

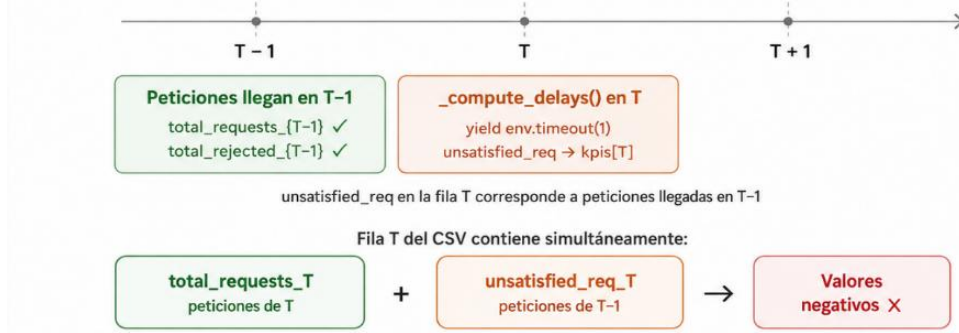


Figura 4.3. Desfase temporal en el registro de métricas de QoS en MintEDGE y corrección aplicada. Fuente: elaboración propia a partir del análisis del código de MintEDGE [27].

El resultado es que cada fila T del CSV contiene simultáneamente `total_requests` y `total_rejected` correspondientes a las peticiones que llegaron en T , y `unsatisf_req_{srv}` correspondiente a las peticiones que llegaron en $T-1$ y fueron evaluadas en T . Las columnas de retardo (`delay_{BS}_{svc}` y `max_delay_{BS}_{svc}`) presentan el mismo desfase por idéntica razón.

Aplicar la fórmula de satisfacción fila a fila sin tener en cuenta este desfase mezcla valores de dos instantes temporales distintos. Para simplificar la notación, se define R_T como el número total de peticiones registradas en el instante T , J_T como el número de peticiones rechazadas en T , U_T como el número de peticiones insatisfechas registradas en T y S_T como el número de peticiones satisfechas. Sin corregir el desfase temporal, la satisfacción se calcularía como:

$$S_T = R_T - J_T - U_T$$

En segundos donde el volumen de tráfico varía bruscamente entre $T-1$ y T , el número de peticiones insatisfechas procedentes de $T-1$ puede superar al número de peticiones aceptadas en T , dando lugar a valores negativos.

Para corregir este sesgo, se aplicó un desplazamiento de un paso mediante `shift(1)` sobre las métricas de entrada del segundo anterior, reformulando el cálculo de satisfacción con las magnitudes del instante temporal correspondiente:

$$S_T = (R_{\{T-1\}} - J_{\{T-1\}} - U_T$$

$$SR_T = \frac{S_T}{R_{\{T-1\}}}$$

donde $R_{\{T-1\}}$ representa las peticiones totales del instante anterior, $J_{\{T-1\}}$ las peticiones rechazadas del instante anterior, U_T las peticiones insatisfechas registradas en el instante actual, S_T las peticiones satisfechas corregidas y SR_T la tasa de satisfacción resultante.

4.3 Preparación del problema predictivo

Una vez construido el conjunto de datos homogéneo, el siguiente paso consiste en preparar el problema predictivo. En esta sección se define la variable objetivo y se justifica su elección, se analiza la distribución de los datos para comprender su variabilidad, se describe la estrategia de división del conjunto de datos en conjuntos de entrenamiento, validación y prueba, y se detalla el preprocesado antes del entrenamiento.

4.3.1 Definición de variables objetivo

Antes de fijar la variable objetivo, se realizó una valoración de distintos candidatos atendiendo a su utilidad para el orquestador en términos de QoS. Se consideraron métricas como la tasa de peticiones rechazadas y la tasa de peticiones satisfechas total, entre otras. Finalmente se seleccionó como variable principal a predecir la tasa de peticiones satisfechas sobre el total de peticiones aceptadas. Al usar como denominador únicamente las peticiones que el sistema aceptó para procesar, excluye el efecto de las peticiones rechazadas por falta de capacidad y se centra exclusivamente en si las peticiones atendidas cumplieron el presupuesto de retardo.

El horizonte de predicción elegido es de 5 segundos ($t+5$). Este horizonte proporciona al orquestador un margen suficiente para actuar antes de que se produzca la degradación prevista, a diferencia de un horizonte de 1 segundo que resultaría insuficiente para completar un ciclo de reasignación de recursos.

La **Figura 4.4** ilustra la relación temporal entre las características de entrada y la variable objetivo, confirmando que no existe fuga de información entre los datos utilizados para la predicción y el valor a predecir.

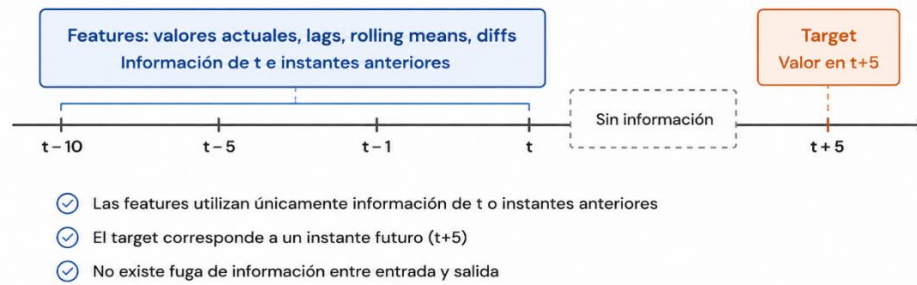


Figura 4.4. Esquema temporal de construcción de características y variable objetivo, donde las variables de entrada utilizan información del instante actual o anterior y la variable objetivo corresponde a un instante futuro. Fuente: elaboración propia.

Se definen cuatro variables objetivo. La primera tiene carácter global y no representa un servicio adicional, sino una medida agregada de la QoS del escenario completo, considerando conjuntamente las peticiones de todos los servicios y estaciones base. Las tres restantes se definen de forma específica para cada servicio (**connected_vehicles**, **augmented_reality** y **video_analysis**), lo que permite evaluar el comportamiento del predictor de manera independiente para cada tipo de carga.

Para comprender la variabilidad de cada variable objetivo y detectar posibles desequilibrios, se analizó la distribución de cada una de ellas mediante histogramas de frecuencia, calculando la media, la desviación típica y el porcentaje de valores cero. Este último incluye los valores negativos residuales descritos en la Sección 4.2.4, redondeados a cero para no eliminar registros de la serie temporal. La **Figura 4.5** muestra la distribución de las cuatro variables.

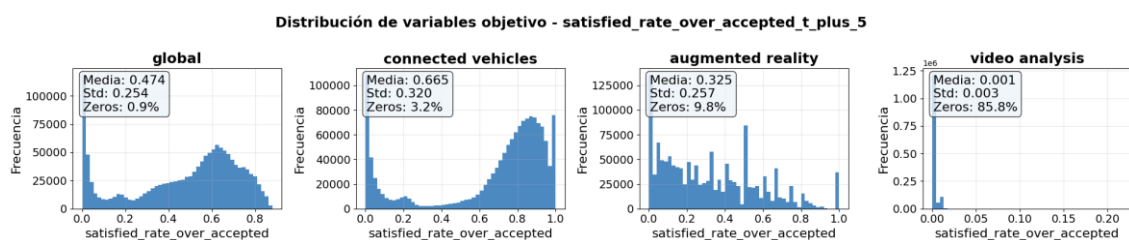


Figura 4.5. Distribución de las cuatro variables objetivo consideradas en el estudio. Fuente: elaboración propia.

Como se puede observar, las variables **global** y **connected_vehicles** presentan distribuciones bimodales con estructura clara, medias de 0.474 y 0.665, y un porcentaje de ceros reducido (0.9% y 3.2%), lo que anticipa una mayor facilidad de predicción. **augmented_reality** muestra una dispersión irregular y un 9.8% de ceros, reflejando

un comportamiento más volátil. Por último, **video_analysis** es el caso más extremo, con una media de 0.001 y un 85.8% de valores cero, indicando que este servicio opera sin peticiones activas durante la mayor parte de la simulación, lo que limita severamente la capacidad del modelo para aprender patrones significativos.

4.3.2 División del conjunto de datos

Como parte del proceso de entrenamiento, uno de los pasos previos fundamentales consiste en dividir el conjunto de datos en: entrenamiento, validación y prueba, con proporciones del 70%, 10% y 20% respectivamente. Para mitigar los riesgos de sobreajuste y garantizar una validación robusta, el método de partición se dividió en dos partes según la arquitectura del modelo.

División tabular. Dado que los algoritmos tabulares basados en árboles procesan las muestras de forma independiente, es posible aplicar un barajado aleatorio sobre el conjunto de datos completo multi-ciudad. Esto asegura que los tres subconjuntos contengan una representación homogénea de todas las ciudades, operadores y niveles de carga simulados. En todo caso, las variables de entrada se construyen únicamente con información disponible en el instante actual o en instantes anteriores, mientras que la variable objetivo corresponde a un instante futuro, evitando así fuga de información temporal y permitiendo evaluar la generalización sobre muestras multi-ciudad.

División secuencial. Las arquitecturas recurrentes exigen preservar el orden cronológico de las series temporales. Para evitar evaluar el modelo en una única franja horaria común, se ha diseñado un esquema de partición rotativo inter-ciudad. El bloque de prueba de cada ciudad se desplaza un 10% respecto a la ciudad anterior, de forma que el prueba agregado no queda concentrado en una única franja horaria sino distribuido a lo largo de diferentes instantes temporales. Los conjuntos de entrenamiento y validación se extraen cronológicamente del resto de datos de cada ciudad. Con este esquema se proporciona una evaluación más robusta de la capacidad de generalización del modelo sobre instantes temporales no vistos. La **Figura 4.6** ilustra este esquema de rotación.

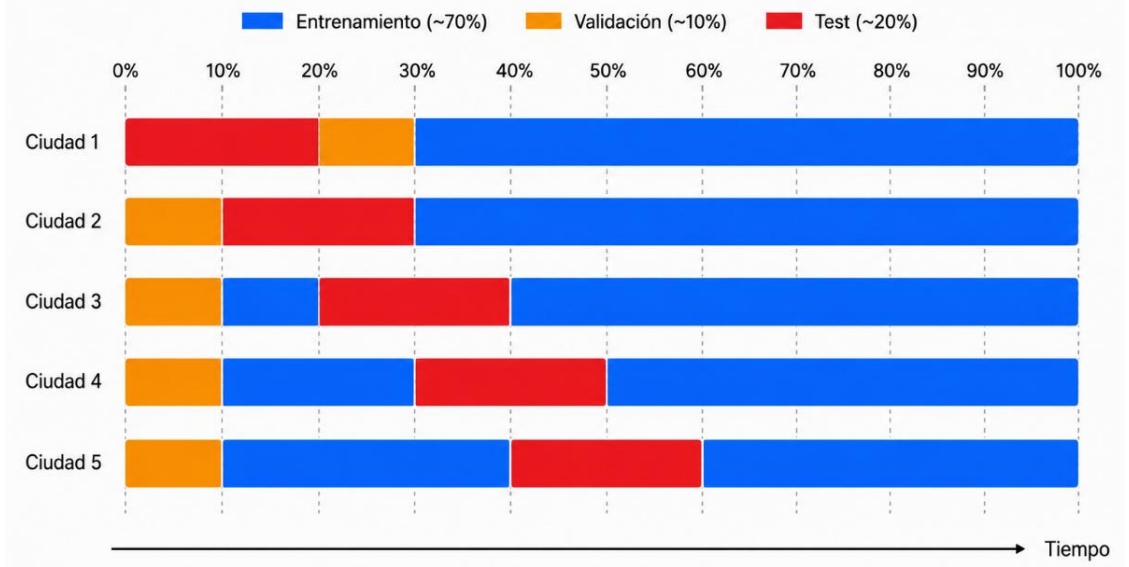


Figura 4.6. Esquema de división temporal rotativa por ciudad para modelos secuenciales. Fuente: elaboración propia.

4.3.3 Selección de variables

Con el objetivo de identificar las variables de entrada con mayor poder predictivo y disminuir la redundancia en el espacio de entrada, se aplican los cinco métodos de selección de variables descritos en la Sección 2.5.2: Pearson, Spearman, Kendall, **f_regression** y CFS Spearman. Todos los métodos se aplican exclusivamente sobre el conjunto de entrenamiento para evitar cualquier filtración de información hacia los conjuntos de validación y prueba.

Formalmente, la selección de variables consiste en obtener, para cada variable objetivo, un subconjunto $S_k \subseteq F$, donde F representa el conjunto completo de variables candidatas y S_k el subconjunto seleccionado. En este trabajo se consideran tres tamaños de selección, de forma que $|S_k| \in \{5, 10, 20\}$.

Para cada método y variable objetivo se obtienen tres configuraciones de selección, formadas por 5, 10 y 20 variables. El análisis de solapamiento para descartar configuraciones redundantes se realiza sobre las selecciones de 5 variables, ya que comparar conjuntos mayores generaría un elevado número de configuraciones prácticamente idénticas en sus variables más relevantes, reduciendo la diversidad experimental. Cuando dos métodos seleccionan las mismas cinco variables para una misma variable objetivo, se consideran equivalentes y uno se descarta. Este proceso se ejecuta de forma independiente para modelos tabulares y secuenciales, dado que sus conjuntos de entrenamiento no son equivalentes. Para los modelos secuenciales, la

selección se aplica además únicamente sobre variables no temporales, ya que estos modelos aprenden las dependencias temporales internamente. Las configuraciones diferenciadas resultantes son las que se utilizan en el entrenamiento.

4.4 Modelos predictivos y configuración experimental

Con el conjunto de datos preparado y los subconjuntos de variables definidos, el siguiente paso consiste en el entrenamiento y evaluación de los modelos predictivos. Para abordar el problema de predicción de KPIs de QoS se han empleado dos familias de modelos en su variante de regresión, dado que la variable objetivo es continua en el intervalo $[0, 1]$: modelos tabulares basados en árboles de decisión y modelos secuenciales basados en redes neuronales profundas. Los modelos tabulares son especialmente adecuados para este tipo de datos porque pueden explotar directamente las variables de contexto temporal (valores históricos, diferencias temporales y medias móviles) creadas en la fase de ingeniería de características, tratando cada fila como una muestra independiente enriquecida con información del pasado. Los modelos secuenciales, por su parte, aprenden las dependencias temporales de forma implícita a partir de ventanas de instantes consecutivos. En este caso, cada muestra se representa como una ventana temporal de longitud L , formada por observaciones anteriores al instante de predicción, es decir, $X_t = [x_{t-L+1}, x_{t-L+2}, \dots, x_t]$. En este trabajo se utiliza $L = 10$, por lo que cada predicción se construye a partir de los diez instantes anteriores para estimar la variable objetivo en el horizonte futuro $t + 5$.

La **Figura 4.7** ilustra la diferencia fundamental en la representación de la entrada entre ambas familias de modelos.

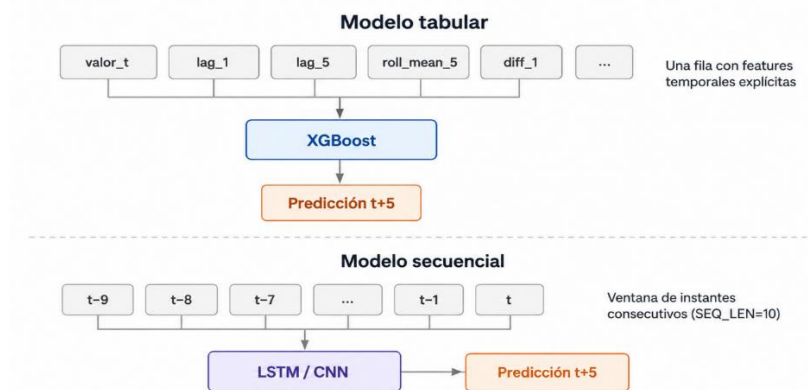


Figura 4.7. Diferencia en la representación de la entrada entre modelos tabulares y secuenciales. Fuente: elaboración propia.

4.4.1 Modelos tabulares

Se entrenaron cuatro modelos de regresión basados en árboles que cubren distintos niveles de complejidad y enfoques: DecisionTree, RandomForest, XGBoost y LightGBM.

El entrenamiento se parametrizó con una configuración base de **n_estimators=100** para los modelos de conjunto (*ensemble*) y los parámetros por defecto de scikit-learn para DecisionTree y RandomForest [64]. Para cada modelo se prueba cada una de las configuraciones de selección de variables obtenidas en la fase anterior, con 5, 10 y 20 variables por cada método no descartado en el análisis de solapamiento, de forma que el espacio experimental cubre tanto la influencia del modelo como la de la selección de variables.

Adicionalmente, se evalúa el impacto de variar los hiperparámetros más determinantes en la complejidad del conjunto de árboles para XGBoost y LightGBM, ya que son los modelos que ofrecen mayor margen de ajuste con un único parámetro de control. En XGBoost se prueban valores de **n_estimators** de 50, 200 y 500, que controlan el número de árboles del conjunto y, por tanto, el equilibrio entre capacidad de aprendizaje y riesgo de sobreajuste [41]. En LightGBM se prueban valores de **num_leaves** de 16, 64 y 128, parámetro que determina la complejidad máxima de cada árbol individual y tiene un impacto directo sobre la capacidad del modelo para capturar relaciones no lineales en los datos [42]. DecisionTree y RandomForest se evalúan únicamente con su configuración base, dado que sus principales hiperparámetros de regularización presentan un impacto menos determinante en problemas con un número reducido de variables de entrada como los que produce la selección de variables [64].

4.4.2 Modelos secuenciales

Se entrenaron dos arquitecturas de red neuronal para series temporales como son LSTM y CNN 1D. Para este caso, el conjunto de datos se transforma en ventanas temporales deslizantes de longitud definida por el parámetro **SEQ_LEN** construidas por separado para cada ciudad, preservando el orden temporal interno de cada simulación. Cada ventana contiene **SEQ_LEN** instantes de variables no temporales y el valor a predecir corresponde al instante inmediatamente siguiente [65]. El análisis de solapamiento entre métodos de selección se realiza de nuevo sobre el conjunto de entrenamiento secuencial, siguiendo el procedimiento de la Sección 4.3.3, y las

configuraciones resultantes con 5 y 10 variables seleccionadas son las que se utilizan para construir las secuencias de entrada.

Con el objetivo de optimizar el coste computacional del entrenamiento, se realizaron experimentos previos sobre la variable objetivo con mayor volatilidad (**augmented_reality**) para determinar las configuraciones óptimas de **dropout** y **SEQ_LEN**. En cada experimento se varía un único parámetro manteniendo el resto fijos. Con ello, se determinaron las constantes de diseño que se describen a continuación.

Selección de la tasa de *dropout*

El *dropout* se emplea como técnica de regularización en redes neuronales, ya que durante el entrenamiento desactiva aleatoriamente una fracción de neuronas con el objetivo de reducir el sobreajuste y mejorar la capacidad de generalización del modelo [66]. Se comparan tres valores (0.1, 0.2 y 0.3) con **units=64** y **SEQ_LEN=10**. **dropout=0.2** obtiene el mejor **score**, mientras que **dropout=0.3** produce un deterioro significativo del rendimiento al eliminar demasiada información en cada paso. Se fijó **dropout=0.2** para el resto de experimentos.

Selección de **SEQ_LEN**

Se comparan tres valores (5, 10 y 20 segundos). Aunque **SEQ_LEN=20** obtiene un R^2 marginalmente superior (0.7335 frente a 0.7226 de **SEQ_LEN=10**), el tiempo de entrenamiento se duplica y experimentos preliminares con arquitecturas más complejas mostraron tiempos que harían inviable el entrenamiento con **SEQ_LEN=20**. Se fija **SEQ_LEN=10** como equilibrio entre capacidad de captura temporal y coste computacional [65].

Una vez fijados los hiperparámetros estructurales (**dropout=0.2** y **SEQ_LEN=10**), se entrena LSTM con una capa recurrente seguida de **Dropout**, una capa densa de 32 neuronas con activación *Rectified Linear Unit* (ReLU) y salida lineal. Por su parte, la arquitectura CNN 1D se entrena con una capa de convolución unidimensional **Conv1D** con **kernel_size=3**, que define el tamaño de la ventana del filtro convolucional, seguida de una capa de reducción temporal **MaxPooling1D**, una capa de aplanamiento **Flatten**, **Dropout** y las mismas capas densas [34]. Ambas arquitecturas se compilan con el optimizador Adam, la función de pérdida de error cuadrático medio (*Mean Squared Error*, MSE), un máximo de 50 épocas, **batch_size=256**, que determina el número de muestras procesadas por lote, y el mecanismo de parada temprana **EarlyStopping**

con **patience=5**, que detiene el entrenamiento tras cinco épocas consecutivas sin mejora [67]. Se prueban tres tamaños de capa (32, 64 y 128 unidades o filtros), descartando 256 unidades a partir de experimentos preliminares que mostraron que no aporta mejora apreciable en ninguna variable objetivo y multiplica por cinco el tiempo de entrenamiento respecto a 128 unidades.

4.5 Organización de experimentos y criterio de selección

El diseño experimental sigue una organización lineal estructurada en torno a un flujo adaptable, de forma que el proceso completo puede aplicarse a cualquier variable objetivo modificando únicamente la variable objetivo en la configuración. Para cada uno de ellos, el espacio de experimentos resulta de la combinación sistemática de modelos, métodos de selección de variables, tamaños de selección de 5, 10 y 20 variables, variaciones de hiperparámetros y, en el caso de los modelos secuenciales, configuraciones de arquitectura. Todos los resultados se registran en estructuras de datos que consolidan las métricas de cada experimento.

Las métricas empleadas para evaluar cada experimento son RMSE, MAE y R^2 , definidas en la Sección 2.5.3, y se calculan en todos los casos sobre el conjunto de validación. Adicionalmente, se registra el tiempo de inferencia por muestra (ms/muestra) como métrica complementaria. En un sistema de orquestación en tiempo real como MintEDGE, el predictor debe producir una estimación antes de que expire el intervalo de orquestación; un modelo que tarde más en predecir de lo que tarda el orquestador en actuar carecería de utilidad práctica, por lo que la eficiencia computacional del modelo es un criterio relevante aunque secundario frente a la precisión.

Para seleccionar automáticamente la mejor configuración por modelo y variable objetivo entre todas las evaluadas, se define formalmente **score**, un sistema de puntuación ponderada que combina las tres métricas anteriores. Dado que las métricas originales operan en escalas diferentes, el MAE y el tiempo de inferencia se normalizaron linealmente en el intervalo [0, 1] dentro de cada variable objetivo:

$$x_{norm} = \frac{x - x_{\{min\}}}{x_{\{max\}} - x_{\{min\}}}$$

La función que calcula **score** se define como:

$$score = 0.6 * R^2 - 0.3 * MAE_{norm} - 0.1 * t_{norm}$$

La distribución de los coeficientes se basa en las siguientes razones. El coeficiente de determinación R^2 recibe el peso más alto (0.6) porque mide la proporción de varianza explicada por el modelo, priorizando modelos que sigan fielmente el comportamiento real del sistema a lo largo del tiempo [59]. A diferencia del MAE, R^2 es más sensible a errores grandes y esporádicos, ya que penaliza de forma cuadrática las desviaciones respecto al valor real. En el contexto del orquestador, esta sensibilidad es deseable ya que un error puntual elevado en la predicción puede derivar en una mala decisión de asignación de recursos en ese instante. Las variables objetivo son tasas definidas en el intervalo $[0, 1]$, por lo que los valores de MAE en modelos bien entrenados sobre grandes volúmenes de datos son inherentemente pequeños (inferiores a 0.1 en la mayoría de casos), lo que hace que diferencias mínimas en MAE entre modelos tengan escasa relevancia práctica. No obstante, el MAE normalizado recibe el segundo peso (0.3) porque, a diferencia del RMSE, no eleva los errores al cuadrado y por tanto ofrece una medida del error medio en las mismas unidades de la variable objetivo, siendo más interpretable y menos sensible a valores atípicos puntuales [58]. El tiempo de inferencia normalizado recibe el peso menor (0.1), actuando como criterio de desempate. Ante dos modelos con R^2 y MAE similares, se prioriza el que produce la predicción de forma más rápida y eficiente para el orquestador.

La evaluación sobre el conjunto de prueba queda reservada para el capítulo 5, donde se aplica únicamente la configuración con mayor **score** en validación para cada variable objetivo. De este modo se garantiza que el conjunto de prueba permanece completamente inédito durante todo el proceso de experimentación, proporcionando una estimación fiable y no sesgada del rendimiento final del modelo [68].

Capítulo 5

Evaluación experimental del servicio predictivo

5.1 Introducción

El presente capítulo recoge los resultados obtenidos a lo largo del proceso experimental descrito en el capítulo anterior, evaluando el impacto de las distintas configuraciones sobre el rendimiento de los modelos y analizando su comportamiento frente a las variables objetivo. Dado el amplio espacio de configuraciones explorado, que combina múltiples modelos, métodos de selección de variables, tamaños de selección e hiperparámetros, los resultados se presentan de forma estructurada partiendo de lo más granular hasta llegar a la evaluación global. Se analiza en primer lugar el impacto de los métodos de selección de variables, seguido de los resultados individuales de los modelos tabulares y secuenciales, la comparación entre ambas familias y, finalmente, la evaluación definitiva sobre el conjunto de prueba con los modelos seleccionados.

Todos los resultados intermedios se obtienen sobre el conjunto de validación, que el modelo no ha visto durante el entrenamiento, reservando el conjunto de prueba exclusivamente para la evaluación final. Este enfoque permite estimar el rendimiento del predictor sin que el conjunto de prueba influya en las decisiones tomadas durante la experimentación. Dado que las variables objetivo presentan distribuciones y naturalezas

distintas, tal y como se analizó en la Sección 4.3.1, se espera que **global** y **connected_vehicles** sean los más estables, que **augmented_reality** presente mayor dificultad predictiva y que **video_analysis** requiera una interpretación cautelosa por su distribución concentrada en valores próximos a cero.

5.2 Evaluación de modelos tabulares

En esta sección se presentan los resultados obtenidos por los cuatro modelos tabulares (DecisionTree, RandomForest, XGBoost y LightGBM) sobre el conjunto de validación, evaluados con las distintas configuraciones de selección de variables resultantes del proceso descrito en la Sección 4.3.3. El análisis comprende la comparativa base por modelo y variable objetivo, el impacto del ajuste de hiperparámetros en los modelos de gradiente (XGBoost y LightGBM) y la ponderación de importancia de variables en los modelos ganadores.

5.2.1 Comparativa por variable objetivo

Los resultados de la configuración base permiten realizar una primera comparación entre modelos tabulares antes del ajuste específico de hiperparámetros. En esta fase se evalúa, para cada variable objetivo y cada modelo, el efecto de combinar distintos métodos de selección de variables con diferentes tamaños de subconjunto. De este modo, la comparación no se limita al algoritmo empleado, sino que también incorpora la influencia del conjunto de variables utilizado como entrada del modelo y permite observar si determinadas variables objetivo requieren configuraciones más amplias o métodos de selección concretos para obtener un rendimiento adecuado.

Para cada caso se selecciona la combinación que obtiene el mejor rendimiento en validación, considerando conjuntamente la capacidad predictiva y el coste de inferencia. Esta comparación inicial permite analizar la dificultad relativa de cada variable objetivo, comprobar la estabilidad de los modelos frente a distintos subconjuntos de entrada y establecer una referencia común para interpretar posteriormente el efecto del ajuste de hiperparámetros sobre XGBoost y LightGBM. La **Tabla 5.1** resume la configuración seleccionada en cada caso, junto con el R^2 , el MAE y el tiempo de inferencia por muestra.

Tabla 5.1. Mejor configuración base por modelo y variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

Variable objetivo	Modelo	Selección	R ²	MAE	ms/muestra
global	DecisionTree	CFS top20	0.9891	0.0166	0.001775
global	XGBoost	Pearson top20	0.9942	0.0116	0.000255
global	LightGBM	Pearson top10	0.9941	0.0117	0.000397
global	RandomForest	CFS top20	0.9946	0.0118	0.013194
connected_vehicles	DecisionTree	CFS top10	0.9892	0.0182	0.001056
connected_vehicles	XGBoost	Pearson top20	0.9941	0.0134	0.000253
connected_vehicles	LightGBM	Pearson top20	0.9940	0.0134	0.000440
connected_vehicles	RandomForest	CFS top10	0.9946	0.0130	0.012339
augmented_reality	DecisionTree	Spearman top5	0.6073	0.1143	0.000623
augmented_reality	XGBoost	Spearman top20	0.7529	0.0864	0.000096
augmented_reality	LightGBM	Spearman top20	0.7477	0.0878	0.000863
augmented_reality	RandomForest	Spearman top20	0.7535	0.0856	0.011133
video_analysis	DecisionTree	Spearman top10	0.6954	0.0005	0.000487
video_analysis	XGBoost	Spearman top20	0.8323	0.0004	0.000107
video_analysis	LightGBM	CFS top20	0.8354	0.0004	0.000458
video_analysis	RandomForest	Spearman top20	0.8363	0.0004	0.003066

Para las variables objetivo **global** y **connected_vehicles**, los cuatro modelos obtienen un rendimiento elevado, con valores de R² superiores a 0.98. Los tres modelos de conjunto, XGBoost, LightGBM y RandomForest, alcanzan resultados casi idénticos entre sí, con R² en torno a 0.994 y MAE próximo a 0.012 para **global** y 0.013 para **connected_vehicles**. DecisionTree queda consistentemente por debajo, aunque mantiene un rendimiento aceptable en estas variables objetivo con estructura más clara.

Las diferencias entre modelos se hacen especialmente evidentes en **augmented_reality**. En este caso, DecisionTree experimenta una caída notable de rendimiento, con un R² claramente inferior al obtenido por los modelos de conjunto, mientras que XGBoost, LightGBM y RandomForest se sitúan en torno a 0.74-0.75. Desde el punto de vista de la capacidad explicativa medida mediante R², **augmented_reality** es el objetivo más difícil de predecir para los modelos tabulares, ya que incluso el mejor resultado alcanzado queda por debajo del obtenido en **global**, **connected_vehicles** y **video_analysis**. Además, su MAE se sitúa en torno a 0.087-0.090 para los modelos de conjunto, un orden de magnitud superior al observado en **global** y **connected_vehicles**.

En cuanto a **video_analysis**, los modelos de conjunto obtienen valores de R² en torno a 0.82-0.84, significativamente superiores a los de DecisionTree. No obstante, el

MAE para esta variable objetivo es extremadamente bajo debido a la naturaleza de su distribución, dominada por valores próximos a cero, por lo que esta métrica debe interpretarse con cautela en este caso.

En cuanto a los métodos de selección de variables, se observa que para las variables objetivo más fáciles de predecir, **global** y **connected_vehicles**, la elección del método tiene un impacto reducido sobre el rendimiento. Sin embargo, en **augmented_reality** y **video_analysis** las diferencias entre métodos resultan más apreciables, con una mayor presencia de configuraciones basadas en Spearman y tamaños de selección de 20 variables entre los mejores resultados.

En cuanto al tiempo de inferencia, los modelos tabulares presentan tiempos muy reducidos por muestra. XGBoost y LightGBM se sitúan entre las opciones más eficientes, mientras que RandomForest muestra un coste mayor por el uso de múltiples árboles, una diferencia relevante en sistemas de orquestación con predicciones continuas.

5.2.2 Impacto del ajuste de hiperparámetros

Tal y como se describe en la Sección 4.4.1, se evalúa si el ajuste de hiperparámetros en los algoritmos de gradiente aporta un valor predictivo más significativo respecto a la configuración base. En concreto, para XGBoost se probaron valores de **n_estimators** iguales a 50, 200 y 500, mientras que para LightGBM, se probaron valores de **num_leaves** iguales a 16, 64 y 128. La **Tabla 5.2** recoge el mejor R^2 obtenido con la configuración base frente al mejor R^2 con hiperparámetros ajustados para cada modelo y variable objetivo.

Tabla 5.2. Impacto de hiperparámetros en modelos de gradiente sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

Variable objetivo	XGBoost	XGBoost HP	LightGBM	LightGBM HP
global	0.9942	0.9944 (n_estimators=500)	0.9941	0.9943 (num_leaves=128)
connected_vehicles	0.9941	0.9945 (n_estimators=500)	0.9940	0.9942 (num_leaves=128)
augmented_reality	0.7529	0.7540 (n_estimators=200)	0.7477	0.7543 (num_leaves=128)
video_analysis	0.8323	0.8331 (n_estimators=50)	0.8354	0.8365 (num_leaves=128)

Nota: HP: hiperparámetros.

Los resultados muestran que el impacto del ajuste es reducido en ambos modelos y para todas las variables objetivo. En **global** y **connected_vehicles**, las mejoras no superan 0.004 en R^2 , resultado coherente con la literatura sobre *gradient boosting*, que indica que el error de prueba tiende a estabilizarse a partir de un número relativamente reducido de árboles en conjuntos de datos de gran volumen [69]. En **augmented_reality** y **video_analysis** las mejoras son del mismo orden, lo que sugiere que la dificultad de estas variables objetivo está más relacionada con su propia naturaleza y con la información disponible en las variables de entrada que con la capacidad del modelo. Aun así, las configuraciones ajustadas con mejor R^2 se incorporan al conjunto de candidatos, ya que mantienen tiempos de inferencia del mismo orden de magnitud.

5.2.3 Selección del mejor modelo tabular y análisis de variables

Clasificación de modelos y aplicación de **score**

Una vez evaluadas todas las configuraciones, tanto con parámetros base como con hiperparámetros ajustados, se aplica **score**, definido en la Sección 4.5, para seleccionar de forma automática el mejor modelo por variable objetivo. La **Tabla 5.3** recoge este resumen resultante del conjunto de validación

Tabla 5.3. Clasificación de modelos tabulares por **score** para cada variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

V. objetivo	Modelo	Selección	Hiperparámetros	R^2	MAE	ms/m	score
global	LightGBM	Pearson top20	num_leaves=128	0.9943	0.0115	0.000562	0.5933
global	XGBoost	Pearson top20	n_estimators=100	0.9942	0.0116	0.000255	0.5910
global	RandomForest	Pearson top5	-	0.9936	0.0122	0.007231	0.5100
global	DecisionTree	CFS top20	-	0.9891	0.0166	0.001775	0.3491
cv	LightGBM	Kendall top10	num_leaves=64	0.9939	0.0133	0.000485	0.5822
cv	XGBoost	Kendall top10	n_estimators=50	0.9939	0.0134	0.000127	0.5812
cv	RandomForest	CFS top10	-	0.9946	0.0130	0.012339	0.5035
cv	DecisionTree	CFS top10	-	0.9892	0.0182	0.001056	0.3913
ar	XGBoost	Spearman top20	n_estimators=200	0.7540	0.0859	0.000401	0.4474
ar	LightGBM	Spearman top20	num_leaves=128	0.7543	0.0862	0.000544	0.4441
ar	RandomForest	Pearson top20	-	0.7535	0.0856	0.011133	0.3521
ar	DecisionTree	Spearman top5	-	0.6073	0.1143	0.000623	0.1470
va	XGBoost	Spearman top20	n_estimators=100	0.8323	0.0004	0.000107	0.4989
va	LightGBM	CFS top20	num_leaves=64	0.8359	0.0004	0.000254	0.4970
va	RandomForest	Spearman top20	-	0.8363	0.0004	0.003066	0.4208
va	DecisionTree	Spearman top10	-	0.6954	0.0005	0.000487	0.2564

Nota: cv: **connected_vehicles**; ar: **augmented_reality**; va: **video_analysis**; V: Variable.

Los resultados muestran que LightGBM resulta ganador para las variables objetivo **global** y **connected_vehicles**, mientras que XGBoost lo hace en **augmented_reality** y **video_analysis**, aunque no siempre obtienen el mayor R^2 . En **connected_vehicles**, RandomForest alcanza un R^2 de 0.9946 frente al 0.9939 del ganador, y en **video_analysis** obtiene un R^2 de 0.8363 frente al 0.8323 de XGBoost. Sin embargo, **score** penaliza su mayor tiempo de inferencia, ya que RandomForest construye la predicción como promedio de múltiples árboles independientes, y requiere tiempos entre 25 y 100 veces superiores a los modelos de gradient boosting. En un sistema de orquestación en tiempo real, esta diferencia justifica la selección de LightGBM y XGBoost, que combinan una precisión similar con mayor eficiencia computacional. De la **Figura 5.1** a la **Figura 5.4** se muestra esta comparativa para cada variable objetivo, donde se observa cómo LightGBM o XGBoost obtienen el mayor **score** a pesar de que RandomForest les supera en R^2 en algunos casos.

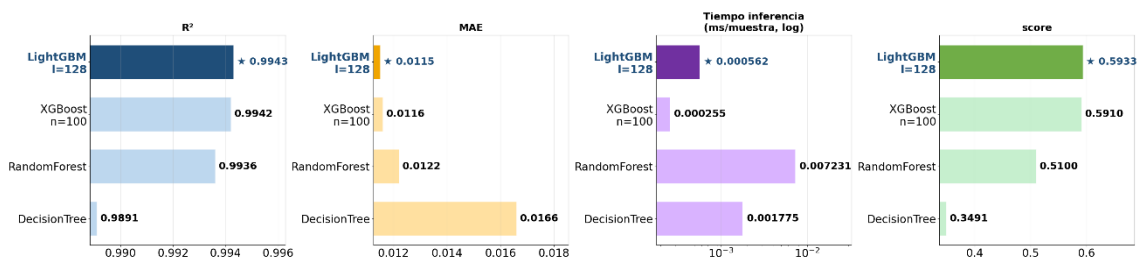


Figura 5.1. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y **score** para la variable objetivo **global**. Fuente: elaboración propia.

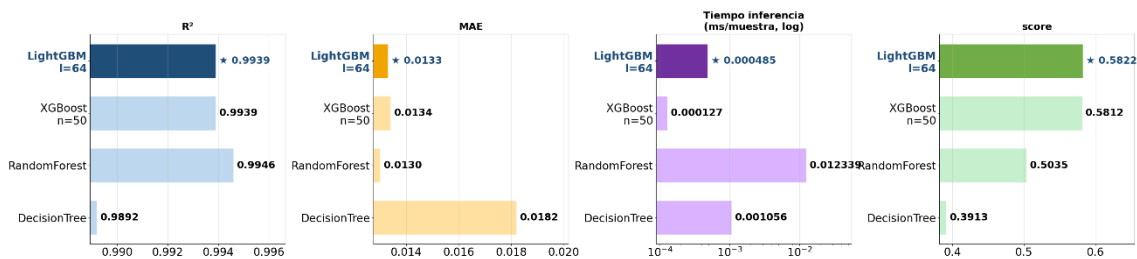


Figura 5.2. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y **score** para la variable objetivo **connected_vehicles**. Fuente: elaboración propia.

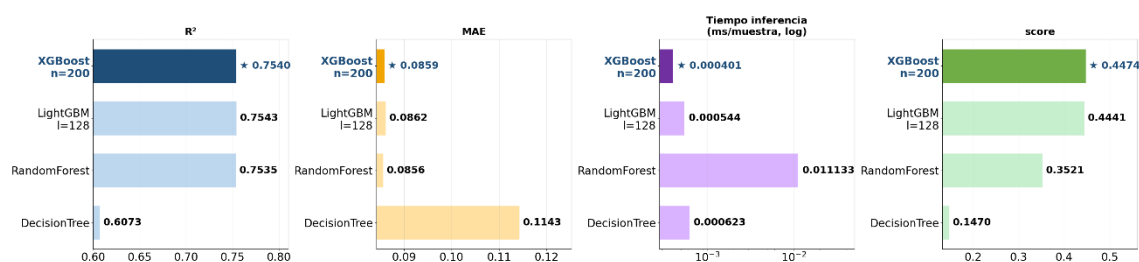


Figura 5.3. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y **score** para la variable objetivo **augmented_reality**. Fuente: elaboración propia.

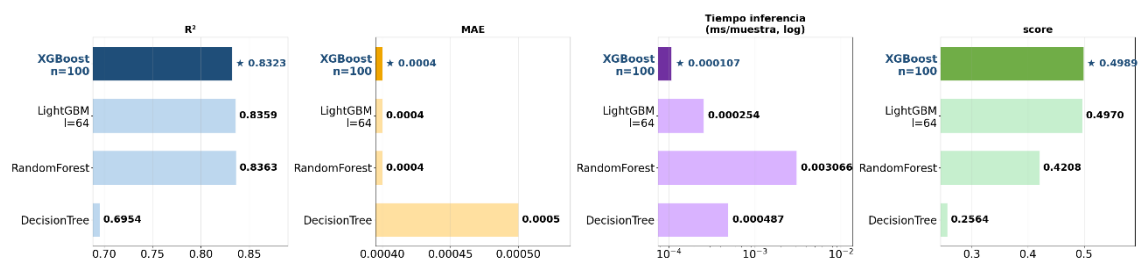


Figura 5.4. Comparativa del modelo ganador frente al resto por R^2 , MAE, tiempo de inferencia y **score** para la variable objetivo **video_analysis**. Fuente: elaboración propia.

Análisis de residuos

Para evaluar la calidad de las predicciones más allá de las métricas agregadas, se analiza la distribución de los errores de predicción (residuos) de cada modelo. Un modelo bien ajustado debe concentrar la mayor parte de sus errores alrededor de cero y no mostrar patrones sistemáticos ni colas excesivamente pronunciadas, entendidas como una acumulación elevada de errores grandes alejados de cero. Esto indica que las desviaciones entre predicción y valor real son reducidas y no siguen una estructura clara que el modelo haya dejado de capturar [70]. Complementariamente, la representación gráfica del valor predicho frente al valor real mediante un diagrama de dispersión permite evaluar visualmente la bondad del ajuste [71].

Para las variables objetivo **global** y **connected_vehicles**, la distribución de residuos de los cuatro modelos presenta un pico muy concentrado en torno a cero con colas simétricas y estrechas, confirmando que los errores son pequeños, aleatorios y sin sesgos sistemáticos. Los diagramas de dispersión correspondientes muestran una nube de puntos muy alineada con la diagonal de predicción perfecta a lo largo de todo el rango de valores de la variable objetivo, lo que corrobora los elevados R^2 obtenidos. Se observa que DecisionTree produce una distribución de residuos ligeramente más ancha que los

tres modelos de conjunto, coherente con su menor R^2 . La Figura 5.5 y la Figura 5.6 muestran estos resultados para `global` y `connected_vehicles` respectivamente.

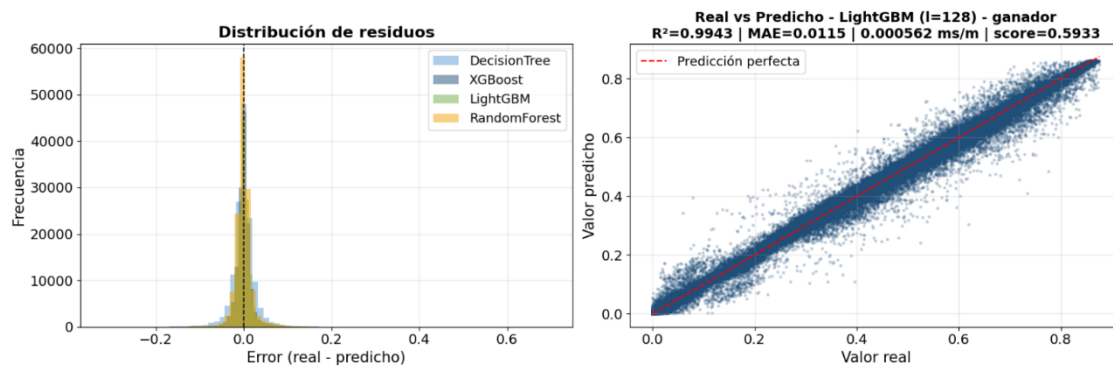


Figura 5.5. Distribución de residuos y diagrama de dispersión para la variable objetivo `global`. Fuente: elaboración propia.

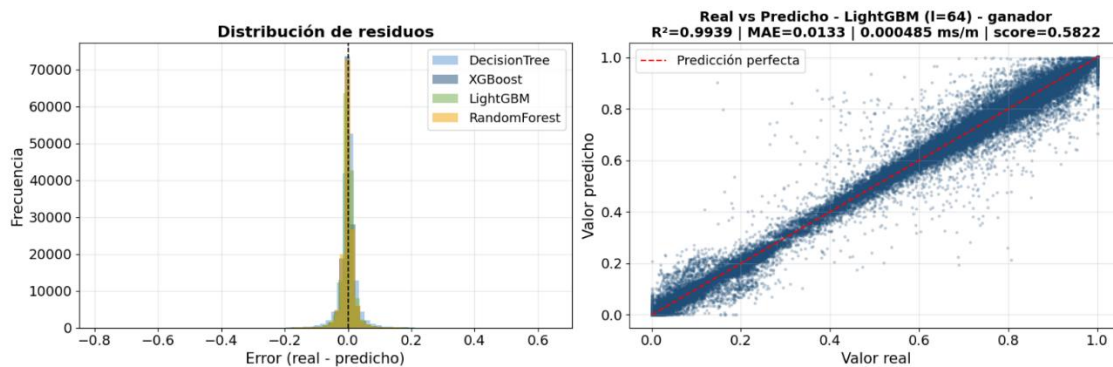


Figura 5.6. Distribución de residuos y diagrama de dispersión para la variable objetivo `connected_vehicles`. Fuente: elaboración propia.

En `augmented_reality`, la distribución de residuos es visiblemente más ancha y con colas más pesadas, especialmente en la cola negativa, lo que indica que el modelo subestima la variable objetivo con mayor frecuencia de lo que la sobreestima. El diagrama de dispersión muestra una mayor dispersión de los puntos respecto a la diagonal, con una concentración apreciable de errores en valores intermedios de la variable objetivo. Este comportamiento es coherente con la naturaleza más irregular de `augmented_reality` y con su R^2 de 0.75, confirmando que es la variable más difícil de predecir entre los que presentan actividad significativa.

En `video_analysis`, la distribución de residuos está concentrada en cero debido a que la gran mayoría de los valores de la variable objetivo son próximos a cero. El diagrama de dispersión revela un patrón característico, ya que, aunque el modelo predice con alta precisión los valores cercanos a cero, tiene dificultades para capturar los picos esporádicos de mayor magnitud, que aparecen como puntos dispersos alejados de la

diagonal. Este comportamiento es inherente a la distribución de la variable objetivo, dominada por un 85.8% de valores cero tal y como se analizó en la Sección 4.3.1. La **Figura 5.7** y la **Figura 5.8** ilustran estos comportamientos para `augmented_reality` y `video_analysis` respectivamente.

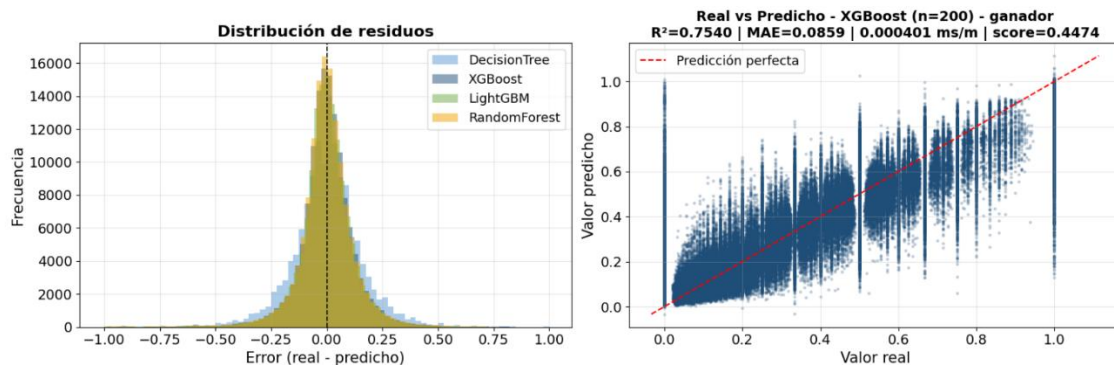


Figura 5.7. Distribución de residuos y diagrama de dispersión para la variable objetivo `augmented_reality`. Fuente: elaboración propia.

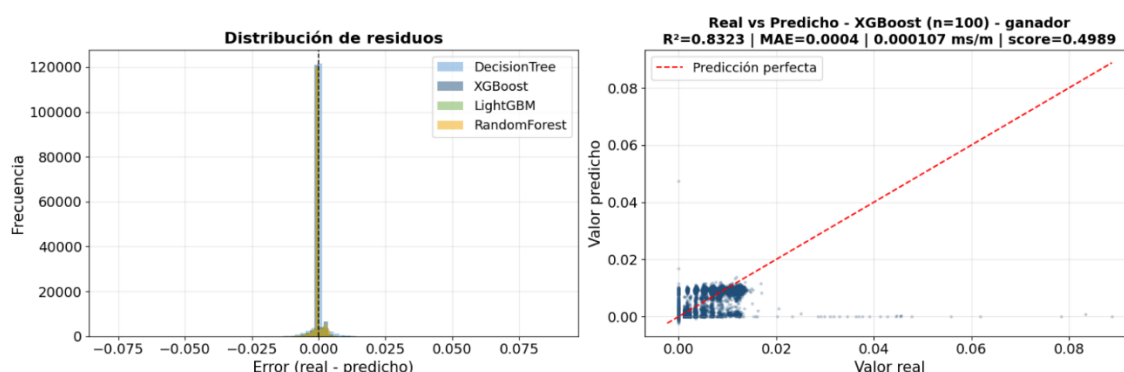


Figura 5.8. Distribución de residuos y diagrama de dispersión para la variable objetivo `video_analysis`. Fuente: elaboración propia.

Análisis de importancia de variables

Es importante distinguir entre la correlación de una variable con la variable objetivo, utilizada en la fase de selección de variables, y la importancia real que cada variable tiene a la hora de realizar la predicción [64]. La correlación mide la asociación estadística de forma aislada, mientras que la importancia refleja cuánto ha influido efectivamente cada variable en la predicción del modelo.

En el escenario `global`, el modelo ganador LightGBM presenta una distribución de importancia repartida entre varias variables temporales. Las dos más relevantes, `satisfied_rate_over_accepted_roll_mean_30` y `unsatisfied_rate_roll_mean_30`, acumulan conjuntamente un 21.7% de la importancia, con valores de 0.1130 y 0.1037, mientras que el resto se distribuye entre las

18 variables restantes. En `connected_vehicles` se observa un comportamiento similar. La variable más importante, `satisfied_rate_over_accepted_connected_vehicles`, concentra el 19.48% de la importancia, y las cinco primeras acumulan aproximadamente el 70%. Este patrón indica que LightGBM combina información propia del servicio con variables globales del sistema. La **Figura 5.9** y la **Figura 5.10** muestran la importancia de las variables de entrada para estas dos variables objetivo.

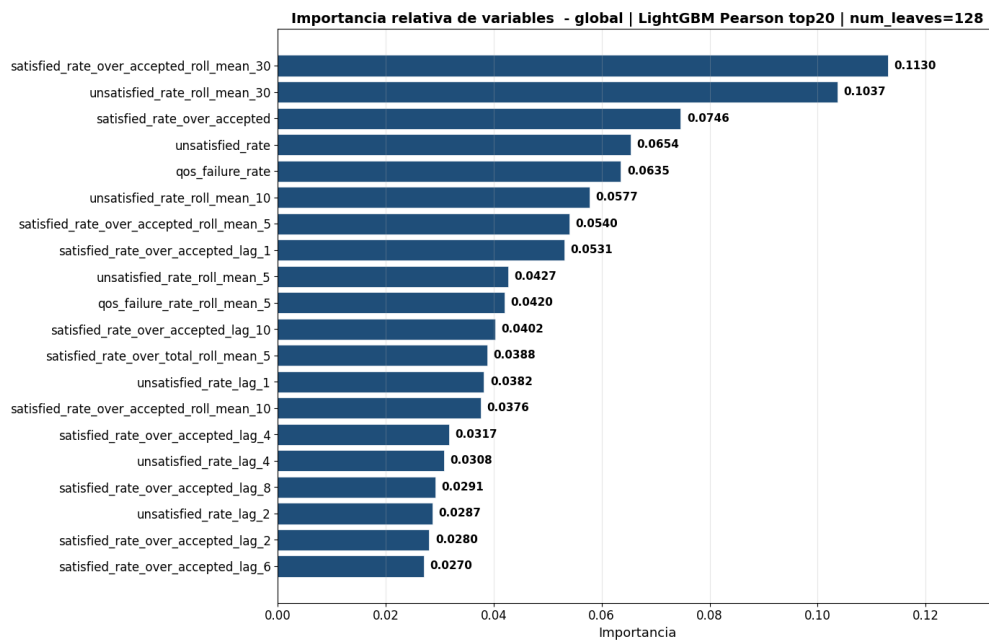


Figura 5.9. Importancia de variables del modelo LightGBM ganador para `global1`. Fuente: elaboración propia.

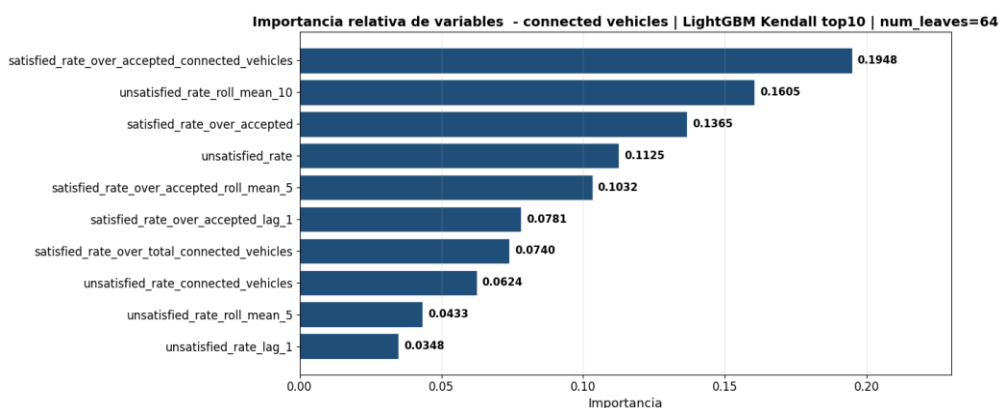


Figura 5.10. Importancia de variables del modelo LightGBM ganador para `connected_vehicles`. Fuente: elaboración propia.

En `augmented_reality`, la importancia se reparte principalmente entre `hot_bs_ratio_augmented_reality` (0.4463) y `satisfied_rate_over_accepted_augmented_reality` (0.4276), lo que refleja

una dependencia más compleja y ayuda a explicar la necesidad de utilizar 20 variables y el menor R^2 obtenido. En **video_analysis**, **satisfied_rate_over_accepted_video_analysis** domina con un 84.70%, seguida de **delay_augmented_reality_p95** (9.12%), mostrando una dependencia más concentrada y cierta relación cruzada entre servicios por el uso compartido de la infraestructura *edge*. La Figura 5.11 y la Figura 5.12 muestran estos resultados, coherentes con la distribución de las variables objetivo descrita en la Sección 4.3.1.

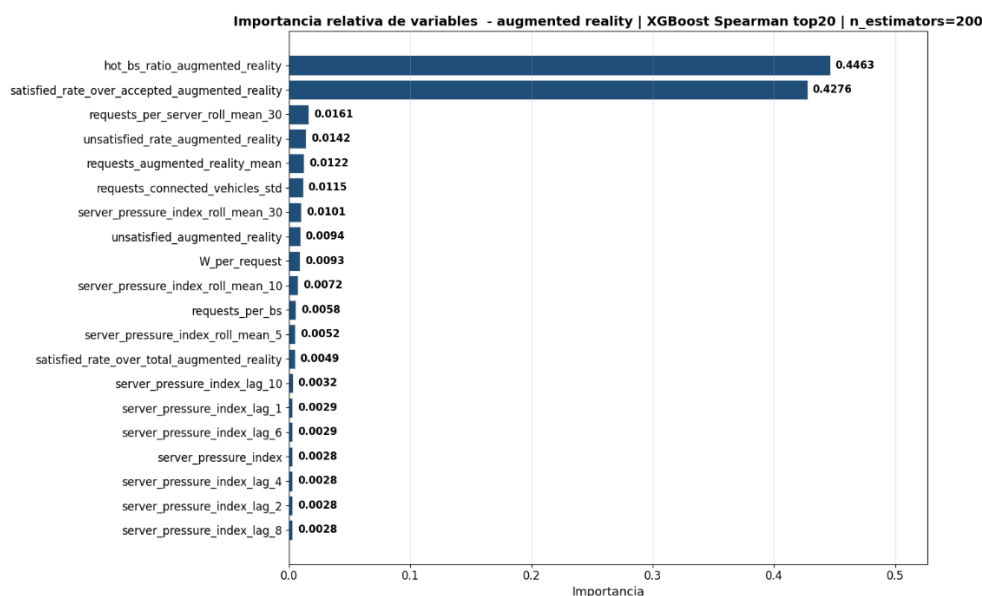


Figura 5.11. Importancia de variables del modelo XGBoost ganador para **augmented_reality**. Fuente: elaboración propia.

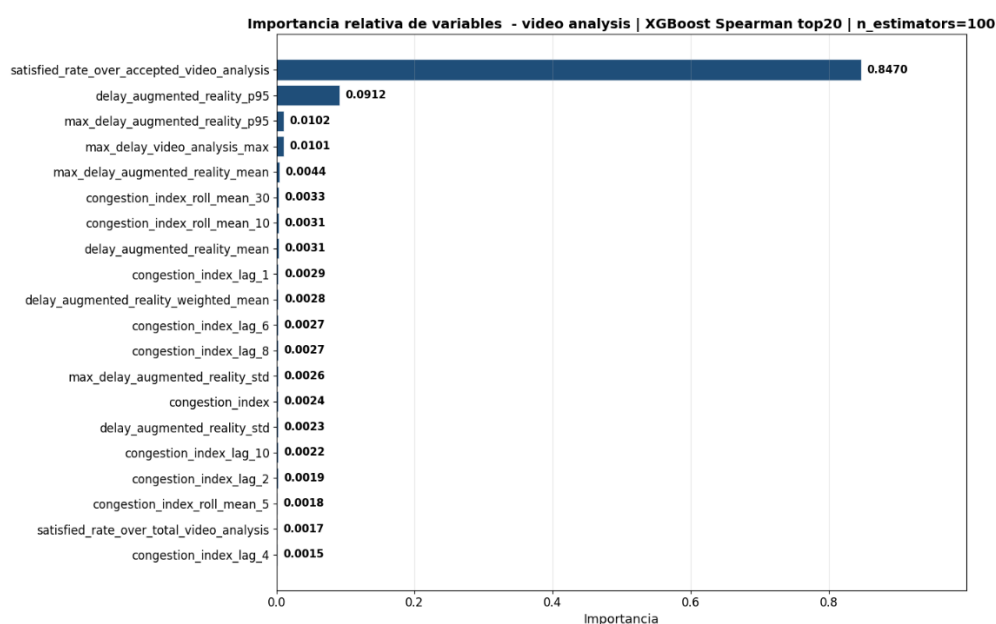


Figura 5.12. Importancia de variables del modelo XGBoost ganador para **video_analysis**. Fuente: elaboración propia.

5.3 Evaluación de modelos secuenciales

A pesar de los buenos resultados obtenidos con los modelos tabulares basados en árboles, los escenarios de **augmented_reality** y **video_analysis** presentan margen de mejora que motiva la exploración de arquitecturas alternativas. En esta sección se evalúan los modelos secuenciales LSTM y CNN 1D, cuya capacidad para aprender dependencias temporales directamente a partir de secuencias podría capturar patrones que los modelos tabulares no explotan, con el objetivo de verificar si esta capacidad permite mejorar las configuraciones de modelos tabulares mostradas.

5.3.1 Comparativa por variable objetivo

Los resultados de los modelos secuenciales muestran un comportamiento diferenciado respecto a los modelos tabulares. LSTM obtiene un R^2 ligeramente superior a CNN en tres de las cuatro variables objetivo (**global**, **augmented_reality** y **video_analysis**), con diferencias de entre 0.0009 y 0.0086. En **connected_vehicles** es CNN quien supera a LSTM en R^2 (0.9889 frente a 0.9727). Sin embargo, CNN resulta seleccionado como modelo ganador en las cuatro variables objetivo por el sistema de scoring. La **Tabla 5.4** recoge la mejor configuración de cada arquitectura por variable objetivo.

Tabla 5.4. Comparativa del mejor CNN frente al mejor LSTM por variable objetivo sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

Variable objetivo	Mejor CNN	R^2	MAE	ms/m	Mejor LSTM	R^2	MAE	ms/m
global	filters=256 pearson_top10	0.9884	0.0183	0.115	units=128 pearson_top10	0.9893	0.0167	0.296
cv	filters=32 pearson_top10	0.9889	0.0191	0.076	units=64 spearman_top10	0.9727	0.0387	0.085
ar	filters=32 pearson_top5	0.7244	0.0870	0.092	units=128 spearman_top5	0.7302	0.0858	0.120
va	filters=128 cfs_top5	0.8186	0.0004	0.035	units=64 spearman_top10	0.8272	0.0004	0.090

Nota: cv: **connected_vehicles**; ar: **augmented_reality**; va: **video_analysis**.

La razón por la que CNN resulta seleccionada como mejor arquitectura secuencial, a pesar de no obtener siempre el mayor R^2 , se debe al efecto del tiempo de inferencia en el cálculo de **score**. LSTM presenta tiempos de inferencia superiores, entre 0.085 y 0.296 ms por muestra, mientras que CNN se sitúa entre 0.035 y 0.115 ms. Dado que las diferencias de R^2 entre ambas arquitecturas son reducidas, esta ventaja en tiempo de

inferencia permite a CNN obtener una mejor relación entre precisión y eficiencia computacional. Esta diferencia es coherente con la naturaleza de ambas arquitecturas: LSTM procesa la secuencia de forma recurrente, mientras que CNN permite aplicar filtros convolucionales de forma más paralelizable [72] [73]. Por este motivo, CNN se selecciona como mejor modelo secuencial en las cuatro variables objetivo evaluadas.

En cuanto a los resultados por variable objetivo, se observan patrones similares a los obtenidos con los modelos tabulares. Las variables **global** y **connected_vehicles** alcanzan R^2 superiores a 0.97 con ambas arquitecturas, mientras que **augmented_reality** se mantiene como la variable objetivo más difícil desde el punto de vista predictivo, con R^2 en torno a 0.73. Aunque **video_analysis** presenta una distribución más desequilibrada y dominada por valores cercanos a cero, alcanza valores próximos a 0.82, por lo que en este trabajo se interpreta como un caso particular en el que métricas como el MAE deben analizarse con cautela, pero no como la variable objetivo con peor capacidad predictiva.

Estos resultados indican que, para el conjunto de datos evaluado, la capacidad de CNN y LSTM para aprender dependencias temporales a partir de secuencias no compensa la pérdida de información que supone prescindir de variables temporales explícitas, como valores históricos, medias móviles y diferencias temporales. No obstante, no se descarta que un conjunto de simulaciones más amplio, una mayor diversidad de escenarios o configuraciones de entrenamiento más extensas pudieran mejorar el rendimiento de los modelos secuenciales. Esta cuestión queda planteada como trabajo futuro.

En cuanto al tamaño de las arquitecturas evaluadas, aumentar la complejidad no produjo una mejora sistemática. En LSTM se mantuvo un límite máximo de 128 unidades, ya que las pruebas con 256 unidades mostraron tiempos de entrenamiento excesivos. En CNN, las configuraciones con 256 filtros obtuvieron buenos resultados para **global**, pero no mantuvieron ese comportamiento en el resto de variables objetivo, llegando a producir valores de R^2 próximos a cero en determinadas combinaciones. Por ello, las arquitecturas más compactas resultaron más robustas para este problema.

5.3.2 Análisis por arquitectura (LSTM frente a CNN)

Comparativa de arquitecturas

Al comparar ambos tipos de redes neuronales se observa que CNN obtiene mejor **score**, aunque LSTM obtiene R^2 ligeramente superiores en varias configuraciones. Esta diferencia se explica por la forma en que ambas arquitecturas procesan la secuencia. LSTM propaga el estado oculto paso a paso a través de sus celdas de memoria, lo que permite capturar dependencias temporales lejanas pero incrementa el coste computacional. CNN aplica filtros convolucionales de forma más paralelizable, extrayendo patrones locales con menor tiempo de inferencia [72]. Dado que la ventana temporal descrita en la Sección 4.4.2 se acotó a 10 segundos (**SEQ_LEN**), la ventaja teórica de LSTM no se traduce en una mejora significativa de precisión, pero sí penaliza el tiempo de inferencia.

En **global** y **connected_vehicles**, CNN y LSTM se intercalan en las posiciones superiores de la clasificación por R^2 , aunque el conjunto completo de configuraciones muestra una dispersión relevante: en **global** el rango va de 0.91 a 0.99, y en **connected_vehicles** de 0.97 a 0.99, lo que indica que la selección de variables y el tamaño de la arquitectura tienen un impacto considerable. La subgráfica de tiempo de inferencia de la **Figura 5.13** y la **Figura 5.14** revela que en **global** la ventaja de CNN es notable, con 0.115 ms por muestra frente a valores de hasta 0.296 ms en LSTM, mientras que en **connected_vehicles** las diferencias son más moderadas, con 0.076 ms del ganador CNN frente a valores de 0.085-0.149 ms en LSTM.

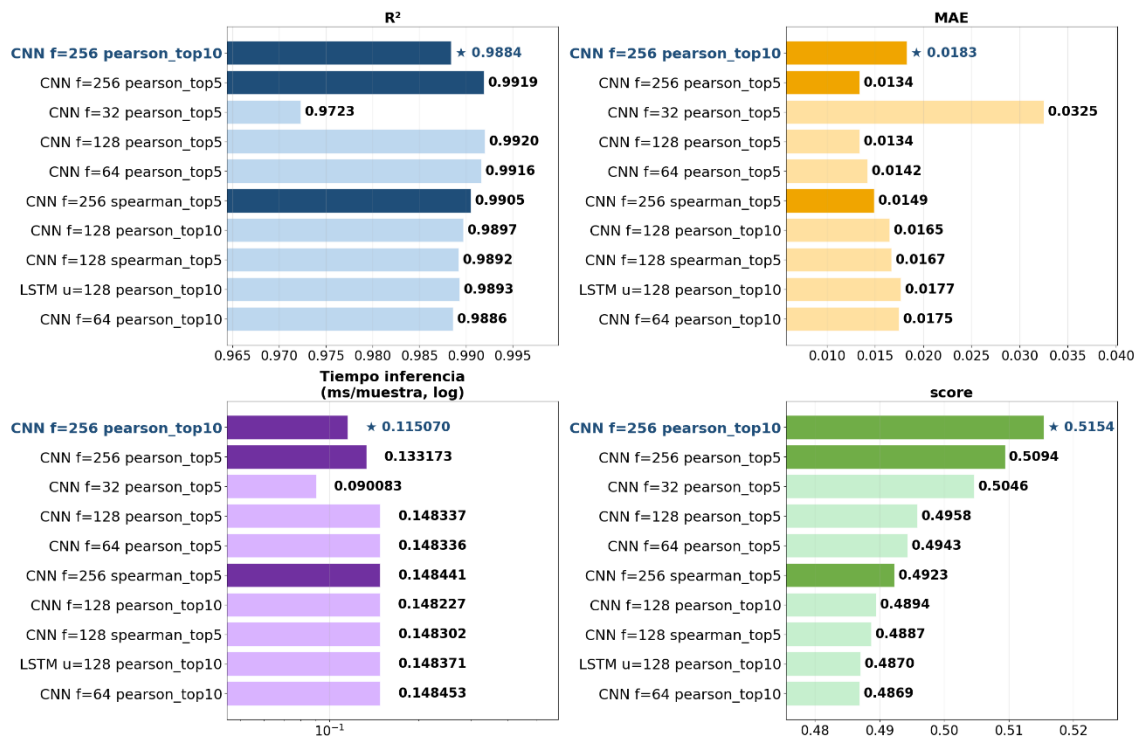


Figura 5.13. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por **score** para la variable objetivo **global**. Fuente: elaboración propia.

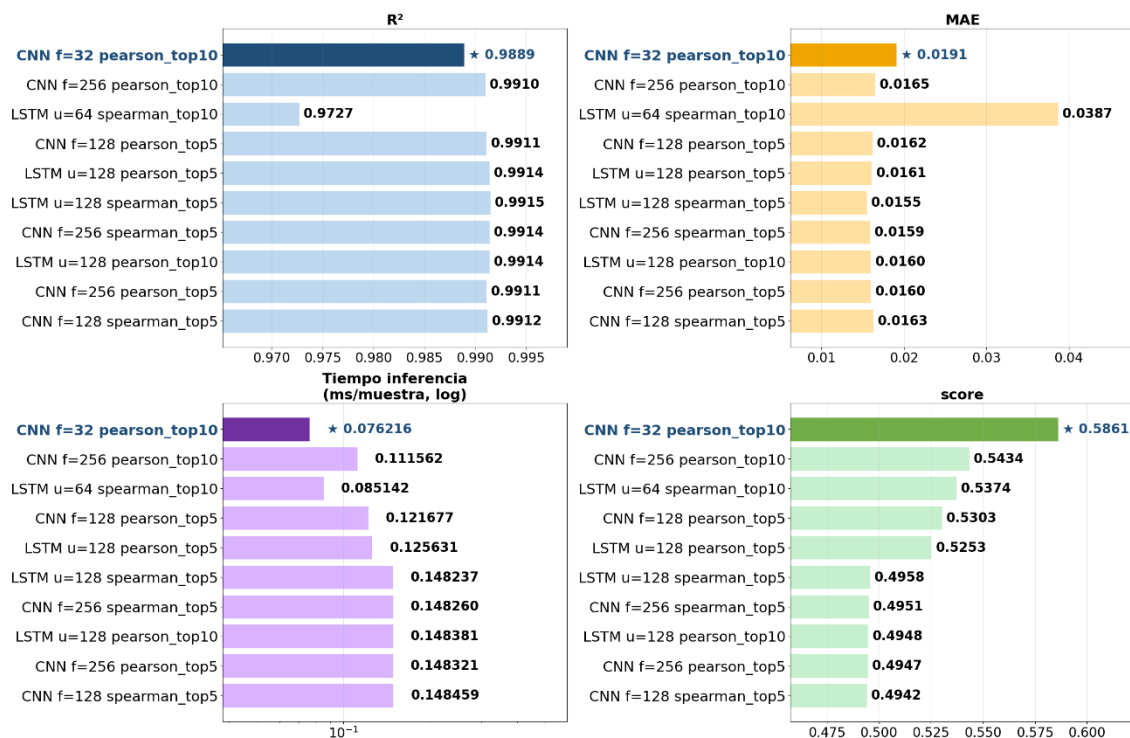


Figura 5.14. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por **score** para la variable objetivo **connected_vehicles**. Fuente: elaboración propia.

En el escenario de **augmented_reality**, como muestra la **Figura 5.15**, CNN domina las posiciones superiores de la clasificación por **score**, aunque el rango de R^2 entre configuraciones es relativamente estrecho (0.70-0.73), indicando que la dificultad de esta variable objetivo limita el rendimiento independientemente de la arquitectura. Los tiempos de inferencia en esta variable objetivo son comparables entre ambas familias, y en este caso la ventaja de CNN es menos pronunciada que en caso anteriores (0.092 ms frente a valores de 0.071-0.131 ms en LSTM). En el caso de **video_analysis**, LSTM y CNN se intercalan con R^2 muy próximos entre sus mejores configuraciones (0.82-0.83); la subgráfica de tiempo de inferencia de la **Figura 5.16** muestra aquí la mayor ventaja de CNN, con el ganador en 0.035 ms frente a valores de 0.074-0.149 ms en LSTM, lo que explica por qué **score** favorece con más claridad a CNN en esta variable objetivo.

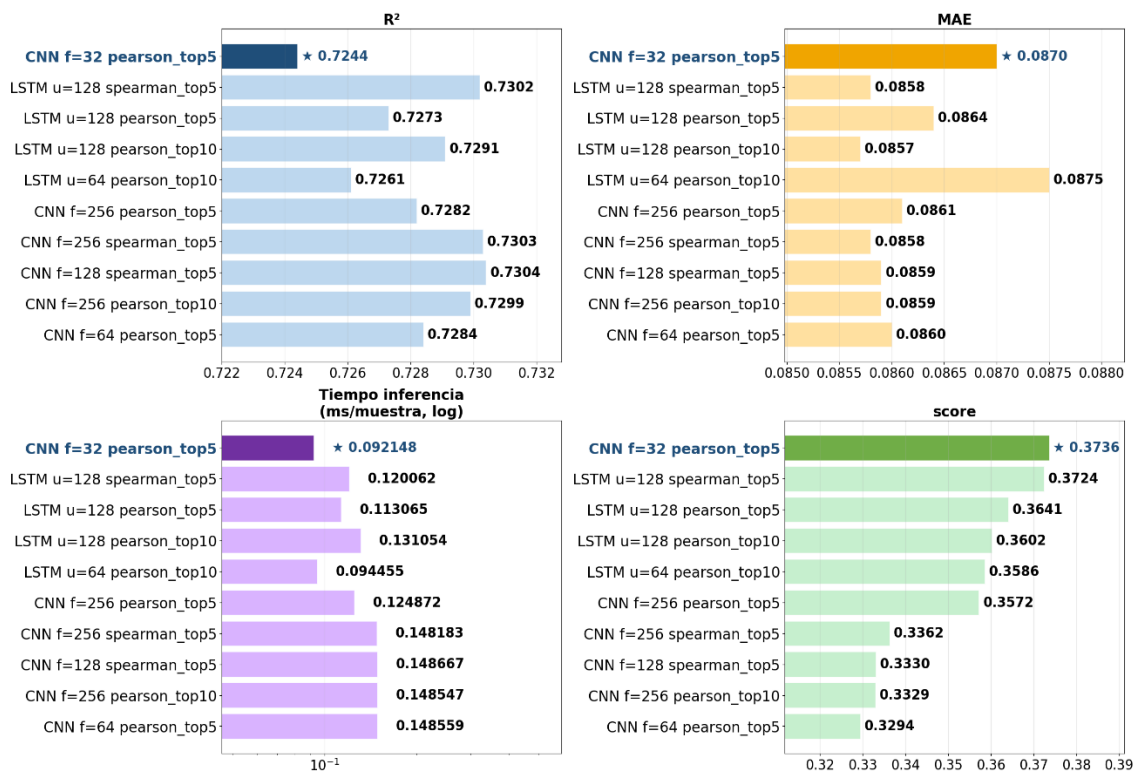


Figura 5.15. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por **score** para la variable objetivo **augmented_reality**. Fuente: elaboración propia.

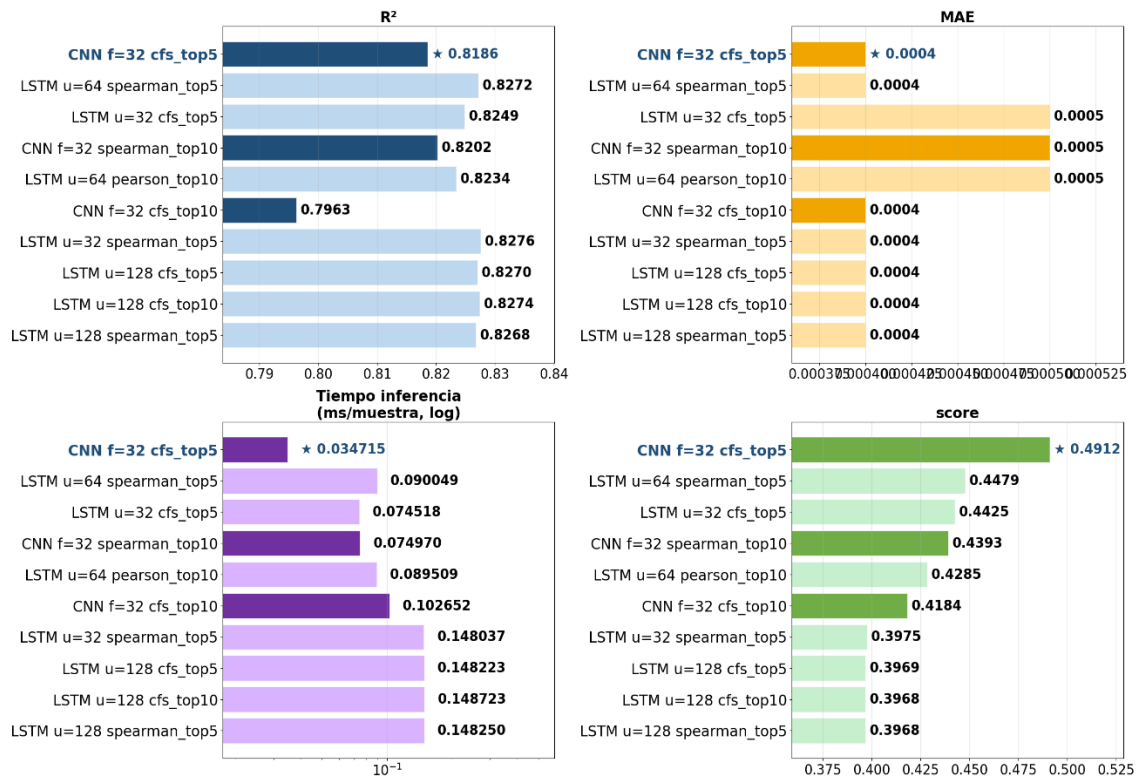


Figura 5.16. Comparativa del modelo secuencial ganador frente al resto por R^2 , MAE, tiempo de inferencia ordenado por **score** para la variable objetivo **video_analysis**. Fuente: elaboración propia.

Análisis de residuos

La distribución de residuos de los modelos secuenciales presenta diferencias apreciables respecto a la de los modelos tabulares. Para el caso **global**, LSTM muestra un pico más estrecho y centrado en cero que CNN, coherente con su R^2 ligeramente superior, mientras que CNN **filters=256** presenta una distribución más ancha y con colas algo más pesadas. El diagrama de dispersión del ganador, CNN **filters=256** con $R^2=0.9884$, mantiene una buena alineación con la diagonal de predicción perfecta, aunque con mayor dispersión que en los modelos tabulares y sin las predicciones negativas pronunciadas observadas en versiones anteriores. En **connected_vehicles**, CNN **filters=32** produce una distribución estrecha y bien centrada en cero, mientras que LSTM **units=64 spearman_top10** presenta una cola izquierda hasta errores de -1.25 , indicando sobrepredicción sistemática en algunos casos. El diagrama del ganador CNN, con $R^2=0.9889$, muestra un ajuste visual razonable a lo largo de toda la diagonal. La **Figura 5.17** y la **Figura 5.18** recogen estos resultados para **global** y **connected_vehicles**, respectivamente.

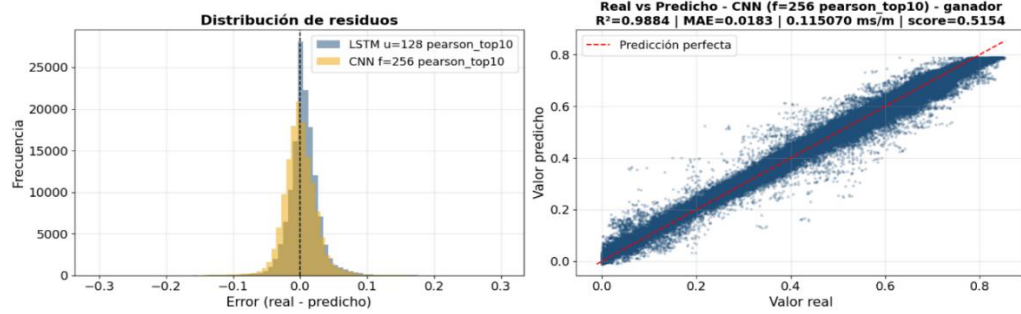


Figura 5.17. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en **global**. Fuente: elaboración propia.

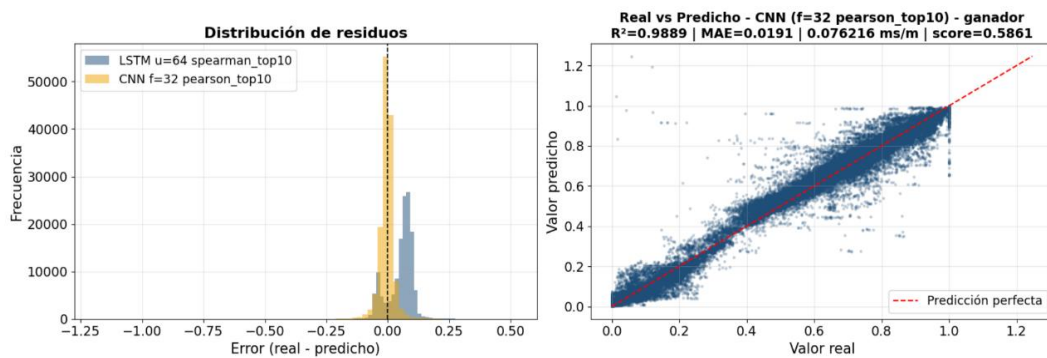


Figura 5.18. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en **connected_vehicles**. Fuente: elaboración propia.

Para el caso de **augmented_reality**, la distribución de residuos es notablemente más ancha que la de los modelos tabulares, con colas pesadas en ambas direcciones y una mayor frecuencia de errores grandes. El diagrama de dispersión muestra patrones verticales de predicción que indican que el modelo tiende a agrupar predicciones en determinados valores. Este comportamiento refleja la dificultad adicional de predecir esta variable objetivo sin variables temporales explícitas, y es más pronunciado que en los modelos tabulares. En **video_analysis**, la distribución de residuos está concentrada en cero de forma extrema, al igual que en los modelos tabulares, y el diagrama de dispersión muestra el mismo patrón de buena predicción para valores cercanos a cero, pero incapacidad para capturar los picos esporádicos de mayor magnitud. La **Figura 5.19** y la **Figura 5.20** muestran estos resultados para **augmented_reality** y **video_analysis**, respectivamente.

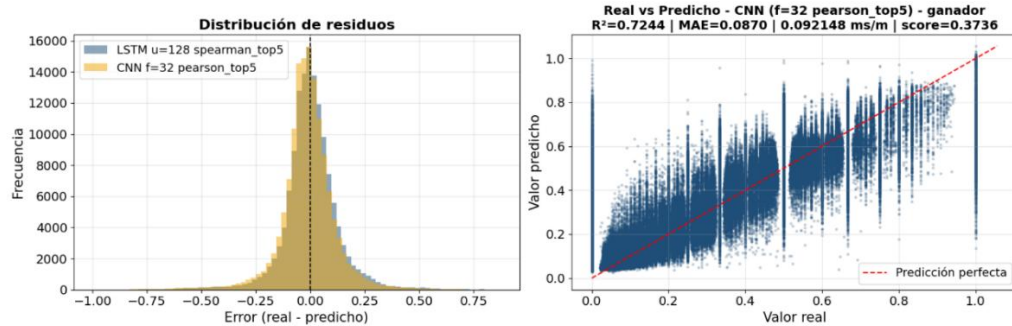


Figura 5.19. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en **augmented_reality**. Fuente: elaboración propia.

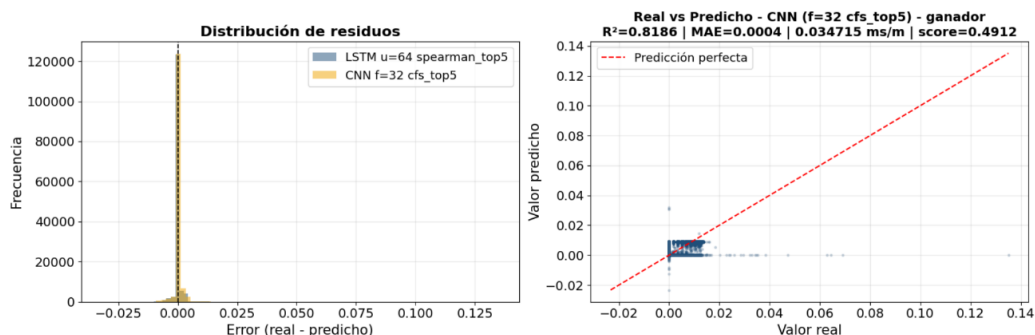


Figura 5.20. Distribución de residuos y diagrama de dispersión del valor real frente al valor predicho para los modelos secuenciales ganadores en **video_analysis**. Fuente: elaboración propia.

En conjunto, los modelos secuenciales muestran menor precisión, mayor dispersión y tiempos de inferencia superiores a los modelos tabulares, anticipando el resultado de la comparación global.

5.4 Comparación global de modelos

En las secciones anteriores se han evaluado y seleccionado los mejores modelos de cada familia de forma independiente, mostrando LightGBM como mejor modelo tabular en los casos **global** y **connected_vehicles**, XGBoost en **augmented_reality** y **video_analysis**, y CNN como mejor modelo secuencial en las cuatro variables objetivo. Esta evaluación separada es intencionada debido a la asimetría en los tiempos de inferencia del orden de microsegundos. Una comparación conjunta desde el inicio habría descartado automáticamente los modelos secuenciales por penalización temporal, sin valorar si su capacidad para aprender dependencias temporales justificaba el mayor coste computacional. Al enfrentar al mejor representante de cada familia, la **Tabla 5.5** recoge la comparativa directa para cada variable objetivo.

Tabla 5.5. Comparativa del mejor modelo tabular frente al mejor modelo secuencial sobre el conjunto de validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

V. Objetivo	Tipo	Modelo	Selección	Hiperparámetros	R ²	MAE	RMSE	ms/muestra	score
global	Tabular	LightGBM	Pearson top20	num_leaves=128	0.9943	0.0115	0.0192	0.000562	0.5966
global	Secuencial	CNN	Pearson top10	filters=256	0.9884	0.0183	0.0257	0.115070	0.1930
cv	Tabular	LightGBM	Kendall top10	num_leaves=64	0.9939	0.0133	0.0249	0.000485	0.5963
cv	Secuencial	CNN	Pearson top10	filters=32	0.9889	0.0191	0.0324	0.076216	0.1933
ar	Tabular	XGBoost	Spearman top20	n_estimators=200	0.7540	0.0859	0.1274	0.000401	0.4524
ar	Secuencial	CNN	Pearson top5	filters=32	0.7244	0.0870	0.1293	0.092148	0.0346
va	Tabular	XGBoost	Spearman top20	n_estimators=100	0.8323	0.0004	0.0014	0.000107	0.4994
va	Secuencial	CNN	CFS top5	filters=32	0.8186	0.0004	0.0014	0.034715	0.3912

Nota: cv: **connected_vehicles**; ar: **augmented_reality**; va: **video_analysis**; V: Variable.

Los resultados muestran que el modelo tabular supera a CNN en las cuatro variables objetivo tanto en R² como en MAE. Para los casos **global** y **connected_vehicles**, LightGBM obtiene un R² de 0.9943 y 0.9939 frente al 0.9884 y 0.9889 de CNN respectivamente. Para **augmented_reality** y **video_analysis**, XGBoost obtiene un R² de 0.7540 y 0.8323 frente al 0.7244 y 0.8186 de CNN. Aunque las diferencias en R² pueden parecer pequeñas en valor absoluto, son significativas en un rango donde ambos modelos se encuentran próximos al ajuste perfecto. La diferencia en tiempo de inferencia es especialmente notable: los modelos tabulares operan entre 150 y 330 veces más rápido que CNN, con tiempos de 0.0001-0.0006 ms frente a 0.035-0.115 ms por muestra. La **Figura 5.21** y la **Figura 5.22** muestran esta comparación para **global** y **connected_vehicles**, respectivamente.

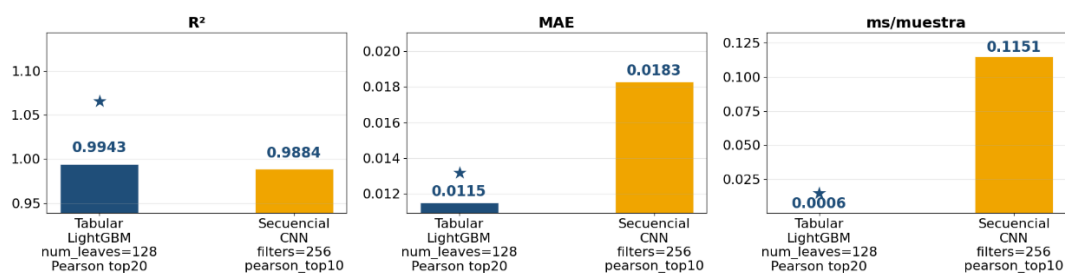


Figura 5.21. Comparativa entre LightGBM y CNN **filters=256** por R², MAE y tiempo de inferencia para **global**. Fuente: elaboración propia.

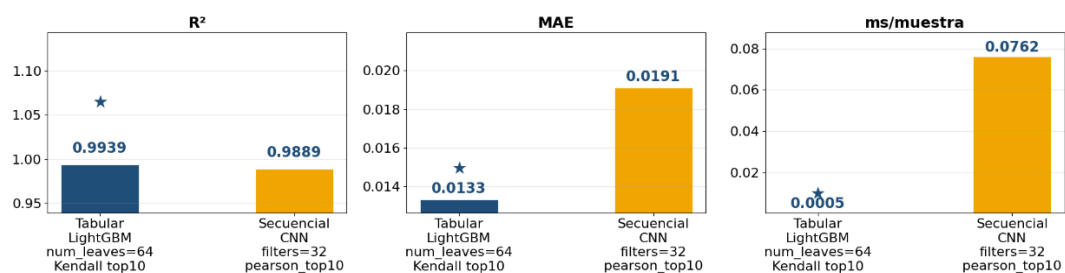


Figura 5.22. Comparativa entre LightGBM y CNN **filters=32** por R², MAE y tiempo de inferencia para **connected_vehicles**. Fuente: elaboración propia.

En el caso de **augmented_reality** la diferencia de R^2 es la más pronunciada (0.7540 frente a 0.7244), lo que sugiere que las variables temporales construidas en la fase de ingeniería de características, como valores históricos, medias móviles y diferencias temporales, aportan información útil a los modelos tabulares y les permiten mejorar el rendimiento frente al modelo secuencial. En **video_analysis** la diferencia es de 0.0137 (0.8323 frente a 0.8186), manteniéndose la ventaja de XGBoost aunque ambos modelos presentan las mismas limitaciones frente a la distribución sesgada de esta variable objetivo. La **Figura 5.23** y la **Figura 5.24** muestran estas comparativas para **augmented_reality** y **video_analysis**, respectivamente.

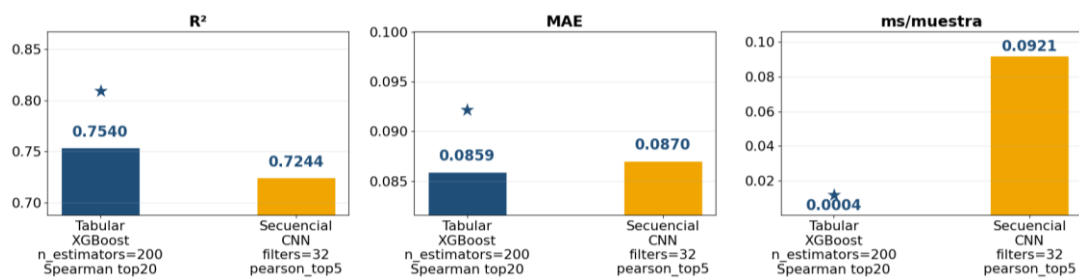


Figura 5.23. Comparativa entre XGBoost y CNN **filters=32** por R^2 , MAE y tiempo de inferencia para **augmented_reality**. Fuente: elaboración propia.

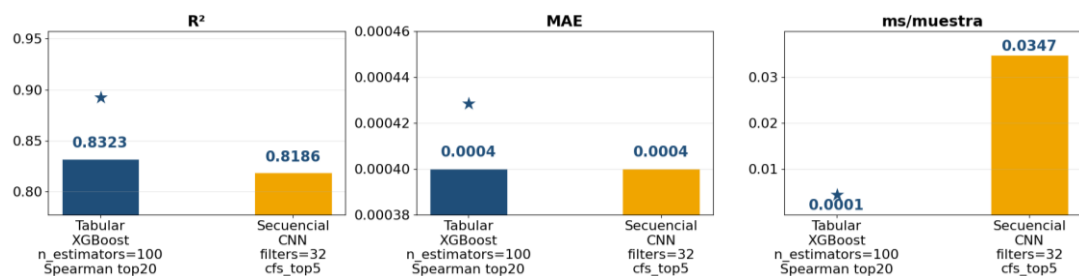


Figura 5.24. Comparativa entre XGBoost y CNN **filters=32** por R^2 , MAE y tiempo de inferencia para **video_analysis**. Fuente: elaboración propia.

En esta comparación global no se recalcula una nueva normalización del score sobre un conjunto reducido de candidatos, sino que se mantiene el criterio ponderado definido en la metodología y se contrasta la decisión con las métricas originales de R^2 , MAE y tiempo de inferencia. Aplicando el criterio de selección basado en **score**, el modelo tabular resulta seleccionado como ganador en las cuatro variables objetivo. Este resultado es coherente con la literatura reciente sobre datos tabulares. Grinsztajn et al. (2022) mostraron, mediante un estudio comparativo sobre 45 conjuntos de datos, que los modelos basados en árboles siguen superando a las arquitecturas de DL en datos tabulares de tamaño medio [74]. De forma complementaria, Shwartz-Ziv y Armon (2021)

observaron que XGBoost supera a modelos recientes de DL en la mayoría de conjuntos tabulares evaluados, requiriendo además un ajuste de hiperparámetros menor [75].

En el contexto de este trabajo, estos hallazgos se confirman. El conjunto de datos homogéneo construido a partir de las simulaciones de MintEDGE es un conjunto tabular estructurado, con variables numéricas enriquecidas mediante ingeniería de características temporales. Este tipo de datos se ajusta al perfil en el que los modelos basados en árboles obtienen buenos resultados, mientras que las CNN no encuentran en las secuencias de 10 instantes una mejora suficiente para justificar su mayor coste computacional.

5.5 Evaluación final sobre el conjunto de prueba y discusión de resultados

La evaluación sobre el conjunto de prueba constituye el paso final y definitivo en el proceso de validación de un modelo de ML [68]. A lo largo de todo el proceso experimental descrito en las secciones anteriores, las decisiones de selección de variables, ajuste de hiperparámetros y elección de modelos se han tomado exclusivamente sobre el conjunto de validación. En esta sección se aplican los cuatro modelos tabulares seleccionados (uno por variable objetivo) sobre este conjunto de datos no visto previamente, con el objetivo de obtener una estimación fiable y no sesgada del rendimiento real del predictor ante datos nuevos.

5.5.1 Resultados sobre el conjunto de prueba

La **Tabla 5.6** recoge los resultados de los cuatro modelos ganadores sobre el conjunto de prueba y los compara con los valores obtenidos previamente en validación. Para cada variable objetivo se muestran las métricas R^2 , MAE y RMSE, junto con el tiempo medio de inferencia por muestra.

Tabla 5.6. Resultados de los modelos ganadores sobre el conjunto de prueba frente a los obtenidos sobre validación. Fuente: elaboración propia a partir de los resultados experimentales obtenidos.

Variable objetivo	Modelo	Configuración	R^2 val.	R^2 prueba	MAE val.	MAE prueba	RMSE val.	RMSE prueba	ms/muestra
global	LightGBM	Pearson top20, num_leaves=128	0.9943	0.9943	0.0115	0.0115	0.0192	0.0193	0.001641
connected_vehicles	LightGBM	Kendall top10, num_leaves=64	0.9939	0.9941	0.0133	0.0133	0.0249	0.0247	0.001000
augmented_reality	XGBoost	Spearman top20, n_estimators=200	0.7540	0.7562	0.0859	0.0858	0.1274	0.1270	0.000506
video_analysis	XGBoost	Spearman top20, n_estimators=100	0.8323	0.8285	0.0004	0.0004	0.0014	0.0014	0.000290

Los resultados sobre el conjunto de prueba confirman el rendimiento observado en validación. Para **global**, el modelo LightGBM obtiene un R^2 de 0.9943 con un MAE de 0.0115, valores muy próximos a los de validación. La **Figura 5.25** muestra que el modelo sigue fielmente el comportamiento del sistema, capturando tanto las tendencias generales como las variaciones puntuales de la tasa de satisfacción **global**. Para **connected_vehicles**, el R^2 sobre prueba es de 0.9941, ligeramente superior al 0.9939 de validación, con un MAE prácticamente igual de 0.0133. La **Figura 5.26** muestra el comportamiento correspondiente para esta variable objetivo.

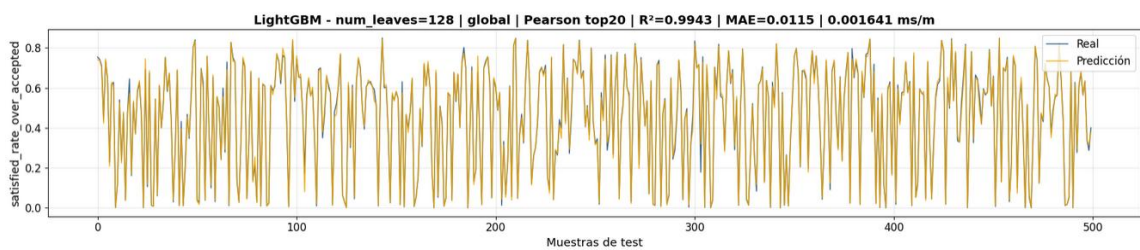


Figura 5.25. Predicción frente al valor real sobre el conjunto de prueba para **global**. Fuente: elaboración propia.

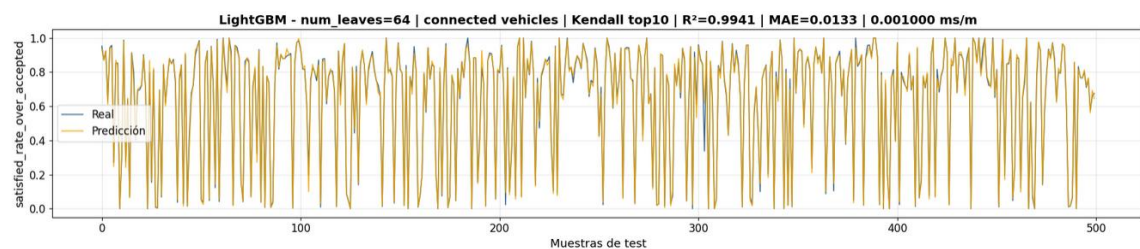


Figura 5.26. Predicción frente al valor real sobre el conjunto de prueba para **connected_vehicles**. Fuente: elaboración propia.

En **augmented_reality**, el modelo obtiene un R^2 de 0.7562 sobre el conjunto de prueba frente al 0.7540 de validación, con una mejora de 0.0022 que indica que el modelo generaliza correctamente incluso en la variable objetivo más difícil de predecir. La **Figura 5.27** muestra una mayor dispersión respecto a las dos variables objetivo anteriores, coherente con la naturaleza más irregular de este servicio, pero el modelo captura la tendencia general de la variable objetivo. En **video_analysis**, el R^2 sobre el conjunto de prueba es de 0.8285, con una ligera caída de 0.0038 respecto al 0.8323 de validación. La **Figura 5.28** muestra el patrón característico de esta variable objetivo. El modelo predice correctamente los valores cercanos a cero pero presenta dificultades

para capturar la magnitud exacta de los picos esporádicos, que en el conjunto de prueba son más pronunciados que en validación.

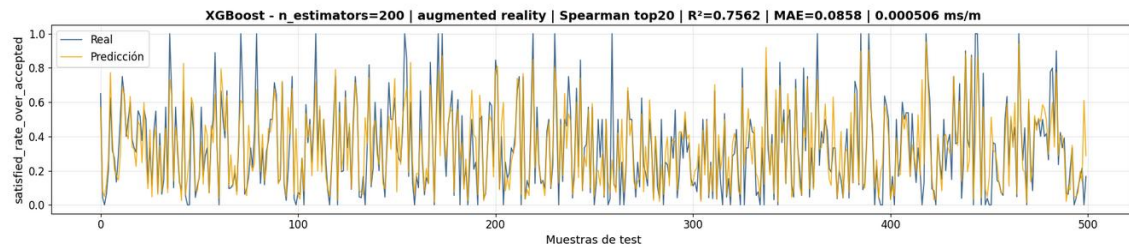


Figura 5.27. Predicción frente al valor real sobre el conjunto de prueba para **augmented_reality**. Fuente: elaboración propia.

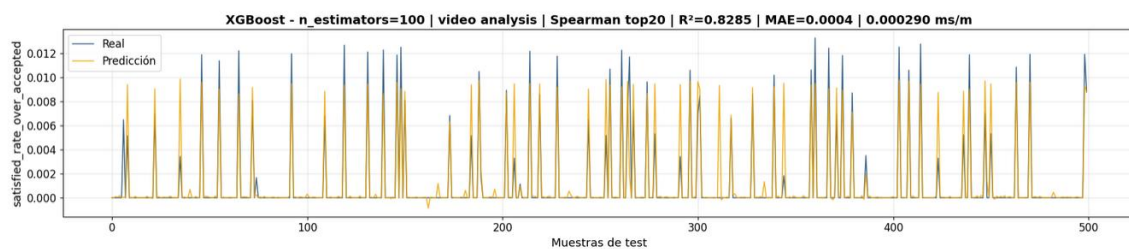


Figura 5.28. Predicción frente al valor real sobre el conjunto de prueba para **video_analysis**. Fuente: elaboración propia.

5.5.2 Comparativa entre validación y prueba

Para confirmar la ausencia de sobreajuste, se comparan las predicciones sobre validación y prueba de forma superpuesta. Las gráficas muestran las predicciones de ambos conjuntos junto al valor real.

En los casos de **global** y **connected_vehicles**, las predicciones de validación y prueba son prácticamente indistinguibles, con diferencias de R^2 de +0.0002 en ambas variables objetivo y mejoras marginales en MAE de 0.0002 y 0.0001 respectivamente. Este resultado confirma que los modelos no presentan sobreajuste y que su capacidad de generalización es plena para estas variables objetivo. La **Figura 5.29** y la **Figura 5.30** muestran esta comparación para **global** y **connected_vehicles**, respectivamente.

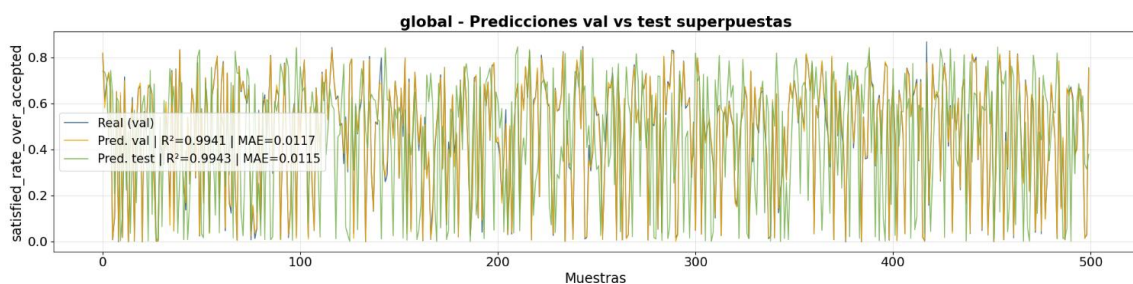


Figura 5.29. Predicciones sobre validación y prueba superpuestas para **global**. Fuente: elaboración propia.

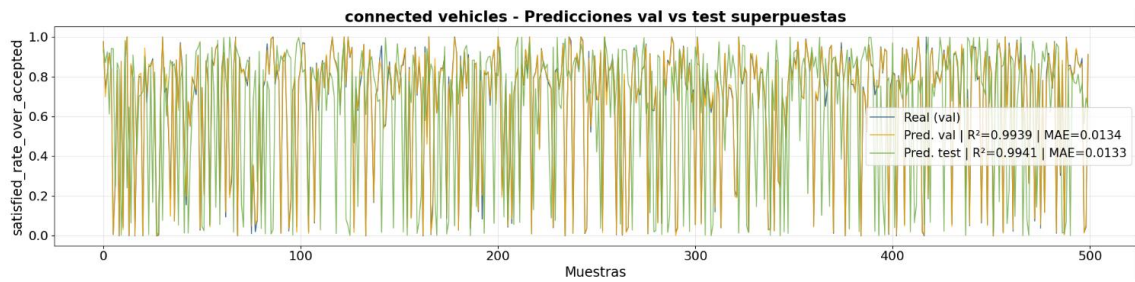


Figura 5.30. Predicciones sobre validación y prueba superpuestas para `connected_vehicles`. Fuente: elaboración propia.

En el caso de `augmented_reality`, el R^2 mejora ligeramente en prueba (+0.0033), lo que puede deberse a que el bloque de prueba, al provenir de una rotación diferente de ciudades, contiene patrones ligeramente más regulares que los del bloque de validación. En `video_analysis`, la caída de R^2 en prueba (-0.0038) se explica por la presencia en el conjunto de prueba de picos de mayor magnitud que en validación. El modelo ha aprendido el patrón general pero no ha visto durante el entrenamiento picos tan pronunciados como los que aparecen en prueba. No obstante, una diferencia de 0.0038 en R^2 es marginal y no indica sobreajuste sino una variabilidad natural entre particiones del conjunto de datos. La **Figura 5.31** y la **Figura 5.32** muestran esta comparación para `augmented_reality` y `video_analysis`, respectivamente.

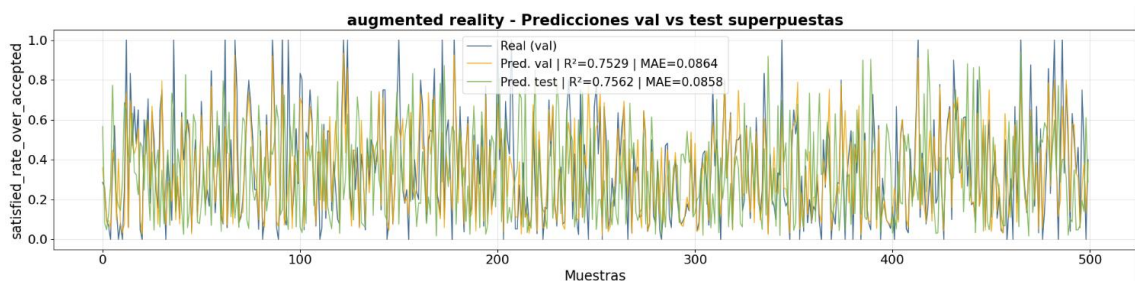


Figura 5.31. Predicciones sobre validación y prueba superpuestas para `augmented_reality`. Fuente: elaboración propia.

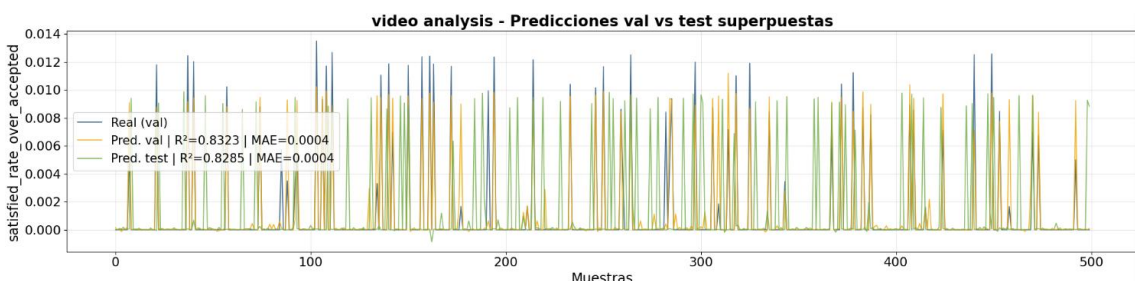


Figura 5.32. Predicciones sobre validación y prueba superpuestas para `video_analysis`. Fuente: elaboración propia.

El análisis de residuos comparando validación y prueba confirma estas observaciones. En **global** y **connected_vehicles**, tanto los diagramas de dispersión como las distribuciones de residuos mantienen la misma forma y amplitud entre validación y prueba, con medias de error prácticamente iguales a cero. En **augmented_reality**, la distribución de residuos de prueba presenta una forma similar a la de validación, con colas de amplitud comparable. En **video_analysis**, el diagrama de dispersión de prueba muestra puntos ligeramente más dispersos que el de validación, coherente con los picos de mayor magnitud en prueba, pero la distribución de residuos permanece centrada en cero. De la **Figura 5.33** a la **Figura 5.36** se muestra este análisis comparativo de residuos para las cuatro variables objetivo.

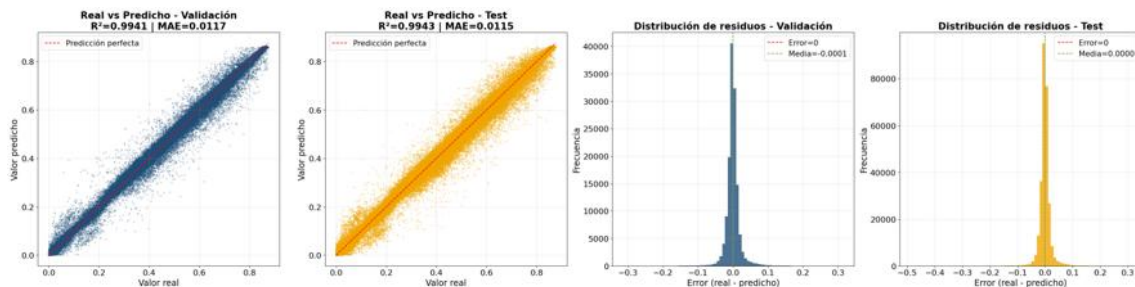


Figura 5.33. Análisis de residuos comparativo entre validación y prueba para **global**. Fuente: elaboración propia.

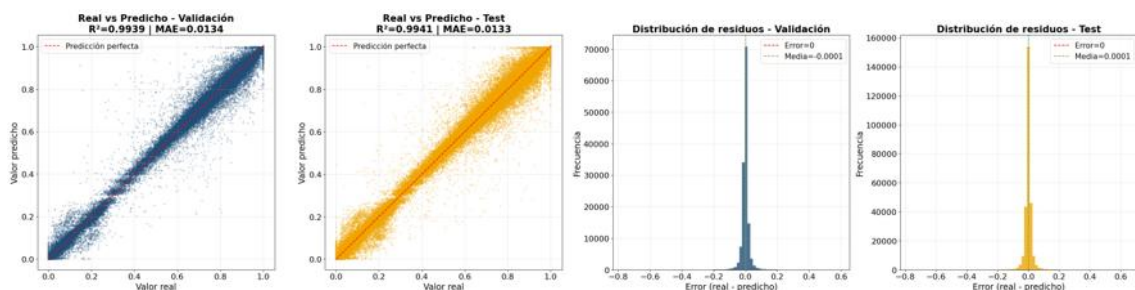


Figura 5.34. Análisis de residuos comparativo entre validación y prueba para **connected_vehicles**. Fuente: elaboración propia.

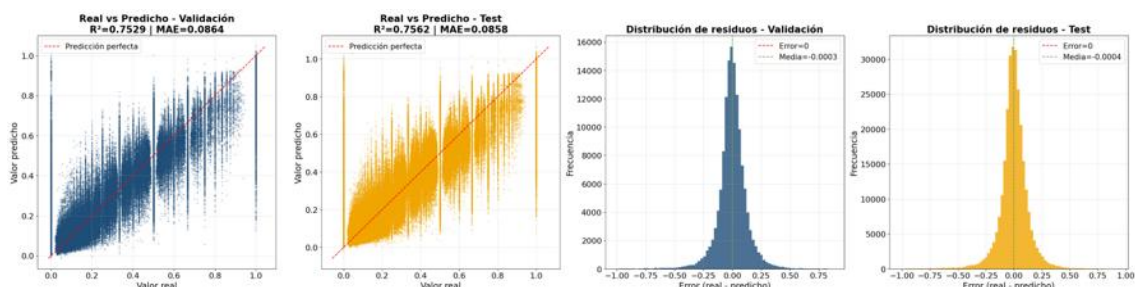


Figura 5.35. Análisis de residuos comparativo entre validación y prueba para **augmented_reality**. Fuente: elaboración propia.

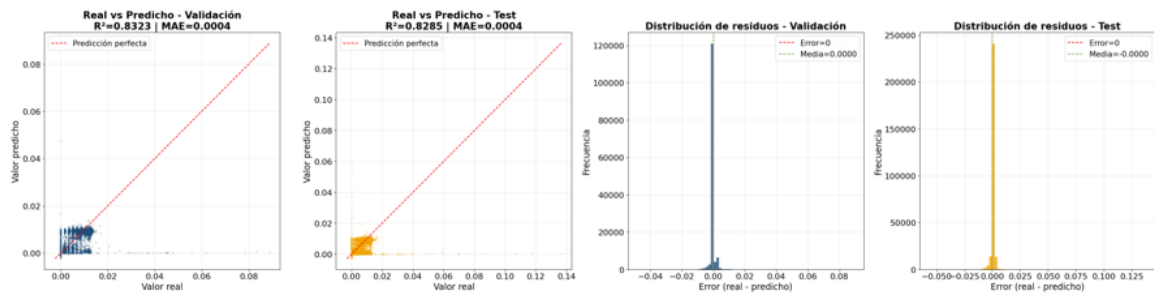


Figura 5.36. Análisis de residuos comparativo entre validación y prueba para **video_analysis**. Fuente: elaboración propia.

5.5.3 Análisis y discusión

Los resultados obtenidos sobre el conjunto de prueba permiten valorar la viabilidad del servicio predictivo diseñado para su integración en un sistema de orquestación como MintEDGE.

Para **global** y **connected_vehicles**, los modelos finales alcanzan un rendimiento alto, con valores de R^2 superiores a 0.994 y MAE inferiores a 0.014, lo que supone una desviación media respecto al valor real inferior al 1.5%. Además, las diferencias de R^2 entre validación y prueba son inferiores a 0.001, lo que indica una buena capacidad de generalización ante escenarios no vistos. Este nivel de precisión, junto con tiempos de inferencia inferiores a $1 \mu s$ por muestra, hace que los modelos resulten viables para su uso en un bucle de control del orquestador.

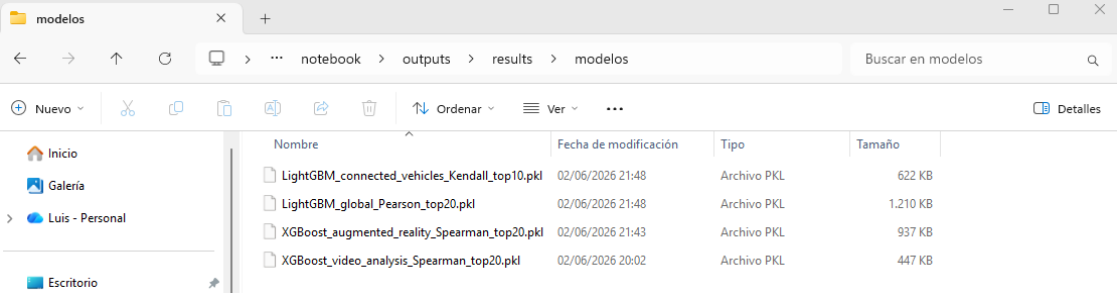
Para **augmented_reality**, el R^2 de 0.756 indica que el modelo explica aproximadamente el 75% de la variabilidad de la variable objetivo. En este trabajo, **augmented_reality** se considera la variable objetivo más difícil desde el punto de vista predictivo, ya que presenta el menor R^2 entre los modelos finales evaluados. Esta mayor dificultad puede atribuirse a la naturaleza más irregular y menos predecible de las peticiones de realidad aumentada. Aun así, el predictor sigue siendo útil para anticipar la tendencia general de la QoS, aunque con menor precisión en los instantes de mayor variabilidad.

El caso de **video_analysis** presenta un comportamiento singular. Su MAE es excepcionalmente bajo, con un valor de 0.0004, debido a que la distribución de la variable objetivo está fuertemente concentrada en valores próximos a cero. Aunque su R^2 es superior al de **augmented_reality**, las métricas deben interpretarse con cautela, ya que el modelo captura bien los periodos de baja actividad, pero puede tener más

dificultad en los instantes con valores altos aislados. Esta limitación está relacionada con la propia distribución de la variable objetivo y aparece también en el resto de modelos evaluados.

Los resultados confirman que las variables temporales incluidas en el conjunto de datos, como valores históricos, medias móviles y diferencias temporales, permiten a los modelos tabulares capturar dependencias relevantes sin recurrir a arquitecturas secuenciales. En **global** y **connected_vehicles** el mejor modelo final es LightGBM, mientras que en **augmented_reality** y **video_analysis** lo es XGBoost. Por tanto, los modelos basados en árboles con *boosting* resultan más adecuados para este problema, al ofrecer mayor precisión y menor tiempo de inferencia que CNN y LSTM.

Como resultado final del proceso experimental, se obtienen cuatro modelos serializados en formato **.pkl**, uno por cada variable objetivo, listos para su integración como servicio predictivo en MintEDGE. La **Figura 5.37** muestra los modelos finales generados y guardados.



The screenshot shows a file explorer window titled 'modelos'. The address bar indicates the path: notebook > outputs > results > modelos. A search bar on the right says 'Buscar en modelos'. The left sidebar shows the navigation pane with 'Inicio', 'Galería', 'Luis - Personal', and 'Escritorio'. The main pane displays a table of files:

Nombre	Fecha de modificación	Tipo	Tamaño
LightGBM_connected_vehicles_Kendall_top10.pkl	02/06/2026 21:48	Archivo PKL	622 KB
LightGBM_global_Pearson_top20.pkl	02/06/2026 21:48	Archivo PKL	1,210 KB
XGBoost_augmented_reality_Spearman_top20.pkl	02/06/2026 21:43	Archivo PKL	937 KB
XGBoost_video_analysis_Spearman_top20.pkl	02/06/2026 20:02	Archivo PKL	447 KB

Figura 5.37. Modelos tabulares finales generados.

Capítulo 6

Conclusiones y trabajo futuro

6.1 Conclusiones

El objetivo principal de este Trabajo de Fin de Grado era el diseño y la evaluación de un servicio inteligente capaz de predecir indicadores de QoS, principalmente la tasa de peticiones satisfechas, en sistemas de *Edge Computing* en el contexto de redes 5G. Este objetivo se ha alcanzado de forma satisfactoria mediante el desarrollo de los objetivos secundarios planteados.

Se ha contextualizado el problema en el marco de las redes 5G, el *Edge Computing* y las técnicas de ML, se ha analizado en profundidad el simulador MintEDGE identificando el punto exacto de integración del servicio predictivo, y se ha desarrollado dicho servicio en su totalidad: 16 simulaciones sobre 6 escenarios geográficos europeos, construcción de un conjunto de datos homogéneo de $1,16 \times 10^6$ registros, selección de variables y entrenamiento de seis arquitecturas de modelos. Los modelos tabulares basados en *gradient boosting* han resultado óptimos para las cuatro variables objetivo: LightGBM para `global` y `connected_vehicles`, y XGBoost para `augmented_reality` y `video_analysis`, con tiempos de inferencia inferiores a $1 \mu s$ por muestra y R^2 de 0.994 para `global` y `connected_vehicles`, 0.756 para `augmented_reality` y 0.829 para `video_analysis` sobre el conjunto de prueba, sin evidencia de sobreajuste.

Las principales limitaciones identificadas son las siguientes:

- En primer lugar, los modelos se han entrenado y evaluado sobre datos simulados. Aunque MintEDGE permite generar escenarios controlados y reproducibles, estos datos no recogen toda la variabilidad de una red real, donde pueden aparecer fluctuaciones de tráfico más irregulares, ruido en las medidas, fallos puntuales de infraestructura o cambios bruscos en la demanda de los servicios.
- En segundo lugar, la precisión no es uniforme entre todas las variables objetivo. Las variables objetivo **global** y **connected_vehicles** presentan un rendimiento muy alto, mientras que **augmented_reality** mantiene mayor margen de mejora debido a la naturaleza más irregular de sus peticiones. En el caso de **video_analysis**, el modelo captura correctamente los periodos de baja actividad, pero tiene más dificultad para estimar la magnitud exacta de los picos esporádicos.
- Por último, el servicio predictivo se ha centrado en la tasa de peticiones satisfechas como indicador principal de QoS. Este KPI permite evaluar el cumplimiento de las peticiones aceptadas, pero no recoge por completo otros aspectos relevantes en un sistema de *Edge Computing*, como el retardo, el consumo energético, el uso de recursos o el estado de la infraestructura *edge*.

6.2 Trabajo futuro

A partir de las conclusiones extraídas y las limitaciones identificadas, es posible definir una serie de trabajos y desarrollos futuros en las siguientes líneas de investigación.

1. Integración del servicio predictivo en MintEDGE

La principal línea de trabajo futuro consiste en integrar los modelos entrenados en el simulador MintEDGE. Esta integración implicaría desarrollar un módulo capaz de cargar los modelos serializados en formato .pkl y generar predicciones de la tasa de peticiones satisfechas a partir de los datos disponibles en cada instante de simulación. De este modo, el orquestador podría utilizar información anticipada sobre la QoS antes de tomar decisiones de asignación de recursos.

2. Mejora de la predicción en las variables objetivo más complejas

Una segunda línea de trabajo futuro consiste en mejorar el rendimiento en las variables objetivo que han presentado mayor dificultad. En **augmented_reality**, podrían explorarse arquitecturas temporales más recientes, como los *Temporal Fusion*

Transformers, o incorporar variables adicionales procedentes de la capa de movilidad del simulador. En **video_analysis**, podrían evaluarse técnicas de sobremuestreo o funciones de pérdida ponderadas que otorguen mayor peso a los instantes de alta actividad, con el objetivo de mejorar la predicción de los picos esporádicos.

3. Ampliación de los indicadores de QoS

Otra ampliación natural sería extender el servicio predictivo a otros indicadores de QoS además de la tasa de peticiones satisfechas. En particular, podrían incorporarse métricas como el retardo, el consumo energético, el uso de recursos o el estado de la infraestructura *edge*, de forma que el orquestador disponga de una visión más completa del estado futuro del sistema.

4. Adaptación del servicio predictivo a entornos reales

Por último, otra línea de trabajo futuro consistiría en estudiar la adaptación del servicio predictivo a entornos reales. Para ello, sería necesario identificar qué variables equivalentes a las utilizadas en MintEDGE podrían obtenerse en una red operativa y adaptar el proceso de ingeniería de características a las métricas disponibles en dicho entorno. Esta adaptación permitiría analizar hasta qué punto el enfoque desarrollado en simulación puede trasladarse a escenarios reales de *Edge Computing* y redes 5G.

Bibliografía

- [1] 3GPP, «5G System Overview,» 3GPP, 8 Agosto 2022. [En línea]. Available: <https://www.3gpp.org/technologies/5g-system-overview>. [Último acceso: 26 Febrero 2026].
- [2] Wikipedia contributors, «5G,» Wikipedia, The Free Encyclopedia, [En línea]. Available: <https://en.wikipedia.org/wiki/5G>. [Último acceso: 9 Febrero 2026].
- [3] IBM, «¿Qué es el edge computing?,» IBM, 31 Marzo 2026. [En línea]. Available: <https://www.ibm.com/es-es/think/topics/edge-computing>. [Último acceso: 4 Abril 2026].
- [4] Red Hat, «¿Qué es el edge computing? Su uso con centros de datos y clouds,» Red Hat, 31 Marzo 2021. [En línea]. Available: <https://www.redhat.com/es/topics/edge-computing/what-is-edge-computing>. [Último acceso: 28 Febrero 2026].
- [5] Red Hat, «What is multi-access edge computing (MEC)?,» Red Hat, 22 Julio 2022. [En línea]. Available: <https://www.redhat.com/en/topics/edge-computing/what-is-multi-access-edge-computing>. [Último acceso: 28 Febrero 2026].
- [6] ETSI, «Multi-access Edge Computing,» European Telecommunications Standards Institute, [En línea]. Available: <https://www.etsi.org/technologies/multi-access-edge-computing>. [Último acceso: 1 Marzo 2026].
- [7] M. Goodwin, «¿Qué es la latencia?,» IBM, [En línea]. Available: <https://www.ibm.com/es-es/think/topics/latency>. [Último acceso: 1 Marzo 2026].

- [8] M. Mosahebfard, C. Torres-Pérez, E. Carmona-Cejudo, A. Cárdenas Córdova, A. Pino Martínez, B. J. S. Camargo, E. Coronado y M. S. Siddiqui, «Intelligent Management at the Edge,» de *Shaping the Future of IoT with Edge Intelligence: How Edge Computing Enables the Next Generation of IoT Applications*, Abingdon, UK, River Publishers, 2024.
- [9] Ericsson, «Standalone and non-standalone 5G NR: two 5G tracks,» Ericsson, 24 Abril 2023. [En línea]. Available: <https://www.ericsson.com/en/blog/2023/4/standalone-and-non-standalone-5g-nr-two-5g-tracks>. [Último acceso: 10 Febrero 2026].
- [10] Nokia, «5G Non-Standalone to 5G Standalone – made real: New opportunities for Communications Service Providers,» Nokia, Espoo, 2020.
- [11] R. Mihai, R. Craciunescu, A. Martian, F. Y. Li y M.-C. Vochin, «Open-source enabled beyond 5G private mobile networks: From concept to prototype,» de *25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*, Herning, Denmark, 2022.
- [12] E. d. E. e. C. y. Tecnología, «SDN NFV: ¿en qué se diferencian?,» Universidad Internacional de Valencia, 6 Noviembre 2023. [En línea]. Available: <https://www.universidadviu.com/es/actualidad/nuestros-expertos/sdn-nfv-en-que-se-diferencian>. [Último acceso: 28 Febrero 2026].
- [13] M. Goodwin, «¿Qué es Network Functions Virtualization (NFV)?,» IBM, [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/network-functions-virtualization>. [Último acceso: 27 Febrero 2026].
- [14] F. Adelantado, «5G: La softwarización de la red móvil,» Universitat Oberta de Catalunya, 3 Junio 2019. [En línea]. Available: <https://blogs.uoc.edu/informatica/es/softwarizacion-5g-network-slices/>. [Último acceso: 27 Febrero 2026].
- [15] INCIBE, «Edge Computing: el futuro de la computación,» INCIBE-CERT, 11 Enero 2023. [En línea]. Available: <https://www.incibe.es/incibe-cert/blog/edge-computing-el-futuro-computacion>. [Último acceso: 3 Junio 2026].
- [16] ETSI, «ETSI Multi-access Edge Computing group publishes white paper on role for 5G,» European Telecommunications Standards Institute, 26 Junio 2018. [En línea]. Available: <https://www.etsi.org/newsroom/press-releases/1314-2018-06-press-etsi->

- multi-access-edge-computing-group-publishes-white-paper-on-role-for-5g/. [Último acceso: 1 Marzo 2026].
- [17] APD, «¿Qué son y qué tipos de KPIs existen?,» APD, 10 Diciembre 2024. [En línea]. Available: <https://www.apd.es/tipos-de-kpis/>. [Último acceso: 1 Marzo 2026].
- [18] J. Martins, «What Are KPIs? Defining Key Performance Indicators,» Asana, 16 Agosto 2025. [En línea]. Available: <https://asana.com/es/resources/key-performance-indicator-kpi>. [Último acceso: 1 Marzo 2026].
- [19] T. Ilan, «Private (5G) Networks: The Details are in the Data,» RADCOM, 20 Junio 2024. [En línea]. Available: <https://radcom.com/private-5g-networks-the-details-are-in-the-data/>. [Último acceso: 1 Marzo 2026].
- [20] E. Lykakis, I. O. Vardiambasis y E. Kokkinos, «Data Traffic Prediction for 5G and Beyond: Emerging Trends, Challenges, and Future Directions: A Scoping Review,» *Electronics*, vol. 14, nº 23, p. 4611, 2025.
- [21] Cloudflare, «What Is Latency? | How to Fix Latency,» Cloudflare, [En línea]. Available: <https://www.cloudflare.com/es-es/learning/performance/glossary/what-is-latency/>. [Último acceso: 2 Marzo 2026].
- [22] Zenarmor, «Understanding Network Congestion: Causes and Fixes,» Zenarmor, 30 Abril 2024. [En línea]. Available: <https://www.zenarmor.com/docs/network-basics/what-is-network-congestion>. [Último acceso: 2 Marzo 2026].
- [23] IBM, «Planning for quality of service,» IBM Documentation, 7 Octubre 2025. [En línea]. Available: <https://www.ibm.com/docs/en/i/7.4.0?topic=service-planning-quality>. [Último acceso: 1 Marzo 2026].
- [24] Ericsson, «Build an energy-efficient core network,» Ericsson, [En línea]. Available: <https://www.ericsson.com/en/core-network/energy-efficiency>. [Último acceso: 2 Marzo 2026].
- [25] Wikipedia contributors, «Network simulation,» Wikipedia, The Free Encyclopedia, 14 Octubre 2025. [En línea]. Available: https://en.wikipedia.org/wiki/Network_simulation. [Último acceso: 1 Marzo 2026].

- [26] B. Gómez, S. Bayhan, E. Coronado, J. Villalón y A. Garrido, «MintEDGE: Multi-tier Simulator for eNergy-aware sTrategies in Edge Computing,» de *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, Madrid, 2023.
- [27] B. Gómez, S. Bayhan, E. Coronado, J. Villalón y A. Garrido, «MintEDGE,» GitHub, [En línea]. Available: <https://github.com/blasf1/MintEDGE>. [Último acceso: 3 Junio 2026].
- [28] ns-3 Consortium, «ns-3 Network Simulator,» ns-3 Consortium, [En línea]. Available: <https://www.nsnam.org/>. [Último acceso: 2 Marzo 2026].
- [29] OpenSim Ltd., «OMNeT++ Discrete Event Simulator,» OpenSim Ltd., [En línea]. Available: <https://omnetpp.org/>. [Último acceso: 2 Marzo 2026].
- [30] G. Nardini, D. Sabella, G. Stea, P. Thakkar y A. Virdis, «Simu5G – An OMNeT++ Library for End-to-End Performance Evaluation of 5G Networks,» *IEEE Access*, vol. 8, pp. 181176-181191, 2020.
- [31] C. Sonmez, A. Ozgovde y C. Ersoy, «EdgeCloudSim: An environment for performance evaluation of Edge Computing systems,» *Transactions on Emerging Telecommunications Technologies*, vol. 29, nº 11, p. e3493, 2018.
- [32] Qlik, «Predictive Modeling,» Qlik, [En línea]. Available: <https://www.qlik.com/us/predictive-analytics/predictive-modeling>. [Último acceso: 11 Marzo 2026].
- [33] Google Cloud, «What is predictive analytics?,» Google Cloud, [En línea]. Available: <https://cloud.google.com/learn/what-is-predictive-analytics>. [Último acceso: 11 Marzo 2026].
- [34] P. Lara-Benítez, M. Carranza-García y J. C. Riquelme, «An Experimental Review on Deep Learning Architectures for Time Series Forecasting,» *International Journal of Neural Systems*, vol. 31, nº 3, p. 2150011, 2021.
- [35] D. Bergmann, «¿Qué es el machine learning (ML) o aprendizaje automático?,» IBM, [En línea]. Available: <https://www.ibm.com/es-es/think/topics/machine-learning>. [Último acceso: 11 Marzo 2026].
- [36] IBM, «Modelos estadísticos,» IBM Documentation, 17 Agosto 2021. [En línea]. Available: <https://www.ibm.com/docs/es/spss-modeler/saas?topic=nodes-statistical-models>. [Último acceso: 11 Marzo 2026].

-
- [37] Wikipedia contributors, «Modelo probabilístico,» Wikipedia, la enciclopedia libre, 19 Abril 2026. [En línea]. Available: https://es.wikipedia.org/wiki/Modelo_probabil%C3%ADstico. [Último acceso: 11 Marzo 2026].
- [38] Amazon Web Services, «¿Qué es el machine learning? - Explicación de la tecnología ML,» Amazon Web Services, [En línea]. Available: <https://aws.amazon.com/es/what-is/machine-learning/>. [Último acceso: 11 Marzo 2026].
- [39] scikit-learn developers, «1.10. Decision Trees,» scikit-learn Documentation, [En línea]. Available: <https://scikit-learn.org/stable/modules/tree.html>. [Último acceso: 27 Mayo 2026].
- [40] scikit-learn developers, «RandomForestRegressor,» scikit-learn Documentation, [En línea]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>. [Último acceso: 27 Mayo 2026].
- [41] T. Chen y C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» de *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, 2016.
- [42] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye y T.-Y. Liu, «LightGBM: A Highly Efficient Gradient Boosting Decision Tree,» de *Advances in Neural Information Processing Systems 30*, 2017.
- [43] IBM, «¿Qué es el deep learning?,» IBM, [En línea]. Available: <https://www.ibm.com/es-es/think/topics/deep-learning>. [Último acceso: 11 Marzo 2026].
- [44] S. Hochreiter y J. Schmidhuber, «Long Short-Term Memory,» *Neural Computation*, vol. 9, nº 8, pp. 1735-1780, 1997.
- [45] ScienceDirect, «Long Short-Term Memory,» Elsevier / ScienceDirect Topics, [En línea]. Available: <https://www.sciencedirect.com/topics/chemical-engineering/long-short-term-memory>. [Último acceso: 27 Mayo 2026].
- [46] ResearchGate, «Long Short-Term Memory (LSTM) cell architecture with key components labelled,» ResearchGate, Agosto 2024. [En línea]. Available:

https://www.researchgate.net/figure/Long-Short-Term-Memory-LSTM-cell-architecture-with-key-components-labelled-including_fig6_382796561. [Último acceso: 28 Mayo 2026].

[47] MathWorks, «What Is a Convolutional Neural Network?,» MathWorks, [En línea]. Available: <https://www.mathworks.com/discovery/convolutional-neural-network.html>. [Último acceso: 28 Mayo 2026].

[48] I. Belcic y C. Stryker, «What is feature selection?,» IBM, [En línea]. Available: <https://www.ibm.com/think/topics/feature-selection>. [Último acceso: 28 Mayo 2026].

[49] ScienceDirect, «Pearson Correlation Coefficient,» Elsevier / ScienceDirect Topics, [En línea]. Available: <https://www.sciencedirect.com/topics/computer-science/pearson-correlation-coefficient>. [Último acceso: 28 Mayo 2026].

[50] S. Ratnasingam y J. Muñoz-Lopez, «Distance Correlation-Based Feature Selection in Random Forest,» *Entropy*, vol. 25, nº 9, p. 1250, 23 Agosto 2023.

[51] Wikipedia contributors, «Spearman's rank correlation coefficient,» Wikipedia, The Free Encyclopedia, 10 Mayo 2026. [En línea]. Available: https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient. [Último acceso: 28 Mayo 2026].

[52] D. Wei, «Essential Math for Machine Learning: Spearman's Rank Correlation,» Medium, 13 Junio 2024. [En línea]. Available: <https://medium.com/@weidagang/essential-math-for-machine-learning-spearman-s-rank-correlation-80e023045c96>. [Último acceso: 28 Mayo 2026].

[53] Emergent Mind, «Kendall's Tau: Nonparametric Rank Correlation,» Emergent Mind, 5 Febrero 2026. [En línea]. Available: <https://www.emergentmind.com/topics/kendall-rank-correlation-coefficient-kendall-s-tau>. [Último acceso: 29 Mayo 2026].

[54] scikit-learn developers, «f_regression,» scikit-learn Documentation, [En línea]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.f_regression.html. [Último acceso: 29 Mayo 2026].

[55] M. A. Hall, «Correlation-based Feature Selection for Machine Learning,» University of Waikato, Hamilton, New Zealand, 1999.

-
- [56] Wikipedia contributors, «Raíz del error cuadrático medio,» Wikipedia, la enciclopedia libre, 7 Marzo 2026. [En línea]. Available: https://es.wikipedia.org/wiki/Raíz_del_error_cuadrático_medio. [Último acceso: 30 Mayo 2026].
- [57] Wikipedia contributors, «Error absoluto medio,» Wikipedia, la enciclopedia libre, 1 Abril 2025. [En línea]. Available: https://es.wikipedia.org/wiki/Error_absoluto_medio. [Último acceso: 30 Mayo 2026].
- [58] DataCamp, «Error absoluto medio explicado: Medición de la precisión en modelos de regresión,» DataCamp, 8 Agosto 2025. [En línea]. Available: <https://www.datacamp.com/es/tutorial/mean-absolute-error>. [Último acceso: 30 Mayo 2026].
- [59] scikit-learn developers, «r2_score,» scikit-learn Documentation, [En línea]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html. [Último acceso: 30 Mayo 2026].
- [60] SimPy Developers, «SimPy: Discrete event simulation for Python,» SimPy Documentation, [En línea]. Available: <https://simpy.readthedocs.io/>. [Último acceso: 4 Febrero 2026].
- [61] DLR, «SUMO: Simulation of Urban MObility,» Eclipse Foundation, [En línea]. Available: <https://www.eclipse.org/sumo/>. [Último acceso: 12 Marzo 2026].
- [62] G. Boeing, «OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks,» *Computers, Environment and Urban Systems*, vol. 65, pp. 126-139, 2017.
- [63] L. M. Jareño, «edge-network-kpi-predictor,» GitHub, 15 Junio 2026. [En línea]. Available: <https://github.com/luismelero-7/edge-network-kpi-predictor>.
- [64] scikit-learn developers, «1.11. Ensemble methods,» scikit-learn Documentation, [En línea]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>. [Último acceso: 30 Mayo 2026].

- [65] C. Stryker, «What is a recurrent neural network?,» IBM, [En línea]. Available: <https://www.ibm.com/think/topics/recurrent-neural-networks>. [Último acceso: 28 Mayo 2026].
- [66] J. Murel y E. Kavlakoglu, «What is regularization?,» IBM, [En línea]. Available: <https://www.ibm.com/mx-es/think/topics/regularization>. [Último acceso: 28 Mayo 2026].
- [67] TensorFlow, «tf.keras.callbacks.EarlyStopping,» TensorFlow Documentation, [En línea]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping. [Último acceso: 28 Mayo 2026].
- [68] scikit-learn developers, «3.1. Cross-validation: evaluating estimator performance,» scikit-learn Documentation, [En línea]. Available: https://scikit-learn.org/stable/modules/cross_validation.html. [Último acceso: 29 Mayo 2026].
- [69] M. Steadman, «Gradient Boosted Regression Trees,» DataRobot, 4 Abril 2014. [En línea]. Available: <https://www.datarobot.com/blog/gradient-boosted-regression-trees/>. [Último acceso: 31 Mayo 2026].
- [70] W. Li, D. Cook, E. Tanaka y S. VanderPlas, «A Plot is Worth a Thousand Tests: Assessing Residual Diagnostics with the Lineup Protocol,» arXiv, 2024.
- [71] NIST/SEMATECH, «Scatter Plot,» NIST/SEMATECH e-Handbook of Statistical Methods, [En línea]. Available: <https://itl.nist.gov/div898/handbook/eda/section3/scatterp.htm>. [Último acceso: 31 Mayo 2026].
- [72] S. Bai, J. Z. Kolter y V. Koltun, «An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,» arXiv, 2018.
- [73] S. Gopali, F. Abri, S. Siarni-Namini y A. S. Namin, «A Comparative Study of Detecting Anomalies in Time Series Data Using LSTM and TCN Models,» arXiv, 2021.
- [74] L. Grinsztajn, E. Oyallon y G. Varoquaux, «Why do tree-based models still outperform deep learning on typical tabular data?,» de *Advances in Neural Information Processing Systems* 35, 2022.

-
- [75] R. Schwartz-Ziv y A. Armon, «Tabular data: Deep learning is not all you need,» *Information Fusion*, vol. 81, pp. 84-90, 2022.
- [76] D. Barney, «What are Network KPIs, Why Should You Care? 16 Metrics/KPIs to Chase,» WhatsUp Gold, 7 Marzo 2024. [En línea]. Available: <https://www.whatsupgold.com/blog/what-are-network-kpis-why-should-you-care-16-metrics-kpis-to-chase>. [Último acceso: 1 Marzo 2026].
- [77] Amazon Web Services, «What is Latency? - Network Latency Explained,» Amazon Web Services, [En línea]. Available: <https://aws.amazon.com/what-is/latency/>. [Último acceso: 2 Marzo 2026].
- [78] IBM, «¿Qué es el análisis predictivo?,» IBM, [En línea]. Available: <https://www.ibm.com/es-es/think/topics/predictive-analytics>. [Último acceso: 11 Marzo 2026].
- [79] Master en Data Science UCM, «¿Qué es Machine Learning?,» Universidad Complutense de Madrid, [En línea]. Available: <https://www.masterdatascienceucm.com/que-es-machine-learning/>. [Último acceso: 2 Junio 2026].
- [80] Red Hat, «¿Qué es el deep learning?,» Red Hat, 10 Febrero 2026. [En línea]. Available: <https://www.redhat.com/es/topics/ai/what-is-deep-learning>. [Último acceso: 11 Marzo 2026].
- [81] Wikipedia contributors, «Feature selection,» Wikipedia, The Free Encyclopedia, 28 Abril 2026. [En línea]. Available: https://en.wikipedia.org/wiki/Feature_selection. [Último acceso: 28 Mayo 2026].
- [82] B. Clark y F. Lee, «What is gradient boosting?,» IBM, [En línea]. Available: <https://www.ibm.com/think/topics/gradient-boosting>. [Último acceso: 31 Mayo 2026].
- [83] LightGBM Developers, «lightgbm.LGBMRegressor,» LightGBM Documentation, [En línea]. Available: <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMRegressor.html>. [Último acceso: 31 Mayo 2026].

